How the CoCoA Emulator Code Works: A Tensor-by-Tensor Guide from Generated Cosmologies to Inference

V. Miranda
Code documentation manuscript
(Dated: Working draft)

This manuscript follows one cosmological sample through the CoCoA emulator program. It begins with generated parameter tables and data-vector dumps, then explains staging, coordinate transforms, neural designs, loss construction, device-aware batching, training, family-specific physics, artifact reconstruction, Cobaya adapters, and executable acceptance gates. The emphasis is operational: every tensor is assigned a shape, dtype, device, and physical meaning, and every non-obvious Python or PyTorch mechanism is defined where it first changes the calculation.

I. HOW TO READ THE PROGRAM

The emulator is not one neural-network call. It is a sequence of ownership boundaries. A generated row begins as numbers in files, becomes a NumPy array, becomes a PyTorch tensor on a selected device, becomes a network prediction in transformed coordinates, returns to physical units, and is finally published in a saved artifact. The value is scientifically useful only if its row identity, column names, units, axis coordinates, dtype, and transformation law survive every boundary.

This manuscript follows the executable order. Each section answers the same five questions:

1. What Python objects exist before the operation?
2. Which function or method owns the operation?
3. What new objects exist afterward?
4. Which operation copied data, returned a view, or deferred work?
5. Which numerical identity proves that the operation preserved its physical meaning?

A. The notation used throughout

Uppercase letters denote a collection of rows. Lowercase letters denote one row. A leading dimension B is a minibatch size. The remaining dimensions describe one sample.

N is the number of rows in a dump, P the number of raw input parameters, P_{enc} the input width after the parameter geometry, D the full physical output width, and K the number of outputs the network predicts. For a plain model, T_{tmpl} is absent. A factored intrinsic-alignment model adds that template axis.

B. The Python object vocabulary

Several Python words have precise meanings in the code:

Array.: A NumPy object containing numerical values on the CPU. An `ndarray` is normally resident in host memory. An `np.memmap` presents a file as an array and reads requested pages on demand.

Tensor.: A PyTorch numerical object. It has a shape, dtype, and device. A tensor can also carry the information PyTorch needs to compute gradients.

Module.: A subclass of `torch.nn.Module`. A module owns parameters, registered buffers, child modules, and a `forward` method.

Parameter.: A tensor registered for optimization. When `requires_grad=True`, backward differentiation accumulates its derivative in `parameter.grad`.

Buffer.: A tensor that belongs to a module and moves with `module.to(device)` but is not optimized. Fixed basis-change matrices are buffers.

Callable.: Any object invoked with parentheses. A function, a class, and an `nn.Module` instance are all callable, but the parentheses do different work: a class call constructs an instance; a module call runs its forward path.

Iterator.: An object that produces one item at a time and remembers its current position. A `for` loop consumes it. It is not necessarily a materialized list.

Closure.: A function that retains values from the surrounding scope where it was created. Loader closures remember where their data live.

View.: A new array or tensor description that shares underlying storage with another object. Mutating a writable view can change the original.

Copy.: A new storage allocation containing duplicated values. Later mutation does not change the source storage.

Symbol	Typical shape	Physical meaning	Program owner
C	(N, P)	Raw cosmological parameters, one cosmology per row	`load_source` and the source dictionary
V	(N, D)	Raw physical data vectors	`load_source`
X	(B, P_{enc})	Encoded model inputs	Parameter geometry and `load_C`
T	(B, K) or $(B, T_{\mathrm{tmpl}}, K)$	Encoded training targets or template stack	Output geometry, loss wrapper, and `load_dv`
$\widehat{T}$	Same target shape	Model prediction in network coordinates	`nn.Module.forward`
r	(B, K)	Prediction-minus-truth residual in physical kept coordinates	Loss wrapper
$\Delta\chi^2$	$(B,)$	One covariance-weighted error per cosmology	Loss wrapper and evaluation code

C. Python mechanics that the later code uses

Dictionary or mapping.: A set of key–value associations. Looking up `cfg[data]` requires the key and raises if it is absent; `cfg.get(data)` returns `None` when absent unless a default is supplied. That hidden default is part of control flow and must not stand in for validation.

List and tuple.: Both are ordered sequences. A list is mutable: an item can be replaced or appended. A tuple is immutable after construction and is commonly used for a fixed shape or fixed return record. Neither is lazy; all contained object references already exist.

`None`.: Python's one "no object" sentinel. It is distinct from zero, an empty list, an empty mapping, and `False`. Code tests it with `is None` when those other values have their own meanings.

Truthiness.: Conditions such as `if value:` convert an object to a boolean. Zero and empty containers are false; nonempty strings are true, including the string `"false"`. Public configuration therefore validates an actual boolean rather than relying on truthiness.

Positional and keyword arguments.: In `f(x, scale=2)`, `x` fills the first parameter by position and `2` fills the parameter named `scale`. Keyword-heavy construction is used where several values share a type or shape, because order alone would not explain their roles.

Generator.: A function containing `yield` returns a lazy iterator. It pauses at each yielded item and resumes on the next request; it does not construct the full list first. Converting it with `list` deliberately materializes every remaining item in memory.

Context manager.: A `with` statement calls an object's entry and exit protocol. File and no-gradient contexts use it so cleanup or gradient restoration occurs even when the body raises.

Class method.: An alternative constructor marked `@classmethod` receives the selected class as `cls`. Returning `cls(...)` preserves subclass behavior. The ordinary `__init__` stores already prepared fields; named constructors such as `from_state` own loading and validation branches.

Scope.: A local name belongs to one function call. `nonlocal` lets an inner closure rebind a name in its enclosing function. A module global outlives all calls. Data arrays, devices, fitted geometries, and configuration state are passed explicitly rather than read from silent globals, so a function's signature states its real inputs.

Mutable defaults deserve a separate rule. Python creates a default object once when the function is defined, not once per call. Therefore `def f(opts={})` shares one dictionary across calls. The safe form is `def f(opts=None)` followed by construction of a new dictionary when `opts is None`.

D. A running numerical example

The small example below will reappear at each boundary. It has four cosmologies, three parameters, and five data-vector entries:

$$C = \begin{bmatrix} 0.29 & 67 & 0.048 \\ 0.31 & 70 & 0.050 \\ 0.33 & 73 & 0.052 \\ 0.30 & 69 & 0.049 \end{bmatrix}, \qquad V = \begin{bmatrix} 10 & 20 & 30 & 40 & 50 \\ 11 & 22 & 33 & 44 & 55 \\ 12 & 24 & 36 & 48 & 60 \\ 10.5 & 21 & 31.5 & 42 & 52.5 \end{bmatrix}.$$

The parameter names are

$$[\texttt{omegam}, \texttt{H0}, \texttt{omegab}].$$

Row 1 means that the cosmology $(0.31, 70, 0.050)$ produced the physical data vector $(11, 22, 33, 44, 55)$. The row number is part of the meaning. If staging pairs that parameter row with row 2 of V, every later tensor can be finite, every training loss can decrease, and the learned function is still wrong.

Assume the likelihood keeps output columns $[0, 2, 4]$. Then $D = 5$ and $K = 3$. The network never predicts columns 1 and 3, but the geometry retains their global positions so it can scatter a kept residual back into a full vector.

E. The complete ownership chain

YAML and generated files
↓ `resolve_cocoa_config`, `EmulatorExperiment.from_config`
validated run description
↓ `stage_train`, `stage_val`
source dictionaries containing NumPy arrays and row coordinates
↓ `build_geometry`
reversible parameter/output transforms and the scientific loss
↓ `build_loaders`, `run_emulator`
device tensors, model, optimizer, scheduler, and training histories
↓ `save_emulator`
weight file plus HDF5 recipe/geometry record
↓ `rebuild_emulator`, `EmulatorPredictor`
physical prediction from a named cosmology
↓ Cobaya adapter
likelihood-facing theory product on a validated coordinate axis.

Each arrow has one owner. If two modules independently reimplement the same physical transformation, they can drift while their local tests remain green. The design therefore reuses geometry and loss decoding at inference instead of writing a second prediction formula.

F. How this manuscript marks alpha-stage gaps

The repository identifies itself as alpha software. An unqualified present-tense sentence in this manuscript describes the current executed path. A paragraph labeled *Current gap* describes behavior still present in the code. A paragraph labeled *Required closure* states the audited contract that must land before the gap is accepted as closed. Required closures are included because they explain why a gate exists, but they are not represented as current features.

The executable gate log at a named commit is the final status authority. This working manuscript can lag a same-day implementation. When code lands, the corresponding current-gap paragraph is deleted only after its gate executes; the didactic explanation of the resulting behavior remains.

II. STARTING A RUN: COMMANDS, PATHS, AND CONFIGURATION

The numerical story starts with a shell command, not with a tensor. This section follows that command until the moment the program is ready to open the first data file. The distinction matters because configuration errors should be rejected before a large array is staged or a GPU allocation is made.

A. The four kinds of driver

A *driver* is a small Python program that chooses how many complete training runs to perform. It does not contain a second implementation of the emulator. Every driver eventually asks `EmulatorExperiment` to perform the same staging, geometry construction, model construction, and training operations.

For example, the ordinary cosmic-shear command has the form

```
python cosmic_shear_train_emulator.py \
  --root projects/lsst_y1/ \
  --fileroot emulators/training_scripts/ \
  --yaml cosmic_shear_train_emulator.yaml \
  --diagnostic diagnostic
```

Driver kind	Unit of work	Question answered
Train	One model from one resolved YAML file	What artifact does this exact configuration produce?
Training-size sweep	One independent model for each requested row count	Is accuracy still improving as more simulations are supplied?
One-knob sweep	One model for each value of one named configuration leaf	What is the isolated effect of that control?
Hyperparameter search	A sequence of proposed configurations recorded in a study	Which configuration minimizes the declared validation objective?
Activation bake-off	One training-size curve per activation family	Does a nonlinearity learn faster across the full data-volume range?

The continuation character at the end of the first three lines tells the shell that the command continues on the next line. It is shell syntax; it is not passed to Python. The optional `-diagnostic` value asks the driver to create a diagnostic report after training. It does not change the training objective.

The family-specific programs are deliberately thin. Scalar, cosmic microwave background (CMB), background, and matter-power drivers select the appropriate family contract and then reuse the same experiment engine. This is important to the reader: a behavior found in the generic experiment or training module normally affects all five families, whereas a behavior in a family geometry affects only that physical output.

B. Three path arguments form one file location

The command uses three different path concepts.

1. `ROOTDIR` is an environment variable exported by Cocoa. An environment variable is a string owned by the shell and inherited by the Python process.
2. `-root` is the project directory relative to `ROOTDIR`. Its `chains` child owns generated data and trained artifacts.
3. `-fileroot` is the directory, under the project, that owns the training YAML and driver reports.
4. `-yaml` is a bare file name inside `-fileroot`; it is not interpreted relative to the current terminal directory.

Thus the configuration file in the example is conceptually

`ROOTDIR/-root/-fileroot/-yaml`.

Training arrays named by the YAML instead resolve under `ROOTDIR/-root/chains`. External likelihood files use a different declared root under Cocoa's external-data directory. A path resolver performs these joins. The numerical code receives resolved absolute paths rather than guessing from the process's current working directory.

This separation prevents a common reuse bug. A relative path such as `chains/train.npy` means different files after `os.chdir`. A resolved path has one meaning throughout the run and can be written into the artifact provenance.

C. The YAML is a construction recipe

YAML is a text representation of nested mappings and sequences. A mapping is a collection of named key–value pairs; a sequence is an ordered list. At the top level, an ordinary training file contains a `data` mapping and a `train_args` mapping:

```
data:
  train_params: train.1.txt
  train_dv: train.npy
  train_covmat: train.covmat
  val_params: validation.1.txt
  val_dv: validation.npy
  n_train: 25000
  n_val: 5000
  split_seed: 0
  ram_frac: 0.7

train_args:
  nepochs: 1600
  bs: 256
  loss:
    mode: sqrt
  model:
    name: resmlp
    mlp:
      width: 128
      n_blocks: 4
```

Indentation creates ownership. For example, `width` belongs to `train_args.model.mlp`; it is not a top-level setting. A dotted name is documentation shorthand for walking nested mappings. It is not the literal key stored in the YAML parser's dictionary.

The `data` block answers where the rows come from, which physical family they represent, how many rows to keep, and how much host memory staging may use. The `train_args` block answers which objective, architecture, optimizer controls, schedules, and run length to construct. Keeping these questions separate allows the same model recipe to be evaluated on different data volumes

without moving file-system facts into a neural-network class.

D. Raw configuration and resolved configuration

The dictionary returned by the YAML parser is the *raw configuration*. It is not yet an executed recipe. Resolution performs explicit transformations:

1. verify that allowed keys are spelled correctly and required keys are present;
2. reject values with the wrong type or physical domain;
3. replace a search range by its declared default in a normal training run;
4. select class objects for the requested geometry, model, activation, loss, optimizer, and scheduler;
5. compute dependent facts such as input width, output width, and the batch-scaled learning rate; and
6. record the values that will actually be consumed.

A search leaf can be written as

```
lr_base: [0.0025, 0.0001, 0.01, log]
```

The four entries mean, in order, default value, lower bound, upper bound, and proposal scale. In a normal run, the resolver chooses `0.0025`. In a hyperparameter search, the study proposes a value between the two bounds on a logarithmic scale. The anonymous positions are therefore not interchangeable; each has a named meaning supplied by the schema.

Resolved configuration is persisted because a saved emulator must be rebuilt from the facts that created it, not from whatever defaults a future checkout happens to contain. Operational facts such as the number of GPUs or whether progress printing was quiet can be reported separately; they do not change the mathematical model.

a. Current gap. The family-specific validators reject unknown keys inside blocks such as `data.grid` and `data.cmb`, but the public configuration surface is not yet total: several top-level and `train_args` names can be ignored, coerced by Python truthiness, or forwarded to a default. A misspelled key can therefore select a different branch instead of producing the cheap error described above.

b. Required closure. Every public configuration namespace has an explicit allowlist and required set. Booleans are actual booleans rather than nonempty strings, finite-real controls reject NaN and infinity before ordered comparisons, and the resolved record contains the values consumed after validation. This closure belongs at the configuration boundary; a later finite-training check is defense in depth, not permission to accept an invalid recipe.

E. Factories turn specifications into live objects

After validation, constructible components are represented by specification dictionaries. Their `cls` entry is a Python class object and their other entries are constructor keyword arguments. A factory performs three steps:

1. copy the specification so the caller's dictionary is not changed in place;
2. remove `cls` and inject values known only at run time; and
3. call the class once to create a fresh instance.

For a model, injected values include encoded input width, target width, dtype, and device. For an optimizer they include the model parameters and the batch-scaled learning rate. For a scheduler they include the optimizer it will control.

The specification contains a class, not a shared module instance. Creating a fresh instance prevents two training runs from sharing parameters or state. Similarly, activation and normalization entries are factories called once per layer. Two layers may use the same class and constructor values while still owning distinct learnable tensors.

F. Precedence is part of the public behavior

A value can appear at more than one level. The code therefore needs an explicit precedence rule: a statement of which value wins. The important cases are:

- a phase-specific trunk or head block replaces the corresponding top-level optimizer, learning-rate, scheduler, or EMA block;
- a head-specific activation overrides the model-wide activation only for that head;
- a one-knob sweep deep-copies the base configuration and changes the one named leaf for that run; and
- a hyperparameter search proposes only leaves written in search-range form; all scalar leaves remain fixed.

Replacing a mapping and merging a mapping are different mechanics. A replace operation discards unspecified fields from the parent. A merge inherits them. The resolver chooses one behavior per block and records the result so a reader does not need to reconstruct inheritance from several YAML locations.

G. Validation precedes expensive work

The safe execution order is

command line
↓
resolved paths and parsed YAML
↓
validated schema and component specifications
↓
opened data and staged rows
↓
model allocation.

This order makes a misspelled key or invalid schedule a cheap error. It also ensures that artifact metadata describes accepted values rather than merely echoing unvalidated user text.

At the end of this section, no cosmological value has been centered, no data vector has been whitened, and no parameter has been moved to a GPU. The program has a validated and resolved recipe. It can now open the files named by that recipe and preserve their row identities.

III. TRAINING CAMPAIGNS: ONE RUN, SWEEPS, SEARCHES, AND BAKE-OFFS

The experiment engine trains one resolved configuration. Campaign drivers call that engine repeatedly to answer comparative questions. Every point is a complete independent training unless the driver explicitly documents shared study state.

A. One training run

A single-run driver is the simplest scientific unit. It resolves one YAML, selects one seeded data subset, constructs one model, and writes one artifact pair plus histories. Its output answers: *what function did this exact recipe learn?*

The optional diagnostic flag adds reporting after training. It does not change the objective or choose a different best epoch. Expensive family diagnostics may have their own bounded-data policy so creating a report does not materialize a tensor wider than training itself.

B. Training-size learning curve

An N_{train} sweep trains the same resolved model at several absolute row counts. Each point applies physical cuts first and then chooses its requested number of rows. The horizontal axis is data volume; the vertical axis is a fixed validation failure statistic such as the fraction with $\Delta\chi^2 > 0.2$.

A curve still falling at the largest point says that additional simulations are likely useful. A flat tail suggests that architecture, objective, or optimization is limiting accuracy. A rising point may indicate optimization instability, a subset-identity error, or ordinary statistical noise; repeated seeds distinguish them.

Every point fits its own data-dependent geometry and analytic base from exactly its selected rows. Borrowing geometry or a PCE fit from the largest point leaks information and no longer measures a training-size effect.

a. Current gap. The scalar, CMB, background, and matter-power training-size wrappers currently call a pool counter that indexes a `param_cuts` bound even when the documented optional block is absent. The shipped no-cut examples can fail before the first sweep point. A one-run success is therefore not evidence that the corresponding learning-curve driver is reachable.

b. Required closure. The shared counter treats an absent cut block as the complete available pool, uses the same selection order as staging, and is exercised by every thin family wrapper both with and without physical cuts.

C. One-knob sweep

A one-knob sweep deep-copies the resolved base recipe and changes one named leaf for each run. Examples include learning rate, batch size, convolution kernel width, or FiLM. Deep copy means nested mappings are duplicated; a shallow copy would let one point mutate later points.

The dotted path identifies the leaf. Intermediate blocks may be constructed only where the schema defines that behavior. An unknown first segment or a phase-specific path on an architecture without that phase is refused. The results table records the actual value and fixed context so its curve can be reconstructed without reading a mutable YAML later.

D. Hyperparameter search

A search-range leaf has four named positions:

$$[\text{default}, \text{minimum}, \text{maximum}, \text{kind}].$$

Kind is integer, linear float, or logarithmic float. Optuna proposes values only for leaves written in this form. All scalar leaves stay fixed, and ordinary non-search drivers use the default.

The objective function returns the declared validation statistic from one complete trial. Trial status is part of the result: a failed trial is not a large finite score and not a completed trial.

a. Current gap. The parallel study parent does not yet require successful current worker exit or a positive count of new completed trials before reporting the best historical trial.

b. Required closure. A resumable study carries a canonical manifest: objective identity, resolved fixed configuration, search-space schema, data digests, family/law facts, and implementation identity. Operational controls such as timeout, worker count, or requested total trials do not change the objective and may vary on resume. A journal with no manifest or a mismatching manifest is refused rather than combining trials from different scientific questions.

The current journal does not yet persist and compare this manifest. Until it does, resume is safe only after manually verifying that objective, fixed configuration, search ranges, data, and implementation are unchanged.

E. Activation bake-off

The activation bake-off nests two comparisons. For each activation family it trains several N_{train} points, then overlays the learning curves. The architecture, seed policy, data, optimizer, and evaluation statistic stay fixed.

A family can win at small data and tie later, or cross another family. The complete curves are more informative than ranking one final point. The bake-off also applies head-liveness checks so a strong trunk cannot make an inactive activation/head combination appear competitive.

F. Parallel execution and GPU packing

Campaign points are independent jobs, not pieces of one distributed model. One worker owns one complete training on one selected device. Training-size points can be scheduled largest-first to reduce the long tail; equal-cost one-knob points can use round-robin assignment.

GPU packing allows several small jobs to share a large card. A preflight estimate converts model, batch, activation-storage, and I/O needs into a fraction of device memory. Tokens limit concurrent jobs so their total estimate stays inside policy. Packing improves throughput but makes per-epoch timing noisier, so timing comparisons run without it.

a. Required closure. Worker lifecycle includes setup and staging, not only the training loop. A worker reports a success or failure result through a finally-protected channel. The parent has a bounded wait and checks process exit status. A crash before the first epoch must not leave the parent waiting forever or allow an old result table to be reported as the current campaign.

b. Current gap. The multi-GPU activation bake-off can currently wait indefinitely when a worker dies before emitting a result. Run it under an external timeout until the bounded parent/worker protocol is gate-proven.

IV. GENERATING THE COSMOLOGIES AND TARGETS

Training cannot begin until an external physics calculation has produced paired cosmologies and targets. The programs under `compute_data_vectors` own that earlier stage. A shared generator core owns sampling, MPI work distribution, checkpointing, and parameter sidecars; each thin family driver states which physical quantities to request and which array files to write.

A. The training distribution is wider than the posterior

An emulator used inside inference must remain accurate where the sampler may walk, including the edges around the posterior. Training only on the posterior center would teach the network densely where calls are common but leave no support just outside that region. The generator therefore offers a tempered distribution

$$\log p_T(\boldsymbol{\theta}) = \frac{1}{T}\left[-\frac{1}{2}(\boldsymbol{\theta}-\boldsymbol{\theta}_0)^{\mathsf{T}}\widetilde{\Sigma}^{-1}(\boldsymbol{\theta}-\boldsymbol{\theta}_0) + \log\pi(\boldsymbol{\theta})\right]. \tag{1}$$

$\boldsymbol{\theta}_0$ is the fiducial cosmology, $\widetilde{\Sigma}$ is the proposal covariance after the declared correlation treatment, π is the prior, and T is the temperature. Dividing the quadratic term by T produces effective covariance $T\widetilde{\Sigma}$: larger T makes a wider cloud.

The maximum-correlation control clips very strong off-diagonal correlations in the proposal covariance. Geometrically, an unmodified Fisher covariance can form a thin tilted sheet. Filling only that sheet gives poor coverage in directions perpendicular to the degeneracy. Reducing the correlation fattens the cloud so the training set occupies volume rather than a nearly one-dimensional ridge.

A uniform mode instead draws throughout a temperature-expanded hard box. It is useful for coverage stress tests. It does not mean that temperature is irrelevant: Gaussian or otherwise unbounded priors still need a declared rule for turning their scale into finite box boundaries.

B. Sampling and MPI work distribution

In the tempered mode, an ensemble sampler proposes cosmologies. An *ensemble* is a set of walkers whose proposal uses the positions of other walkers. Differential-evolution and snooker moves explore elongated correlations without requiring a separate hand-tuned step size for every parameter. The resulting chain is reduced to the requested distinct rows, and its autocorrelation estimate is reported.

MPI, the Message Passing Interface, distributes expensive physics calls across processes. One master process

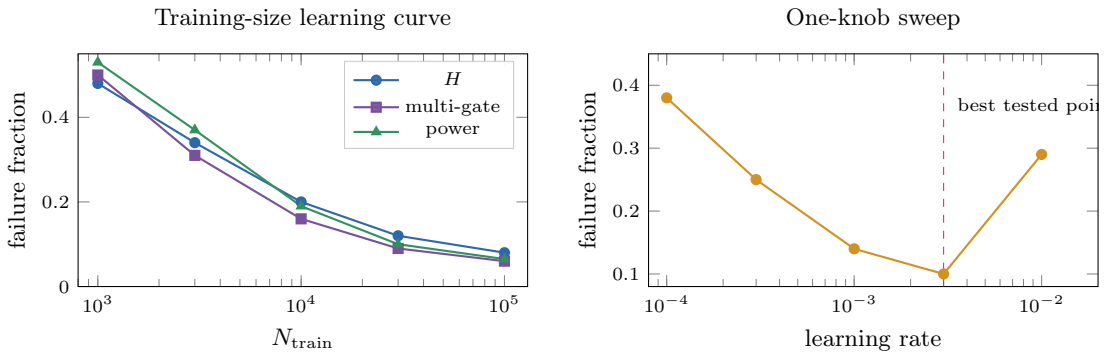

FIG. 1. How campaign curves are read. The illustrative activation curves compare data efficiency under fixed context. The one-knob curve changes only the learning rate; its minimum does not become a universal default outside that recorded context.

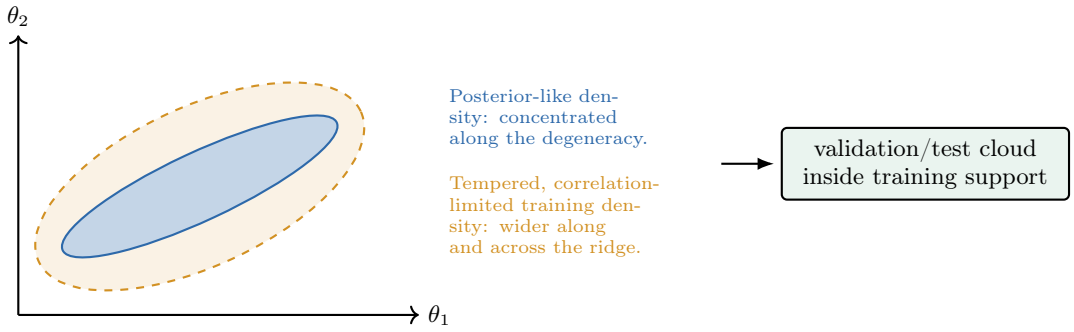

FIG. 2. Conceptual two-parameter coverage. Temperature widens the proposal; correlation control fills directions across the posterior degeneracy. Validation belongs inside the trained support so its score measures interpolation rather than accidental edge extrapolation.

owns the row queue and output coordinates. A worker receives one complete cosmology, asks the family truth code for its outputs, and returns a payload plus an acceptance verdict. Workers do not independently append anonymous rows to the final array; the master places each returned payload into the slot belonging to the parameter row that was sent.

The family truth codes differ:

- cosmic shear calls CosmoLike and writes one masked data-vector dump;
- CMB calls the Code for Anisotropies in the Microwave Background (CAMB) and writes TT, TE, EE, and lensing-potential spectra;
- background calls CAMB for $H(z)$ and distance quantities on a named redshift axis; and
- matter power calls CAMB for linear power and boost surfaces on named z and k axes, with any analytic-base companions.

C. A checkpoint is a coordinated file set

Generation is expensive, so the master periodically checkpoints. A valid checkpoint is not merely one array that can be opened. It is a coordinated set containing the parameter rows, parameter names, covariance and ranges, family payloads, coordinate sidecars, and failure flags. Resume must prove that these files describe the same generation identity and completed-row set.

For a rejected physics evaluation, the verdict must remain false until the payload has been validated in its storage dtype and written successfully. Writing zeros or stale values and clearing the failure flag would turn an external failure into a training example. Before training, the file-set contract therefore requires every row to have a current successful verdict, the payload shape expected by its family, and finite values after conversion to the on-disk dtype.

Publication of several files is a transaction in the conceptual sense: readers must see either the complete old checkpoint or the complete new one, not a mixture. A manifest records file identities, coordinate axes, row counts, dtypes, and digests. Resume refuses mismatches rather than treating a load exception as permission to overwrite the same roots with a fresh run.

a. Current gap. The generators do not yet enforce that full contract. A run can preserve failed-row flags while still exiting successfully; the trainer does not use the failure sidecar as a mandatory readiness verdict; several checkpoint sets omit parameter-name or coordinate-

axis sidecars; and a checkpoint-load exception can fall through to a fresh run at the same output roots. A file that exists is therefore not, by itself, a certified training set.

b. Required closure. Transactional publication and one canonical manifest bind generator identity, ordered parameter names, coordinate axes, completed rows, storage dtype, failure state, and file digests. Resume compares that manifest before any write, and a nonempty failure set produces a nonzero process result. Training revalidates the same facts at ingress rather than trusting the generator's self-report.

D. Sidecars carry scientific meaning

The parameter text file contains bookkeeping columns as well as sampled parameters. Its parameter-name sidecar states which columns are physical inputs. The covariance header provides an independent order declaration. The trainer must resolve columns by name, not by assuming that parameters always occupy a fixed slice between two bookkeeping columns.

Axis sidecars are equally important. A CMB array of width 999 could represent $\ell = 2, \ldots, 1000$ or $\ell = 10, \ldots, 1008$. Width equality does not distinguish them. Background and matter-power files likewise need their actual redshift and wavenumber values. The artifact can then state that its axis was verified against the generated dump rather than inferred from an unrelated covariance.

E. The handoff to training

Generation ends with an immutable row relation:

$$\text{parameter row } i \longleftrightarrow \text{accepted physics payload } i.$$

The training YAML names the completed files. Its `train_args` block is not the generator's block of the same name: the generator configures the external physics model, while the trainer configures a neural approximation. The next section begins where the generator stops. It opens the file set, revalidates names, axes, rows, and failure state, and selects a reproducible training subset without breaking the row relation.

V. FROM COSMOLOGIES AND DATA VECTORS TO A STAGED TRAINING SOURCE

Suppose the data-generation calculation has finished. We have a table of cosmologies, a list that names the cosmological parameters, and one data vector for each cosmology. What does the emulator code do first?

There are two answers because the program has a control path and a numerical path. The control path first reads the YAML file, resolves its file names, checks which emulator family was requested, and chooses the corresponding model class. No cosmological row has been transformed yet. The first numerical operation is *staging*: the code aligns the parameter table with the data-vector dump, selects a reproducible subset of valid rows, and decides whether that subset lives in ordinary memory or remains backed by a file on disk.

This section follows that process in execution order. It stops just before the code constructs the whitening geometries. Section VI begins with the staged arrays and derives the parameter-space and data-vector transforms.

A. The four files that must agree

For an ordinary data-vector emulator, one training source is described by four files. The validation source has the same structure but different rows.

Let N be the number of generated cosmologies, P the number of modeled parameters, and D the length of one physical data vector. The two arrays that matter numerically are

$$C = \begin{bmatrix} \boldsymbol{\theta}_0^\mathsf{T} \\ \boldsymbol{\theta}_1^\mathsf{T} \\ \vdots \\ \boldsymbol{\theta}_{N-1}^\mathsf{T} \end{bmatrix} \in \mathbb{R}^{N \times P}, \qquad V = \begin{bmatrix} \mathbf{d}_0^\mathsf{T} \\ \mathbf{d}_1^\mathsf{T} \\ \vdots \\ \mathbf{d}_{N-1}^\mathsf{T} \end{bmatrix} \in \mathbb{R}^{N \times D}. \tag{2}$$

Row i of C and row i of V must describe the same cosmology. This row identity is the central invariant of staging. A network trained on C_i paired with V_j for $i \neq j$ does not learn a noisy approximation. It learns the wrong function.

a. The current parameter-table layout. The nonscalar loader currently reads the complete text table and then uses the positional slice `[:, 2:-1]`. In other words, it assumes that the first two columns are the sample weight and minus log posterior, the modeled parameters occupy the middle columns, and the last column is not a model input. The `.paramnames` file is checked against the covariance header when it is present, but the returned name resolution is not yet used to form the slice. This is a current implementation constraint, not a property of the mathematics. A table with extra derived columns in the middle can pass the name comparison and still violate the positional assumption.

The scalar-family loader is different. Its inputs and targets are both columns of the parameter table, so it resolves the requested input and output names explicitly through the `.paramnames` sidecar.

B. Control path: resolving a run before touching the rows

The single-run driver is `cosmic_shear_train_emulator.py`. Its family sib-

File	Contents	Why the code needs it
Parameter table, `*.txt`	One row per cosmology	Supplies the input parameters $\boldsymbol{\theta}_i$.
Parameter names, `*.paramnames`	One name per parameter-table column	States which physical parameter occupies each column.
Parameter covariance, `*.covmat`	A square covariance matrix with a name header	Supplies a second declaration of the input order and later defines the input whitening transform.
Data-vector dump, `*.npy`	One vector $\mathbf{d}_i$ per cosmology	Supplies the targets that the network will learn.

lings call the same package layer. The startup chain is

command-line strings
↓ `resolve_cocoa_config`
parsed YAML with absolute data paths
↓ `EmulatorExperiment.from_config`
validated family, model class, device, and run settings
↓ `exp.run()`
stage train, stage validation, build geometries, train.

The path resolver reads the environment variable `ROOTDIR`. It joins the project name and the `chains` directory to each relative data-file name. An absolute path remains absolute. This step changes strings in a Python dictionary. It does not load the large arrays.

`EmulatorExperiment.from_config` is an alternative constructor. A `classmethod` receives the class as its first argument, conventionally named `cls`, and returns a newly constructed instance. It validates the configuration and chooses one branch: cosmic shear, scalar outputs, a CMB spectrum, a one-dimensional background grid, or a two-dimensional matter-power grid. The ordinary constructor then stores cheap state: parameter names, the selected compute device, the logger, and empty slots for objects that do not exist yet.

The important empty slots are

This delayed construction matters in parameter sweeps. A sweep can keep the validation source fixed while replacing the training subset, or keep both sources fixed while rebuilding only a model. Expensive state is therefore created by named methods instead of being hidden inside the constructor.

C. Numerical path: opening the source

`stage_train` creates a local random-number generator from the YAML value `split_seed`, then calls `load_source` in `emulator/data_staging.py`. The generator is local state. It is passed as an argument, so row selection does not depend on a module-level random state left behind by an earlier call.

The two large inputs are opened differently:

```
dv = np.load(dv_path,
             mmap_mode="r",
             allow_pickle=False)
C_all = np.loadtxt(params_path,
                   dtype="float32")
C = C_all[:, param_cols]
```

`np.loadtxt` is eager: it parses the complete parameter table and creates a new in-memory NumPy array. The requested dtype is `float32`, four bytes per element. The slice that follows selects the modeled columns.

`np.load(..., mmap_mode="r")` is lazy with respect to the payload. It creates an `np.memmap`, an array-like object whose elements remain in the `.npy` file until a later indexing operation asks the operating system to read them. The mode `"r"` makes this mapping read-only. The code can inspect `dv.shape` without loading ND floating-point values.

At this point the data still live on the host, meaning ordinary CPU memory or disk-backed virtual memory. Nothing has moved to CUDA or Apple Metal Performance Shaders (MPS). Device placement occurs later, after the geometry exists and the batching code knows which rows a particular minibatch needs.

1. *The first row-identity check*

The loader immediately compares the row counts:

$$C.\text{shape}[0] = V.\text{shape}[0]. \tag{3}$$

If the counts differ, it raises an error that names both files and both counts. Equality is necessary but not sufficient. It proves that the files have the same number of rows; it cannot prove by itself that row i in one file belongs to row i in the other. The generator must preserve that ordering when it writes the pair.

D. A seeded permutation, followed by physical cuts

The loader next creates a permutation of the integers $0, \ldots, N-1$:

```
order_tensor = torch.randperm(
    n,
    generator=gen)
order = order_tensor.numpy()
```

`torch.randperm` returns a one-dimensional PyTorch tensor containing every integer exactly once in pseudorandom order. It is not an iterator and it is not lazy: all N

Attribute	Meaning before and after the corresponding method runs
`train_set`	`None` initially; a staged training-source dictionary after `stage_train`.
`val_set`	`None` initially; a staged validation-source dictionary after `stage_val`.
`pgeom`	`None` initially; the parameter transform after `build_geometry`.
`geom`	`None` initially; the output transform after `build_geometry`.
`chi2fn`	`None` initially; the loss wrapper after `build_geometry`.
`model`	`None` initially; the trained PyTorch module after `train`.

indices are materialized. The call to `.numpy()` exposes the CPU tensor as a NumPy array. Because both objects use CPU storage, this conversion normally shares memory rather than copying the index values.

The physical-window function receives C, the permutation, and the ordered parameter names. Each active window locates its columns by name and computes one derived quantity per candidate row:

$$\omega_b h^2 = \Omega_b \left(\frac{H_0}{100}\right)^2, \tag{4}$$

$$(\Omega_m h)^2 = \left(\Omega_m \frac{H_0}{100}\right)^2, \tag{5}$$

$$\omega_m = \Omega_m \left(\frac{H_0}{100}\right)^2, \tag{6}$$

$$\omega_m n_s = \Omega_m \left(\frac{H_0}{100}\right)^2 n_s. \tag{7}$$

For an active lower bound a and upper bound b, the comparison is strict: $a < q_i < b$. The code builds a Boolean mask for each window and intersects it with the masks already applied. A Boolean mask is an array of `True` and `False` values, one per candidate row. Indexing `order[mask]` uses NumPy advanced indexing and creates a new array of the surviving integer indices. Their order remains the shuffled order.

The result, called `phys`, is therefore not a new cosmology table. It is a list of row coordinates into the original files:

$$\texttt{phys} = [i_0, i_1, \ldots],$$
$$C[i_k] \text{ and } V[i_k] \text{ remain a matched pair.}$$

E. Selecting an absolute number of rows

The YAML states an absolute number of training rows, `n_train`, and an absolute number of validation rows, `n_val`. After the cuts, the loader checks that the requested number is at least one and that the physical pool is large enough. It then takes

```
idx = phys[:n_keep]
```

The slice is the first n_{keep} entries of the seeded, cut permutation. Using the same seed at multiple training sizes creates nested learning-curve subsets: the 2,000-row set is the prefix of the 4,000-row set rather than an unrelated draw.

Training and validation use different files. Reusing the numerical seed therefore repeats the selection algorithm, not the cosmologies themselves. The scientific assumption is that the two file sets are disjoint. That assumption must be checked from file identity or row identity; a seed does not establish disjointness.

F. Staging: copy the subset or retain the disk mapping

The word *stage* means "put the selected rows in the storage form from which later batches will read them." It does not yet mean "move them to the GPU." The helper `stage_source` estimates whether the selected rows fit within a configured fraction of available host memory. In the current implementation, this estimate counts the selected data-vector array but not the selected parameter-array copy. The resident branch nevertheless creates both arrays. The estimate can therefore be low by $n_u P \,\text{sizeof}(C_{ij})$ bytes. This is a memory-accounting defect in the present code, not a license for consumers to ignore the parameter copy.

There are two outcomes.

1. *Resident host-memory outcome*

If the pair fits, the helper gathers the sorted unique selected rows into compact arrays. Advanced integer indexing is eager and returns a copy. The compact arrays have shapes

$$C_{\text{src}} \in \mathbb{R}^{n_u \times P}, \qquad V_{\text{src}} \in \mathbb{R}^{n_u \times D}, \tag{8}$$

where n_u is the number of distinct selected rows. The source's `idx` is then expressed in the compact arrays' local row coordinates. If original dump row 91 becomes compact row 3, later code asks the resident array for row 3, not row 91.

2. *Disk-backed outcome*

If the data vectors do not fit, the helper retains the original read-only `np.memmap`. The source's `idx` remains in global on-disk row coordinates. A later batch read such as `dv[idx]` asks the operating system for only those rows. This advanced indexing operation materializes the

requested batch; it does not turn the entire dump into an in-memory array.

The parameter table was already loaded eagerly by `np.loadtxt`. Thus, "disk-backed source" describes the data vectors, not every object in the source dictionary.

G. The source dictionary and its coordinate systems

`load_source` returns a small dictionary:

```
dump_rows = sorted_unique_original_rows
src = {"C":         C_src,
       "dv":        dv_src,
       "idx":       idx_src,
       "dump_rows": dump_rows}
```

The fields have distinct jobs.

`dump_rows` is needed by the two-dimensional matter-power path. That path has a second, row-aligned base dump. Even when staging has converted the primary arrays to local coordinates, the sibling file still needs the original disk row numbers. Sorting makes its reads sequential. Applying `np.unique` removes duplicates and also sorts; it returns a new array.

The complete coordinate flow is

original rows $0, \ldots, N-1$
$\downarrow$ seeded permutation and physical mask
`idx` : selected rows in shuffled training order
$\downarrow$ sorted unique gather, only if copied to RAM
`idx_src` : local coordinates into compact arrays

`dump_rows` : original coordinates retained for sibling files.

H. Training statistics are owned by the training source

For the training source, `with_means=True`. The loader computes the mean of every parameter column and every staged data-vector column:

$$\bar{C}_j = \frac{1}{n}\sum_{i=1}^{n} C_{ij}, \qquad \bar{V}_k = \frac{1}{n}\sum_{i=1}^{n} V_{ik}. \tag{9}$$

These arrays are stored as `C_mean` and `dv_mean`. The validation source uses `with_means=False`; it does not estimate a second zero point. Training and validation must be expressed in the same coordinate system, so the training-derived geometry transforms both.

Subtracting a center does not change a residual-based loss. If prediction and truth are both centered by $\bar{V}$, then

$$(V_{\text{pred}} - \bar{V}) - (V_{\text{true}} - \bar{V}) = V_{\text{pred}} - V_{\text{true}}. \tag{10}$$

Centering changes the zero point the network learns, not the physical prediction-minus-truth residual.

The matter-power path is again special. Its raw surface is transformed into "law space"—for example, $\log(P/P_{\text{base}})$—and may be thinned along the wavenumber axis. Its final output mean must therefore be computed after that transformation. The generic loader skips the raw `dv_mean`; the bounded matter-power staging code streams the transformed mean and variance over row chunks.

I. Worked staging example: the same rows in two coordinate systems

Return to the four-row example in Sec. I. Suppose the seeded permutation is

$$\texttt{order} = [2, 0, 3, 1]$$

and a physical cut rejects original row 0. The surviving pool is

$$\texttt{phys} = [2, 3, 1].$$

If `n_keep=2`, the selected global rows are $[2, 3]$. Before staging, the following expressions all refer to the original files:

$$\begin{aligned} C[2] &= (0.33, 73, 0.052), \\ C[3] &= (0.30, 69, 0.049), \\ V[2] &= (12, 24, 36, 48, 60), \\ V[3] &= (10.5, 21, 31.5, 42, 52.5). \end{aligned}$$

If the subset fits in host RAM, `stage_source` makes compact copies:

$$C_{\text{src}} = C[[2, 3]], \qquad V_{\text{src}} = V[[2, 3]].$$

The new arrays have two rows. Their local coordinates are $[0, 1]$, so the returned `idx` becomes $[0, 1]$. `dump_rows` remains $[2, 3]$ because a separately opened base dump still uses original file coordinates.

If the subset does not fit, the program returns the original C and memmapped V with `idx=[2,3]`. No local

Key	Coordinate system	Meaning
`C`	Matches `idx`	Parameter rows. It is compact in the resident outcome and the full eager table in the disk-backed outcome.
`dv`	Matches `idx`	Data-vector rows. It is compact in the resident outcome and a file mapping in the disk-backed outcome.
`idx`	Local or global	The row coordinates that consumers must use with this particular `C`/`dv` pair.
`dump_rows`	Always global	Sorted unique row numbers in the original files. A sibling dump uses these coordinates to retrieve the same cosmologies.

reindexing occurred. A consumer must always apply the `idx` returned with its particular arrays. Reusing the resident branch's $[0, 1]$ against the file-backed branch would silently select the wrong cosmologies.

J. Worked memory calculation

Assume $n_u = 250{,}000$, $P = 20$, $D = 3{,}000$, and both arrays are stored as `float32`. One element occupies four bytes. The resident copies need

$$B_C = 250{,}000 \times 20 \times 4 = 20{,}000{,}000 \text{ bytes},$$
$$B_V = 250{,}000 \times 3{,}000 \times 4 = 3{,}000{,}000{,}000 \text{ bytes},$$
$$B_{\text{total}} = B_C + B_V = 3{,}020{,}000{,}000 \text{ bytes}.$$

The current implementation compares only B_V with the host-memory allowance even though it creates both copies. The missing 20 MB is small in this example, but the correctness rule cannot depend on it usually being small. A storage planner must count every allocation whose lifetime it creates.

K. What has not happened yet

At the end of `stage_train` and `stage_val`, the program has selected and located the rows. It has not yet built a neural network, an optimizer, or a covariance-weighted loss. Apart from the seeded permutation, the arrays are still NumPy objects on the host.

The next method, `build_geometry`, creates two maps:

$$\text{parameter geometry: } \boldsymbol{\theta} \longmapsto x, \qquad \text{raw cosmology to centered, rescaled network input,}$$
$$\text{output geometry: } \mathbf{d} \longmapsto t, \qquad \text{physical data vector to centered, covariance-scaled target.}$$

Only after those maps exist can the batching code convert a NumPy batch with `torch.from_numpy`, cast it to the model's `float32` dtype, move it to the selected device, and encode it. That is the subject of Sec. VI.

L. Section summary

The first numerical job is not neural-network training. It is identity preservation. The staging code must keep four statements true:

1. Parameter row i and data-vector row i describe the same cosmology.
2. Parameter names and column order match the covariance that will transform them.
3. The seeded subset is selected after the declared physical cuts and contains the requested absolute number of rows.
4. Every row index is interpreted in the coordinate system of the array it indexes: compact local rows for a resident copy, original global rows for a file-backed dump.

Once those conditions hold, the program has a staged training source. The next problem is numerical geometry: how to transform correlated parameters and correlated outputs into coordinates a neural network can learn without silently changing the χ^2 that defines emulator accuracy.

VI. GEOMETRIES: CHANGING COORDINATES WITHOUT CHANGING THE SCIENCE

The staged source contains physical numbers. The columns can differ by many orders of magnitude and can be strongly correlated. A neural network trains more easily when its inputs and targets have comparable scales. The geometry classes own that coordinate change.

Here, *geometry* does not mean spacetime geometry. It means a reversible map between physical coordinates and network coordinates. There are two maps:

$$\boldsymbol{\theta} \xleftrightarrow[\text{decode}]{\text{encode}} x, \qquad \mathbf{d} \xleftrightarrow[\text{decode}]{\text{encode}} t.$$

x is the tensor presented to the model. t is the tensor the model is trained to predict. The inverse maps are part of the saved artifact because an emulator prediction is useful only after it has returned to physical units.

A. The parameter geometry

The ordinary input transform is `ParamGeometry`. The constructor stores four facts:

The covariance is symmetric, so NumPy computes

$$\Sigma = Q\,\mathrm{diag}(\lambda_1, \ldots, \lambda_P)Q^{\mathsf{T}}. \tag{11}$$

For a batch of raw parameter rows $\Theta \in \mathbb{R}^{B\times P}$, encoding is

$$X = (\Theta - \boldsymbol{\mu})Q\,\mathrm{diag}(s_1^{-1}, \ldots, s_P^{-1}). \tag{12}$$

Subtracting the mean centers the cloud at zero. Multiplication by Q rotates the correlated ellipsoid onto its principal axes. Division by s_j gives each rotated direction unit scale. This complete operation is called *whitening*.

Decoding performs the inverse operations in reverse order:

$$\Theta = [X\,\mathrm{diag}(s_1, \ldots, s_P)]\,Q^{\mathsf{T}} + \boldsymbol{\mu}. \tag{13}$$

Because Q is orthonormal, $QQ^{\mathsf{T}} = I$. A numerical identity test must therefore verify

$$\texttt{decode(encode(theta))} \simeq \texttt{theta} \tag{14}$$

over representative rows, with an error consistent with the stored `float32` representation.

a. Alternative constructors. The method `from_covmat` reads and diagonalizes the training covariance. The method `from_state` rebuilds the object from saved arrays. The ordinary `__init__` method only stores fields. Keeping those jobs separate means inference never needs the original covariance file.

B. Views, copies, dtype, and device

The geometry converts each array with

```
torch.as_tensor(array,
                dtype=torch.float32,
                device=device)
```

If the source already has the requested dtype and device, PyTorch may reuse its storage. If dtype or device changes, it creates new storage. Moving from a NumPy CPU array to CUDA necessarily copies the values to GPU memory.

The model uses `float32`. Covariance construction and diagnostic reductions often use `float64` on the CPU because sums and eigendecompositions lose more information than a neural-network forward pass. The program must state where a cast occurs. A finite nonzero scale in `float64` is not automatically usable if it becomes zero in `float32`.

C. The data-vector geometry

Cosmic-shear output uses `DataVectorGeometry`. Its first operation is a mask. The full physical vector has width D, but the likelihood keeps only K entries. `dest_idx` stores their global column positions.

For a batch $V \in \mathbb{R}^{B\times D}$,

$$V_{\text{keep}} = V[:, \texttt{dest\_idx}] \in \mathbb{R}^{B\times K}.$$

This is PyTorch advanced indexing. It gathers the selected columns in the specified order and returns a new tensor. The method is named `squeeze`, but it is not `torch.squeeze`: no size-one dimension is removed.

The inverse method, `unsqueeze`, allocates a zero-filled $B \times D$ tensor and assigns the K values back into `dest_idx`. Masked locations remain zero. This is a scatter operation.

After masking and centering, the dense-covariance geometry applies the same eigenvector construction as Eq. (12). If

$$C_{\text{keep}} = U\,\mathrm{diag}(\eta)U^{\mathsf{T}}, \tag{15}$$

then a centered kept vector r is whitened as

$$t = rU\,\mathrm{diag}(\eta^{-1/2}). \tag{16}$$

The ordinary squared length of t equals the covariance-weighted physical length of r:

$$\|t\|^2 = rC_{\text{keep}}^{-1}r^{\mathsf{T}}. \tag{17}$$

This identity is why training in whitened space does not redefine the scientific metric.

D. Diagonal output families

Some families have one independent scale per output instead of a dense covariance rotation. They use

$$t_k = \frac{d_k - \bar{d}_k}{\sigma_k}, \qquad d_k = t_k\sigma_k + \bar{d}_k. \tag{18}$$

The axes and units are artifact facts. A consumer reads them from the saved geometry rather than reconstructing them from a filename or a current code default.

Field	Meaning
`names`	Parameter names in the exact column order of the covariance.
`center`	Training-set mean $\boldsymbol{\mu}$.
`evecs`	Orthonormal covariance eigenvectors Q.
`sqrt_ev`	Square roots of the covariance eigenvalues, $s_j = \sqrt{\lambda_j}$.

Geometry	Coordinate axis	Stored physical facts
`ScalarGeometry`	Named derived parameters	Output names, center, scale.
CMB diagonal	Multipole ℓ	Spectrum, ℓ, units, fiducial spectrum, cosmic-variance scale, amplitude law.
`GridGeometry`	Redshift z	Quantity, units, target law, offset, stored z grid.
`Grid2DGeometry`	Flattened (z, k) surface	Quantity, units, target law, both axes, constant-point mask.

E. The geometry acceptance test

A geometry is ready only when four numerical statements hold:

1. Every scale is finite and representably nonzero in its stored dtype.

2. `decode(encode(raw))` returns the raw value within the declared floating-point tolerance.

3. Saving and rebuilding the state leaves the transform unchanged.

4. The squared whitened residual agrees with a direct physical covariance contraction.

VII. THE NEURAL DESIGNS AND THEIR CONSTRUCTIBLE SPECIFICATIONS

Once the geometries exist, the program knows the model's input width and output width. It can construct a neural network without embedding data-file knowledge in the network class.

A. Define each design before comparing it

A *design* is the rule for connecting trainable operations, fixed coordinate operations, and any analytic physics component into one predictor. The names used by this library mean:

A multilayer perceptron, abbreviated MLP, is a sequence of affine maps and nonlinear activations. A dense or fully connected affine map lets every output feature depend on every input feature. "Residual" means a learned correction is added to an unchanged skip path; it does not mean a separate physical residual unless the loss explicitly constructs one. The following subsections define the mechanics behind each row.

B. A class is passed as data

The model configuration is a dictionary whose `cls` entry holds a class object and whose remaining entries are constructor arguments:

```
model_opts = {"cls":         ResMLP,
              "int_dim_res": 256,
              "n_blocks":    4,
              "block_opts":  block_opts}
```

A class object is callable. Calling `ResMLP(...)` creates a new module. `make_model` removes the bookkeeping entries, injects the data-dependent input and output dimensions, constructs the module, and moves it to the selected device.

The same pattern constructs the optimizer and scheduler. Computed values such as batch-scaled learning rate and device-dependent fused behavior are injected by the factory instead of being stored as stale configuration facts.

C. Factories create distinct submodules

Residual blocks receive activation and normalization factories. A factory is a callable such as `act(width)` that returns a new `nn.Module`. Calling it once per layer gives every layer its own learnable activation parameters. Passing one already-created activation instance would share its parameters across layers.

Parameter-free activations do not need the width, but they still obey the same interface:

```
def relu_factory(dim):
    return nn.ReLU()
```

The argument is intentionally unused. The uniform interface lets the model construction loop treat learnable and fixed activations identically.

Design	Definition	What is learned and what is fixed
ResMLP	A residual multilayer perceptron: dense layers act on the complete feature vector, and each block adds a learned correction to its input.	All dense weights and activation parameters are learned; input/output geometries are fixed after fitting.
ResCNN	A residual model with a one-dimensional convolutional correction head. A convolution reuses one local kernel along an ordered coordinate.	The trunk, kernels, and head gate are learned; scatter/gather coordinates and validity masks are fixed.
ResTRF	A residual model with a transformer correction head. A transformer repeats attention, feature mixing, and residual additions so each token can use information from other tokens.	Query/key/value projections, mixing layers, and head gate are learned; token/coordinate layout is fixed.
IA template model	A factored intrinsic-alignment model. Intrinsic alignment means galaxy shapes correlated by local structure rather than gravitational lensing. The network predicts basis templates and a closed-form law combines them with named IA amplitudes.	Template values are learned; the amplitude combination law is fixed physics.
PCE plus neural residual	A polynomial-chaos expansion supplies a frozen smooth base; a neural network learns what that finite polynomial basis misses.	Selected polynomial terms and coefficients are fitted once, then frozen; the residual network remains trainable.

D. Registration: why ModuleList matters

PyTorch discovers trainable parameters by walking registered child modules. `nn.Sequential` and `nn.ModuleList` register their contents. A bare Python list does not. A plain list is safe only as a temporary builder that is immediately expanded into `nn.Sequential(*layers)` or copied into `nn.ModuleList(layers)`.

Registration controls four operations:

1. inclusion in `model.parameters()`,
2. movement by `model.to(device)`,
3. inclusion in `state_dict()`, and
4. switching between training and evaluation modes.

E. ResMLP: the baseline map

The baseline network is

$$\begin{gathered} X\ (B \times P_{\mathrm{enc}}) \\ \downarrow \quad \text{input linear layer} \\ H\ (B \times W) \\ \downarrow \quad N_{\mathrm{block}} \text{ residual blocks} \\ H'\ (B \times W) \\ \downarrow \quad \text{output linear layer and affine map} \\ \widehat{T}\ (B \times K). \end{gathered}$$

A residual block learns a correction $F(h)$ and returns $h + F(h)$. The skip path gives gradients a direct route through a deep network. The final affine module supplies a learnable output scale and offset.

F. Structured correction heads

`ResCNN` and `ResTRF` retain the ResMLP trunk and add a correction head. The trunk predicts every kept output. The head reshapes that output into physical bins, learns local or cross-bin structure, and adds a gated correction:

$$\widehat{T} = T_{\mathrm{trunk}} + g\, T_{\mathrm{correction}}. \tag{19}$$

The convolutional head treats tomographic bins as channels and angular positions as the one-dimensional coordinate. The transformer head treats bins as tokens. A token is one row presented to attention; attention mixes information between token rows.

Ragged bins have different lengths. The implementation scatters them into a rectangle of width equal to the longest bin. Padding positions require an explicit validity mask because convolution, affine modulation, and attention can turn an initial zero into a nonzero hidden value.

The correction is initialized to zero, so the initial structured model equals its trunk. A nonzero outer gate lets gradients reach the zero-initialized last correction layer on the first optimizer step. An activation placed after a zero-initialized layer must have a nonzero derivative at zero, or the head can remain permanently asleep.

G. Factored intrinsic-alignment designs

The factored models do not ask the network to learn known polynomial dependence on intrinsic-alignment amplitudes. The input geometry appends the raw amplitudes after the whitened non-amplitude parameters. The model reads only the non-amplitude block and emits T templates:

$$X \longmapsto \left(\widehat{t}_1, \ldots, \widehat{t}_T\right), \qquad \widehat{t}_j \in \mathbb{R}^K.$$

The loss reads the appended amplitudes and combines the templates with the known coefficients. This separation keeps the exact amplitude law outside the neural approximation.

H. One dense layer, written without shorthand

The basic learned operation is an affine map. For a minibatch $H \in \mathbb{R}^{B\times W}$, a PyTorch `nn.Linear(W,W)` computes

$$Z_{bi} = \sum_{j=0}^{W-1} H_{bj} W_{ij} + a_i. \tag{20}$$

W_{ij} is a learnable weight and a_i is a learnable bias. The first index b selects a cosmology in the minibatch. The index j selects an incoming feature, and i selects an outgoing feature. PyTorch stores the weight matrix with shape (`out_features`, `in_features`) and applies the transpose convention needed for a batch whose feature axis is last.

An affine map alone cannot represent a general nonlinear physical response. Composing two affine maps still gives one affine map. The activation is therefore applied element by element after selected linear layers:

$$A_{bi} = \phi(Z_{bi}).$$

Element by element means that the activation changes each scalar without mixing feature columns. The linear layer performs the mixing; the activation provides the bend.

The entry and exit projections are rectangular. If the encoded cosmology has $P_{\mathrm{enc}} = 12$ features, the hidden width is $W = 128$, and the target has $K = 780$ values, their shapes are

$$(128, 12), \qquad (780, 128).$$

All dense layers between them are square $(128, 128)$. This fixed width lets a residual block add its input to its correction without another shape adapter.

I. A residual block, operation by operation

Figure 3 names the two paths. The *skip path* is a reference to the input tensor used by the final addition. The learned path computes $F(h)$. The output is $h+F(h)$, not a concatenation. Concatenation would append columns and change the width; addition preserves the width.

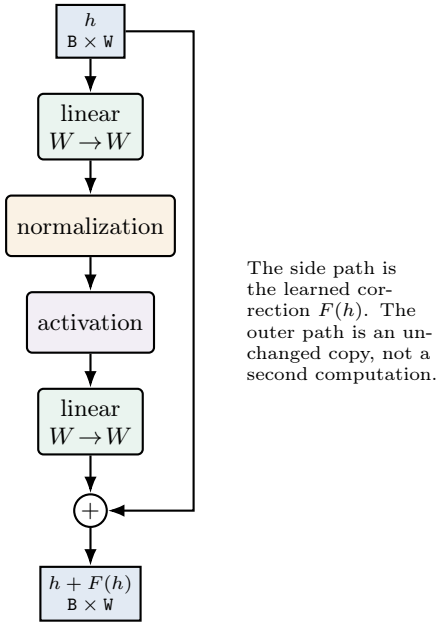

FIG. 3. A residual block. Both paths have the same shape, so addition is coordinate by coordinate. The skip path gives the gradient a direct route around the learned correction.

The input is not modified in place. Operations construct new tensors whose autograd records point back to their parents. *Autograd* is PyTorch's automatic-differentiation system: it records the executed tensor operations and later applies the chain rule in reverse during `backward()`. Avoiding an in-place overwrite keeps the original value available to the skip addition and to backward differentiation.

J. Normalization controls the activation's operating range

The model's normalization slot is an affine rescaling, not batch normalization. In its shared form,

$$\widetilde{Z} = gZ + b,$$

where g and b are learned scalars. In its per-feature form, g_i and b_i are vectors of length W and are broadcast across the batch axis. *Broadcasting* means that PyTorch treats the missing batch dimension as repeated without allocating B copies of the vector.

The purpose is to keep values in the region where the activation derivative is useful. If a saturating activation receives extremely large magnitudes, its output becomes nearly constant and its derivative approaches zero. A zero derivative prevents upstream weights from receiving a learning signal. The affine normalization learns a scale

and shift that can return those features to the curved portion of the activation.

No statistic of the current minibatch is used. Consequently, prediction for one cosmology does not depend on which other cosmologies happen to share its batch. The same learned g and b are used during training and inference, and the module has no running mean or variance buffer to synchronize with an exponential moving average.

K. Activation factories and learnable activation parameters

The activation configuration names a family. A factory creates one module per activation site. In the default smooth family, the module owns learned shape parameters such as a left-hand slope and a transition sharpness. They are ordinary `nn.Parameter` objects: they appear in `model.parameters()`, receive gradients, move with `model.to(device)`, and are saved in the state dictionary.

A *state dictionary* is a mapping from stable string names to tensors. It contains parameters and registered buffers, not Python source code. Rebuilding therefore creates the architecture first and then loads the recorded tensors into matching names and shapes.

An activation used immediately after an exactly zero-initialized correction layer needs special scrutiny. At initialization its input is exactly zero. If $\phi'(0) = 0$, the chain rule multiplies the correction layer's gradient by zero and that path cannot wake on the first update. The structured-head schema therefore considers both $\phi(0)$, which controls exact identity at initialization, and $\phi'(0)$, which controls whether the head can learn.

L. The convolutional correction head

Cosmic-shear outputs are not an arbitrary flat vector. Values are grouped by correlation type and tomographic-bin pair, and each group varies along angular separation. The convolutional head first changes layout without changing values:

$$(\mathtt{B}, \mathtt{K}) \longrightarrow (\mathtt{B}, \mathtt{N}_{\mathrm{bin}}, \mathtt{N}_\theta).$$

The first operation is a fixed coordinate scatter. A scatter writes each kept output into a declared rectangle position. It is not a learned layer.

A one-dimensional convolution slides the same kernel along the angular axis. For kernel half-width q, an output at angle index u has the schematic form

$$y_{c,u} = \sum_{c'} \sum_{s=-q}^{q} k_{c,c',s}\, x_{c',u+s} + a_c.$$

The index c' ranges over input bins, so the layer can mix bin pairs at nearby angular positions. Reusing k at every u is weight sharing: the number of learned values does not grow with the number of angular positions.

Bins can be ragged, meaning that they contain different numbers of valid angles. They are stored in a rectangle by padding shorter rows. The validity mask marks real positions. Zeroing once at the input is insufficient because a convolutional bias, cross-bin mixing, or feature-wise modulation can make a padded cell nonzero. The mask must be applied wherever the architecture can repopulate invalid cells and again before gathering the correction.

M. The transformer correction head

The transformer head uses the same physical rectangle but a different mixing mechanism. Each bin is a token; a token is one row presented to attention. Attention constructs query, key, and value representations:

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V,$$

then computes

$$\mathrm{Attention}(X) = \mathrm{softmax}\left(\frac{QK^{\mathsf{T}}}{\sqrt{d}}\right) V. \tag{21}$$

The softmax converts each row of similarity scores into nonnegative weights that sum to one. Unlike a convolution, attention can couple distant bins in one layer.

The layout here is not the usual natural-language layout. Angular positions are feature coordinates inside each bin token. A textbook token mask alone therefore cannot hide every ragged angular feature; the implementation needs the same per-block coordinate-validity mask used by the scatter. The mask's shape and meaning are persisted with the artifact because reconstructing it from counts can assign the right number of cells to the wrong coordinates.

N. FiLM: conditioning a head on the cosmology

Feature-wise linear modulation, abbreviated FiLM, lets a correction head depend directly on the encoded cosmology. A small parameter network computes a scale $\gamma(x)$ and shift $\beta(x)$, then applies

$$H' = \gamma(x) \odot H + \beta(x).$$

$\odot$ denotes elementwise multiplication. The scale and shift are broadcast over the declared spatial axis. FiLM is initialized as the identity, $\gamma = 1$ and $\beta = 0$, so adding the option does not perturb the initial trunk prediction.

The ownership distinction is useful: the main trunk learns the full cosmology-to-output map, the convolution or transformer learns structured corrections, and FiLM tells that correction which cosmology it is correcting. Those roles should remain visible in tensor names and artifact metadata.

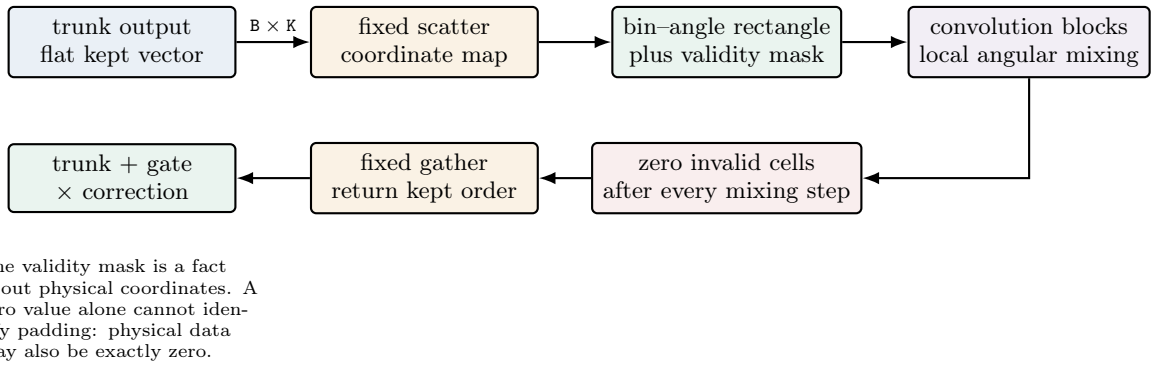

FIG. 4. The convolutional correction-head path. Reshaping and scatter/gather operations are fixed coordinate transformations. Only the convolution blocks and outer gate contain learned values.

VIII. POLYNOMIAL-CHAOS BASES AND NEURAL RESIDUALS

A. Definition: polynomial, basis, expansion, and chaos

A *polynomial* in one variable is a finite sum of nonnegative integer powers:

$$p(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_d x^d.$$

The numbers $a_0, \ldots, a_d$ are coefficients and d is the degree when $a_d \neq 0$. A multivariate polynomial also contains products such as $x_1 x_2$ or $x_1^2 x_3$.

A *basis* is a set of building-block functions whose weighted sum can represent the approximation. This library uses Legendre polynomials on $[-1, 1]$:

$$P_0(x) = 1, \qquad P_1(x) = x,$$
$$P_2(x) = \tfrac{1}{2}(3x^2 - 1), \qquad P_3(x) = \tfrac{1}{2}(5x^3 - 3x).$$

Higher degrees follow the three-term recurrence

$$(n+1)P_{n+1}(x) = (2n+1)xP_n(x) - nP_{n-1}(x).$$

The input parameters are mapped from their fitted training box to $[-1, 1]$ before these curves are evaluated.

Legendre polynomials are *orthogonal* under constant weight on that interval:

$$\int_{-1}^{1} P_m(x)P_n(x)\,dx = 0 \quad \text{when } m \neq n.$$

Orthogonality means different basis directions do not duplicate one another under this inner product. If a uniformly distributed input defines the probability measure $dx/2$, then

$$\int_{-1}^{1} \left[\sqrt{2n+1}P_n(x)\right]^2 \frac{dx}{2} = 1.$$

This is why the implementation multiplies degree n by $\sqrt{2n+1}$. Under the unnormalized measure dx, the unit-norm factor would instead be $\sqrt{(2n+1)/2}$; naming the measure prevents a hidden factor of two.

An *expansion* is the weighted sum of selected basis terms. The word *chaos* is historical terminology from representing functions of random inputs with orthogonal polynomials; it does not mean that the emulator assumes chaotic cosmological dynamics. A polynomial chaos expansion, abbreviated PCE, is therefore an orthogonal-polynomial approximation to the mapping from input parameters to output quantities.

A multivariate PCE uses separable terms. Separable means that one term is a product of one-dimensional basis curves:

$$\Psi_{\boldsymbol{\alpha}}(x) = \prod_{j=1}^{P} \sqrt{2\alpha_j + 1}\, P_{\alpha_j}(x_j). \tag{22}$$

The multi-index $\boldsymbol{\alpha} = (\alpha_1, \ldots, \alpha_P)$ states which degree is used for each parameter. A total-degree rule keeps terms satisfying $\sum_j \alpha_j \leq p_{\max}$. A hyperbolic rule can further discourage terms that combine high degrees in many parameters at once.

B. From one-dimensional curves to a term matrix

For each box-mapped parameter x_j, the basis builder evaluates $P_0(x_j), \ldots, P_{p_{\max}}(x_j)$. Degree zero is the constant function. A multi-index selects one degree from each parameter and multiplies those values according to Eq. (22).

For two parameters and $\boldsymbol{\alpha} = (2, 1)$,

$$\Psi_{(2,1)}(x_1, x_2) = \sqrt{5}\, P_2(x_1)\sqrt{3}\, P_1(x_2).$$

This is an interaction term because both parameters affect it. A total-degree bound of two excludes the term since $2 + 1 = 3$.

Allowed multi-indices are enumerated once in deterministic order. That order identifies coefficient rows and is persisted. Rebuilding under a different enumeration can preserve every matrix shape while assigning coefficients to the wrong polynomials.

The term matrix has shape $(\mathtt{N}, \mathtt{M})$, where M is the number of candidate multi-indices. The coefficient matrix later has shape $(\mathtt{M}, \mathtt{K})$, where K is the number of accepted output modes. Their product produces $(\mathtt{N}, \mathtt{K})$ mode amplitudes; multiplying by the stored mode vectors and adding the output mean produces $N \times K_{\text{out}}$ target predictions. Least-angle regression can select a sparse subset of columns, but selection must terminate when every column is active; re-appending an already-active column does not add a new model.

C. Fitting output modes, not target columns independently

Let $Y \in \mathbb{R}^{N \times K_{\text{out}}}$ be the whitened training targets. The fit first subtracts the row mean $\bar{Y}$ and computes a singular-value decomposition:

$$Y - \bar{Y} = USV^{\mathsf{T}}.$$

SVD is a matrix factorization. Columns of V are orthogonal directions in output space; the corresponding columns of US are one scalar amplitude per training row. Leading modes carry the largest target variance.

For mode k, let $z_k = (Y - \bar{Y})V_k$. Sparse least squares chooses a small set of PCE columns and coefficients c_k so that

$$\Phi c_k \approx z_k.$$

The implementation adds terms greedily and judges each candidate model by leave-one-out error. Conceptually, one row is omitted, the remaining rows are fit, and the omitted row is predicted. The prediction residual sum of squares (PRESS) identity calculates these errors from the ordinary fit's residual and leverage without performing N complete refits. Leverage is the diagonal of the fitted model's projection matrix; it measures how strongly a row influences its own fitted value.

Only modes whose relative leave-one-out error is below `loo_max` enter the base. Rejected modes are left for the neural refiner. Reconstruction is

$$\widehat{Y}_{\text{PCE}} = \bar{Y} + \Phi C V_{\text{kept}}^{\mathsf{T}}.$$

This mode-first design directs polynomial capacity toward the dominant smooth directions of the covariance-whitened target.

Selection maintains a set of distinct active indices. When every term is active, no candidate remains and the loop terminates. Filling unavailable scores with a sentinel and applying unconditional `argmax` can otherwise return column zero and append an already active term.

The basis is refit independently for every training-size sweep point. Reusing a fit from the largest point leaks rows into smaller points and invalidates the learning-curve comparison.

The PCE may serve as a frozen base and the neural network may learn its residual. This composition is called neural PCE, abbreviated NPCE. The word "neural" refers to the learned correction; the polynomial base itself is a linear combination of fixed basis functions after fitting. In additive form,

$$\widehat{t}(x) = t_{\text{PCE}}(x) + \delta_\theta(x).$$

In multiplicative form, the domain of the base and correction must make the product meaningful. The artifact records the allowed input domain and the combination law. Inference may clamp or refuse an out-of-domain point only according to that persisted policy; an undocumented clamp silently changes the mathematical model.

D. Residual and ratio modes

In residual mode, the neural target is

$$\delta(x) = t(x) - t_{\text{PCE}}(x).$$

In ratio mode it is based on t/t_{PCE} or the declared gain form, so zeros and signs of the base require an explicit policy. The code does not silently switch to addition near a small denominator.

The loss reconstructs the complete prediction before computing physical $\Delta\chi^2$. Scoring only correction coordinates can favor a base whose scale makes the residual numerically small without improving the physical prediction.

E. Persistence and inference

The artifact stores input-domain facts, basis family, ordered multi-indices, selected terms, coefficient matrix, combination mode, and correction recipe. The predictor evaluates base and correction on the same encoded input.

Out-of-domain behavior is named and persisted. If clamping is the declared approximation, its bounds are stored and reporting identifies a clamped request. If the model is valid only inside its training domain, prediction raises outside it. Coefficient values alone cannot reveal this policy.

a. Current gap. The current predictor clamps each mapped coordinate to $[-1, 1]$ through an unversioned implementation rule; the artifact does not yet persist a named domain policy. The preceding paragraph is the required closure, not a claim that current artifacts already distinguish clamp from refusal.

IX. LEARNABLE ACTIVATION FUNCTIONS AND THEIR GENERALIZATIONS

An activation function is the scalar nonlinear map placed between learned linear layers. Scalar means that

the function is applied independently to every feature value. Learnable means that the curve itself contains `nn.Parameter` tensors updated by the optimizer. In this library every hidden feature receives its own curve parameters, so two columns in the same layer can learn different left-tail slopes, transition positions, or tail exponents.

The implemented families form a hierarchy. The baseline H has one sigmoid transition and linear tails. Multi-gate adds movable transitions in the bulk. Power-gated H gives the tails a bounded learned exponent. The gated-power family combines both extensions.

A. The baseline H: an identity–Swish interpolation

Define the logistic sigmoid

$$\sigma(z) = \frac{1}{1+\exp(-z)}.$$

Its value lies strictly between zero and one. The baseline activation is

$$H(x) = [\gamma + (1-\gamma)\sigma(\beta x)]\, x. \tag{23}$$

All operations are elementwise when x, γ, and β are feature vectors. Equation (23) can also be written

$$H(x) = \gamma x + (1-\gamma)\, x\sigma(\beta x).$$

The second term is a Swish-shaped branch. Thus γ interpolates between an identity branch and a Swish branch, while β sets how quickly the sigmoid gate changes around zero.

For positive β, the limiting slopes are easy to interpret. Far on the negative tail, $\sigma(\beta x) \to 0$, so $H(x) \sim \gamma x$. Far on the positive tail, $\sigma(\beta x) \to 1$, so $H(x) \sim x$. Both tails remain linear. This differs from `tanh`, whose output approaches a constant and whose derivative approaches zero on both tails.

The implementation initializes $\gamma = 0$ and $\beta = 0$. Since $\sigma(0) = 1/2$, every feature begins as $H(x) = x/2$. The initial map has derivative $1/2$ everywhere and therefore transmits a gradient even when its input is zero. Training then changes γ and β independently for each feature.

B. Multi-gate: learn several transitions in the bulk

H contains one sigmoid transition fixed at the origin and ties the positive-tail slope to one. The multi-gate generalization replaces that single transition with a sum of K learned sigmoids:

$$\begin{aligned} g_K(x) &= a_0 + \sum_{k=1}^{K} w_k\, \sigma[\beta_k(x-\mu_k)]\,, \\ G_K(x) &= g_K(x)x. \end{aligned} \tag{24}$$

Each term has three roles. w_k controls how much the local slope schedule changes, β_k controls how sharp the change is, and μ_k moves its center away from zero. a_0 supplies the left-tail gate value.

The output remains asymptotically linear because every sigmoid is bounded:

$$\begin{aligned} G_K(x) &\sim a_0 x && (x \to -\infty), \\ G_K(x) &\sim \left(a_0 + \sum_k w_k\right) x && (x \to +\infty). \end{aligned}$$

The general form can therefore learn a piecewise-smooth slope schedule through the bulk without inheriting `tanh`'s flat tails.

H is contained in Eq. (24): choose $K = 1$, $a_0 = \gamma$, $w_1 = 1-\gamma$, $\mu_1 = 0$, and $\beta_1 = \beta$. The implemented initialization instead uses a convenient common start: $a_0 = 0$, the first gate has $w_1 = 1$ and $\beta_1 = 0$, and extra gates have zero weight. The total gate is again $1/2$, so the output is $x/2$. Extra centers are spread across a finite interval and become active only if training moves their initially zero weights.

C. Bounded power tail: learn how rapidly the tails grow

Multi-gate changes the slope schedule but retains linear tails. The power generalization changes a different property: the magnitude growth far from zero. First define the signed power transform

$$\psi_p(x) = \operatorname{sign}(x)\frac{(1+|x|)^p - 1}{p}. \tag{25}$$

The base $1+|x|$ is at least one, so a fractional real exponent does not take a power of a negative number. Dividing by p gives $\psi_p'(0) = 1$ for every allowed exponent. Consequently, changing p reshapes the tail while preserving the local first-order behavior at zero.

The exponent is not optimized as an unconstrained number. The stored parameter ρ is mapped through a sigmoid:

$$p = p_{\min} + (p_{\max} - p_{\min})\sigma(\rho). \tag{26}$$

With the default interval $[0.5, 1.5]$, p ranges between square-root-like sublinear growth and mildly superlinear growth. At $\rho = 0$, $p = 1$ and $\psi_1(x) = x$. The power activation is

$$P(x) = [\gamma + (1-\gamma)\sigma(\beta x)]\, \psi_p(x). \tag{27}$$

It therefore recovers H when $p = 1$. This is a bounded generalization: it allows the tail to adapt without allowing an arbitrary exponent to run to a numerically explosive value.

D. Gated power: combine bulk and tail freedom

The two extensions act on orthogonal aspects of the curve. Multi-gate changes where and how the gate transitions in the central region. ψ_p changes the tail-growth law while keeping unit slope at the origin. The combined activation is

$$GP_K(x) = g_K(x)\psi_p(x). \tag{28}$$

It is the most flexible implemented family and also has the most learned shape vectors. More flexibility is not automatically better: it increases the number of parameters, enlarges the search space seen by the optimizer, and makes an activation bake-off more important.

E. Parameter count and qualitative comparison

The count matters because a network creates a fresh activation module at every activation site. For width $W = 128$, $K = 3$, and L sites, multi-gate adds

$$(3K + 1)WL = 1280L$$

learned scalars. Those values are small compared with a 128×128 dense matrix at each site, but they are not free, and their gradients enlarge the optimization problem.

F. PyTorch shape mechanics in the multi-gate code

For an input of shape $(\mathtt{B}, \mathtt{W})$, the multi-gate parameter tensors have shape $(\mathtt{K}, \mathtt{W})$. The code calls `x.unsqueeze(-2)`, which inserts a size-one axis immediately before the feature axis:

$$(\mathtt{B}, \mathtt{W}) \longrightarrow (\mathtt{B}, 1, \mathtt{W}).$$

Broadcasting against $(\mathtt{K}, \mathtt{W})$ then produces $(\mathtt{B}, \mathtt{K}, \mathtt{W})$ without explicitly copying the input K times. The sigmoid is evaluated for every sample, gate, and feature. Summing over axis `-2`, the gate axis, returns $(\mathtt{B}, \mathtt{W})$. Finally the gate is multiplied elementwise by the original input.

This implementation is eager tensor algebra: every expression executes when `forward` is called. It is not a Python generator and does not use `yield`. The parameter tensors live on the same device and use the same dtype as the module because they are registered `nn.Parameter` objects.

G. How the family is selected and persisted

`make_activation(name,n_gates)` returns a factory with the interface `act(dim)` returning a new module. For `relu` and `tanh`, the factory's `dim` argument is unused because those classes have no per-feature parameters. Keeping the same callable interface lets residual-block construction remain uniform.

The resolved artifact stores the family name and gate count. Rebuilding calls the same factory before loading the state dictionary. This order is required because the state dictionary contains tensor values but not the Python class that gives them meaning. A warm start inherits the source artifact's activation family; silently rebuilding identical tensor shapes under a different formula would create a different function while making weight loading appear successful.

H. A fair activation comparison

An activation bake-off holds the data subset, model architecture, optimizer rules, schedule, random-seed policy, and evaluation statistic fixed while changing only the activation family. One final score is insufficient. The driver trains each family at several values of N_{train} and plots the validation failure fraction against training-set size. A family whose curve is lower across the range learns more accurately at the same simulation budget; curves that cross reveal a data-regime dependence.

The comparison also checks liveness. A structured correction head that remains equal to its trunk may still produce a respectable aggregate score. The gate must therefore verify that at least one head weight receives a nonzero finite gradient and changes after an optimizer step. This is especially important for ReLU: its derivative at exactly zero is zero, so a ReLU placed after an exactly zero-initialized final head layer can make the head permanently inactive even though the trunk continues to learn.

X. THE LOSS: WHERE NETWORK ERROR BECOMES A SCIENTIFIC ERROR

The model emits network coordinates. The loss decides whether their error is acceptable in the physical likelihood. It is therefore not an interchangeable training convenience.

A. Plain covariance-weighted loss

For a prediction $\widehat{t}$ and encoded truth t, the geometry can return both to physical kept coordinates. Their residual is

$$r = \widehat{d}_{\text{keep}} - d_{\text{keep}}. \tag{29}$$

The per-sample statistic is

$$\Delta\chi^2 = r^{\mathsf{T}} C_{\text{keep}}^{-1} r. \tag{30}$$

It must be finite and nonnegative, apart from a small negative roundoff band derived from floating-point precision and contraction width.

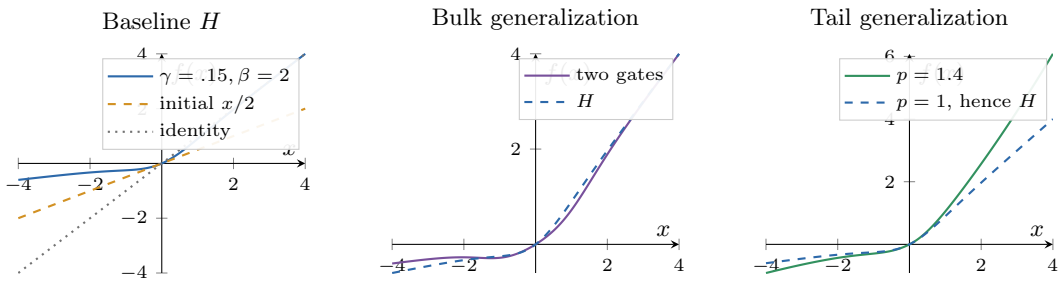

FIG. 5. Representative activation curves. Multiple gates can introduce more than one transition in the bulk. The bounded power changes the tail law while preserving the local slope of the signed transform at zero. The plotted values illustrate mechanisms; learned parameters are feature-specific and need not equal these examples.

Family	Learned scalars per feature	Bulk behavior	Tail behavior	Primary comparison question
H	2	One transition centered at zero	Linear, left slope γ, right slope 1	Is the compact non-saturating default sufficient?
Multi-gate	$3K + 1$	K movable transitions	Linear, both limiting slopes learned	Is one central bend too restrictive?
Power	3	The same single gate as H	Signed $\lvert x\rvert^p$-like growth with bounded p	Does tail curvature, rather than bulk gating, limit the fit?
Gated power	$3K + 2$	K movable transitions	Bounded learnable power growth	Do both restrictions matter together?
ReLU	0	One hard kink; negative side exactly zero	Zero on the left, linear on the right	How does a parameter-free rectifier compare?
tanh	0	Smooth S-curve	Saturates to constants	How does a classical smooth but saturating baseline compare?

TABLE I. The implemented activation families. Per feature means that a layer of width W owns the listed count multiplied by W.

When the geometry is whitened exactly, Eq. (17) reduces this to a sum of squared whitened residuals. The direct physical contraction remains the independent reference used to verify that reduction.

B. The training objective and the reported metric are distinct

The program may transform $\Delta\chi^2$ before averaging it over a minibatch. For example, a square-root objective reduces the influence of catastrophic rows during optimization. This transformation changes how gradients vote; it does not change the reported per-sample $\Delta\chi^2$.

The training reduction must handle an exact fit without producing the undefined derivative $0/0$. A safe square-root implementation preserves the forward value and defines the exact-fit gradient as zero. A negative or nonfinite input is rejected before the transformation.

C. Physics-aware loss wrappers

Several wrappers modify how a prediction is assembled before the same scientific metric is evaluated.

The rule is the same in every case: the reported $\Delta\chi^2$ must refer to the final physical prediction. A preprocessing amplitude is not allowed to multiply the metric accidentally.

D. Frozen bases and packed targets

A frozen base need not run during every optimizer step. During staging, the loss evaluates the base under `torch.no_grad()`, which tells PyTorch not to record operations for backward differentiation. It packs the base output beside the truth:

$$\texttt{target} = [\texttt{base}; \texttt{truth}].$$

The minibatch loss unpacks both pieces and evaluates only the small correction network with gradients enabled. In a refine stage the base becomes live, enters the compu-

Wrapper	Operation before the final residual
Analytic rescaling	Divide out a known approximate cosmology dependence for training and multiply it back before scoring.
Factored IA	Combine neural templates with exact amplitude-polynomial coefficients.
CMB amplitude law	Apply the persisted amplitude and optical-depth law before comparing physical spectra.
NPCE residual	Evaluate a polynomial-chaos base and add or multiply the network correction.
Transfer loss	Combine a saved base emulator with a new correction network.

tation graph, and receives its own optimizer parameter group.

E. The finite contract

A NaN comparison with a positive threshold is false. Without an explicit guard, a diverged row can therefore be counted as below every error threshold and appear perfect. The code rejects nonfinite losses, gradients, validation rows, reductions, and parity comparisons before they can rank or save a model.

The ordering is important:

1. compute the loss,
2. reject a nonfinite loss,
3. run backward,
4. inspect unscaled gradients for finiteness,
5. clip finite gradients when clipping is enabled,
6. update parameters,
7. apply any anchor, and
8. update the exponential moving average.

If a step is rejected, none of the later state may advance.

XI. OPTIMIZER, LEARNING RATE, SCHEDULER, AND CLIPPING

The loss produces derivatives; the optimizer turns them into parameter updates. These are different responsibilities. Backward differentiation writes a gradient into each trainable parameter's `.grad` field. AdamW reads those fields, updates running first and second moments, applies decoupled weight decay to the declared group, and changes the parameters.

A. Batch-scaled learning rate

The configured base learning rate ℓ is anchored at batch size B_0. For actual training batch size B, the resolved rate is

$$\eta = \ell\sqrt{\frac{B}{B_0}}. \tag{31}$$

The square-root rule acknowledges that averaging more samples reduces gradient noise without assuming a fully linear scaling. The factory computes η and injects it into the optimizer. Persisting only ℓ would not state the executed step size; the resolved record carries both the inputs and resulting value.

A warmup ramps the rate from zero to η during the declared early epochs. The ramp is evaluated before the optimizer step it controls. An off-by-one implementation that steps first and then asks whether warmup is complete executes one unintended full-rate update.

B. AdamW state and module-role weight decay

AdamW maintains moving estimates of the gradient and squared gradient for each parameter. Those optimizer tensors are state: rewinding only model weights while retaining moments from a later bad trajectory is not a complete rewind.

Decoupled weight decay applies

$$w \leftarrow w - \eta\lambda w$$

beside the adaptive gradient update. The factory assigns decay by module role. True weight matrices in linear, convolutional, and bin-linear modules may decay. Biases, normalization gains, and activation-shape parameters do not. Tensor rank is not a reliable substitute for ownership: a matrix-shaped activation parameter is still a curve-control value, and a custom weight owner may not match a generic rank heuristic.

a. Current gap. The optimizer factories currently overwrite an explicit `fused: false` with `fused=True` on CUDA and forward that keyword without proving the selected optimizer supports it. They also accept a class whose `step()` requires a closure even though the training loop always calls `step()` with no argument. Constructor compatibility is not execution- protocol compatibility.

b. Required closure. The public surface is bounded to the supported closure-free Adam/AdamW family, or validates an equivalent protocol before data staging. Device-specific `fused` injection occurs only when the selected class supports it, and an explicit user value is preserved. The resolved record reports the class and executed fused state.

C. Plateau scheduling and rewind

`ReduceLROnPlateau` is a metric-driven scheduler. At the end of an epoch, it receives the raw validation median and decides whether improvement has stalled for its patience interval. Calling `step()` without that metric follows a different scheduler protocol and is therefore not a constructible drop-in replacement.

When rewind is enabled and the scheduler cuts the rate, the program restores the best coherent snapshot while keeping the newly reduced rate. A coherent snapshot contains live model weights, optimizer moments, and the EMA state when present. The purpose is to leave a bad basin while preserving the scheduler's decision to take smaller steps.

D. Gradient clipping

Let g denote the concatenated gradient across all trainable parameters. Global-norm clipping performs

$$g \leftarrow g \min\left(1, \frac{c}{\|g\|}\right),$$

where $c > 0$ is the configured ceiling. Multiplying the whole gradient by one scalar preserves its direction. A zero ceiling means clipping is off; negative, NaN, and infinite ceilings are invalid configuration rather than alternative off spellings.

Under float16 loss scaling, the order is scale loss, backward, unscale gradients, reject nonfinite unscaled gradients, clip the unscaled global norm, and step. Clipping the scaled values would make the effective ceiling depend on the current scaler.

XII. TRIM: TEMPORARILY EXCLUDE THE HARDEST ROWS

Trim removes a declared fraction of the largest per-sample losses before forming the training mean. It is a hard selection: excluded rows contribute no gradient in that minibatch. Validation never trims because its purpose is to report the full held-out distribution.

For batch size B and fraction q, the implementation determines the integer number of rows to keep under its documented rounding rule, sorts or selects by detached hardness, and averages only those rows. Detaching the selection statistic means gradients do not flow through the discrete ranking. The loss values that remain still carry their gradients.

The trim fraction can follow a schedule: hold the start value, anneal to the end value, then remain at the end. Cosine interpolation changes with zero slope at both endpoints; linear interpolation changes at constant rate; step interpolation quantizes the ramp; constant holds the start indefinitely. An increasing schedule is valid only if the owner intends it. The validator checks start, end, durations, and shape before training so a NaN cannot disable the branch through a false ordered comparison.

a. Current gap. The shared stepped schedule currently applies `max(end, floored_value)`. That expression is correct only for a decreasing ramp. For an increasing zero-to-one ramp, it returns the final value at the first live epoch and skips the intended anneal. The same helper is reused by trim, BerHu, and EMA, so the owner must be tested in both directions.

b. Required closure. The validator requires strictly increasing step fractions, and the evaluator quantizes then clamps according to the sign of `end-start`. Decreasing shipped schedules remain byte-identical; increasing fixtures prove that the intermediate steps are actually visited.

Trim protects early optimization from a small contaminated tail. It does not justify ignoring the removed rows forever. A nonzero final floor is an explicit robust-objective choice, and the untrimmed validation distribution reveals whether those hard rows remained scientifically unacceptable.

XIII. FOCUS: SMOOTHLY WEIGHT SAMPLES BY HARDNESS

Focus is a continuous alternative to hard trimming. For per-sample chi-square c_i, it uses

$$w_i = \left(\frac{c_i}{c_i + \kappa}\right)^{\gamma(e)}. \tag{32}$$

κ is the chi-square scale at which the base fraction is one half, and $\gamma(e)$ is an epoch-dependent exponent. At $\gamma = 0$, every finite positive base raised to zero gives weight one, so the reduction is the plain mean. Increasing γ emphasizes harder rows.

The weights are detached from autograd before multiplying the losses. This mechanic is essential. Without detachment, a sample could lower the objective partly by changing its own weight rather than by lowering its physical error. With detachment, the current errors decide voting strength, while the gradient acts only through the loss being fitted.

Focus and BerHu can both emphasize a tail. Using them together is a deliberate composition, not two independent defaults. The resolved configuration and banner state both effects so a sweep can distinguish a changed loss curve from a changed sample-weight curve.

XIV. EXPONENTIAL MOVING AVERAGE: A SECOND, SMOOTHED MODEL STATE

After a successful optimizer step, EMA updates an averaged parameter copy

$$\bar{\theta} \leftarrow \beta\bar{\theta} + (1-\beta)\theta, \qquad \beta = 1 - \frac{1}{HS}. \tag{33}$$

H is the requested horizon in epochs and S is the executed number of optimizer steps per epoch. Expressing the horizon in epochs makes its physical training duration comparable across batch sizes.

The averaged copy is not part of the forward graph that produces the current batch gradient. It is updated after the live optimizer step under no-gradient semantics. Evaluation and best-epoch selection may use the averaged state, while the plateau scheduler intentionally watches the raw model median. The artifact must say which state produced its selected metrics.

An annealed horizon avoids carrying early poor weights. During its hold, the average remains inactive according to the resolved schedule. After the live point, the horizon grows toward its requested value. Saving, rewind, and resume treat θ, optimizer state, and $\bar{\theta}$ as one coordinated training record.

XV. LOADERS, MINIBATCHES, AND THE TRAINING LOOP

The staged source is a host-side description. The training loop needs fixed size tensors on the model's device. Loaders bridge those representations.

A. A loader is a closure

A closure is a function that retains values from the scope in which it was created. `build_loaders` returns functions with the interface

`load(rows)` $\longrightarrow$ device tensor.

The closure remembers whether its source is GPU-resident, in host RAM, or file-backed. The training loop does not branch on storage.

There are two loaders per source. `load_C` returns encoded parameter rows. `load_dv` returns encoded targets. Both accept row indices and return tensors on the compute device.

A common transfer line has the form

```
t_tr = torch.from_numpy(C_tr).float().to(device)
```

Read it left to right. `torch.from_numpy(C_tr)` creates a CPU tensor that initially shares storage with the NumPy array when its dtype is already compatible. `.float()` requests PyTorch `float32`; it allocates a converted copy when the source is not already `float32`. `.to(device)` places the resulting tensor on the selected CPU, CUDA GPU, or Apple MPS device and copies storage when the device changes. The final name `t_tr` is a tensor; it is not a lazy iterator and does not keep a deferred transfer for the first minibatch.

B. Three storage regimes

On CUDA, a host tensor may be pinned before transfer. Pinned memory cannot be paged out by the operating system, which allows an asynchronous device copy. Pinning is a host-memory property; it does not move the tensor to the GPU.

C. Fixed batch shapes

Compiled PyTorch graphs work best with stable tensor shapes. Training uses full batches. Validation pads its final short batch with copies of a valid row, runs the fixed-size graph, and discards the padded scores. Padding is not allowed to enter the reported median, mean, or threshold fractions.

For one batch,

$$X_b = \texttt{load\_C(rows)}, \quad T_b = \texttt{load\_dv(rows)},$$
$$\widehat{T}_b = \texttt{model}(X_b).$$

`model(X_b)` uses positional syntax because the tensor is the direct subject of the call. Configuration-heavy calls use named arguments.

D. Optimizer and scheduler

The optimizer owns parameter updates. Parameters are divided by module role: true linear and convolutional weight matrices may receive weight decay; biases, affine gains, and activation-shape parameters do not.

The scheduler changes learning rates. `ReduceLROnPlateau` requires a validation metric in `scheduler.step(metric)`. A scheduler with a different protocol cannot be substituted merely because its constructor fits the same factory.

E. One optimizer step

For a normal full-precision batch, the executed sequence is

```
optimizer.zero_grad()
prediction = model(inputs)
loss = lossfn.loss(
    prediction,
    target=targets,
    ...)
loss.backward()
optimizer.step()
```

`zero_grad` clears gradients accumulated by earlier backward calls. `backward` traverses the recorded computation graph and adds each parameter's derivative to its `.grad` field. Gradients accumulate by default, so omitting `zero_grad` changes the algorithm.

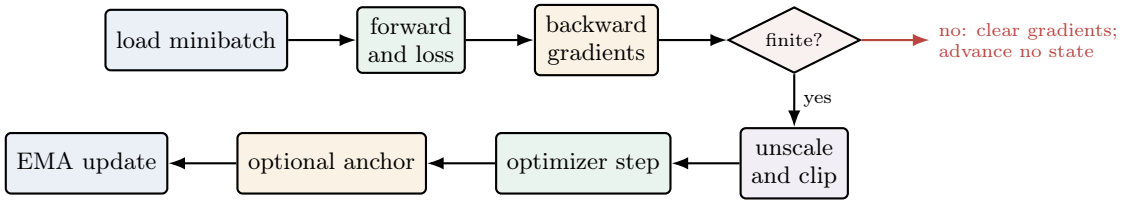

FIG. 6. One accepted training step. The order is part of the algorithm. Finiteness is checked before any persistent state advances; EMA is last because it summarizes the accepted post-step parameters.

Regime	Persistent storage	Work performed for a minibatch
Device resident	Encoded parameters and targets on GPU/MPS	Gather rows by device indexing.
Host RAM stream	Encoded parameters resident; raw targets in a NumPy array	Copy selected target rows to the device and encode them.
Disk stream	Encoded parameters resident; raw targets in a memmap	Read a bounded host block, copy it to the device, and encode it.

Mixed-precision float16 training requires loss scaling. Scaling makes small gradients representable during backward. The gradients are unscaled before finiteness checks and clipping. Bfloat16 has a wider exponent range and does not automatically use the float16 scaling protocol.

`model.train()` and `model.eval()` select a module's training or evaluation behavior; they do not enable or disable gradient recording. `torch.no_grad()` separately disables construction of the backward graph inside its context. Evaluation uses both `eval()` and `no_grad()`: the first chooses inference behavior, and the second avoids retaining activations that no backward pass will consume. Returning to `train()` before the next epoch restores training behavior.

a. Current gap. The current Apple MPS branch selects float16 autocast but does not construct a gradient scaler. Small early gradients—especially those produced by zero-start correction heads and soft gates—can underflow to exact zero while all losses remain finite. CUDA bfloat16 is not evidence that this distinct float16 path is safe.

b. Required closure. An MPS float16 run either uses a supported scaler with the order scale, backward, unscale, finite check, clip, step, anchor, EMA, or refuses the mode with a teaching error. No persistent state advances after a skipped/nonfinite step. The resolved artifact records compute dtype and scaler policy.

F. Epoch metrics and model selection

After every epoch, `eval_val` produces one $\Delta\chi^2$ per validation row. It then reports a median, mean, and the fraction above each configured threshold. The even-sample median is the ordinary 50th percentile, not the lower of the two central samples.

The best epoch is selected by the primary threshold fraction, with the median as the tie-breaker. The saved best record must contain the exact weights that produced the recorded metrics. If an exponential moving average is used, evaluation, selection, restoration, and persistence must all refer to that same averaged state.

XVI. TWO-PHASE TRUNK AND CORRECTION-HEAD TRAINING

A model with a correction head can run two phases. The trunk phase bypasses and freezes the head. The head phase may freeze the trunk or train trunk and head jointly, according to the resolved configuration. Each phase receives its own optimizer and learning-rate schedule. A frozen parameter has `requires_grad=False`; PyTorch does not construct its gradient.

The phase boundary is a state transition, not merely a banner. Tests count trainable trunk and head parameters and verify which weights can change.

A. Phase one: learn the complete baseline map

The trunk predicts the complete target on its own. During the trunk phase, the correction head is bypassed and its parameters do not receive gradients. The optimizer contains only the parameters intended for this phase.

This produces a scientifically meaningful intermediate model. If the head later fails to improve, the trunk still represents a full emulator rather than an incomplete feature extractor. The head is therefore a correction, not the only path from inputs to outputs.

B. Phase two: expose the structured correction

At the phase transition, the head enters the forward graph. With a frozen trunk, its input is treated as fixed and only head parameters are optimized. With joint training, both sets have `requires_grad=True` and appear in the new optimizer.

Each phase resolves its own learning rate, warmup, scheduler, loss transform, focus/trim controls, and EMA policy according to the configuration precedence. The transition creates fresh optimizer state for the parameter groups it will update; it does not accidentally carry trunk-only Adam moments into a differently grouped head optimizer.

The correction's final learned layer begins at zero so phase-two epoch zero equals the trunk. The outer gate begins representably nonzero, and the head activation must transmit a derivative at zero. These three facts together give exact initial identity and a live first gradient.

C. Frozen does not mean absent

A frozen trunk still executes forward to provide the head input. PyTorch does not store its backward activations when the computation is placed under no-gradient semantics, reducing memory. Its registered buffers and evaluation mode still matter. The code must not mutate the source module in a way that changes later prediction.

Joint-mode evidence counts trainable trunk and head parameters, verifies optimizer membership, checks finite nonzero gradients after a discriminating batch, and observes weight changes. Timing alone is only supporting evidence: workstation load can make a frozen epoch slow or a joint epoch fast.

D. What is selected and saved

Validation after each phase evaluates the same final physical metric. The best trunk snapshot seeds phase two. The best final snapshot contains the exact model state that produced its recorded metrics, including EMA when enabled.

The artifact records the resolved phase configuration and final architecture, not two ambiguous top-level dictionaries. A single-phase model resolves or refuses phase-shaped keys according to its public rule rather than forwarding them into a constructor with no head.

XVII. STARTING FROM A SAVED EMULATOR: FINE-TUNING AND TRANSFER

Random initialization discards knowledge already stored in a trusted artifact. The library provides two reuse strategies. Fine-tuning adapts the source network itself. Transfer freezes the source and trains a separate correction. Both obey the same epoch-zero contract:

$$\widehat{d}_{\text{new}}(\boldsymbol{\theta}; e = 0) = \widehat{d}_{\text{source}}(\boldsymbol{\theta})$$

for every parity fixture before the first optimizer update.

A. Fine-tuning extends the existing function

The fine-tune configuration names a source artifact and omits a new model description. Architecture, activation family, parameter order, and output geometry are inherited from the source. Repeating them in YAML would create two potentially conflicting authorities.

If the new table contains additional input parameters, the input projection is expanded in a way that gives the new columns zero initial influence. The old input columns, hidden weights, output projection, and output geometry retain their source values. Thus the initial function agrees exactly on the old parameter subspace. A smaller learning rate can then adapt all trainable weights to the extended data.

Output geometry is pinned because changing its center or whitening basis would change the numerical network coordinates represented by the loaded output weights. Centering cancels in a physical residual only when encode, network, and decode agree on the same center. Recomputing one side from new data breaks epoch-zero parity even if the physical target family is unchanged.

An anchor, when supported by the validated public configuration, penalizes movement from selected source weights:

$$L_{\text{total}} = L_{\text{data}} + \lambda \sum_{j \in M} \|\theta_j - \theta_{j,0}\|^2.$$

M is a named mask of matched parameters, not a positional guess. The implementation reports matched, masked, frozen, and unexpected keys. Anchor application follows the optimizer step and precedes EMA so the averaged model summarizes the actual anchored trajectory.

a. Current gap. The public fine-tune validator currently refuses `finetune.anchor` before training begins. The downstream mask and update machinery exists, but it is not a usable feature while its configuration door is closed. Unanchored fine-tuning is the current public path; `transfer.refine.anchor` is a different control owned by transfer refinement.

b. Required closure. Opening the key requires an executed from-config test, finite nonnegative validation, canonical source-to-new parameter mapping, honest matched/masked/ frozen/unexpected reporting, and save/rebuild proof of the executed anchor. The preceding equation describes that closure, not a currently accepted YAML key.

c. Current sweep-lifecycle gap. Fine-tune preparation changes the loaded source parameter geometry to its live device state and does not restore the original representation afterward. A family sweep reuses one experiment across points, and the next point clones the source as it currently exists rather than reloading an immutable source. Thus point order can affect later warm starts even though every point names the same artifact.

d. Required closure. Each point receives the same immutable source snapshot. Any temporary in-place device/liveness transition is restored in a `finally` path before the next point, and a digest plus two-point forward/reverse-order gate proves order independence.

B. Transfer keeps the base frozen

Transfer creates a new correction network that sees the complete new parameter space. The base receives only the named subset it was trained on. During ordinary correction training, base evaluation is performed once per training row under `torch.no_grad()` and cached with the target. No gradient graph or base activation storage is needed for each minibatch.

Two composition forms are supported conceptually:

$$\begin{aligned} \text{sum:} \quad & y = y_0 + r, \\ \text{gain:} \quad & y = y_0(1 + r). \end{aligned}$$

The composition space is equally important. In physical space, y_0 and r are physical output values. In whitened space, they are geometry coordinates. Addition, multiplication, and nonlinear output laws do not generally commute, so the family validator accepts only combinations whose meaning is defined.

For factored intrinsic-alignment models, composition occurs per template before the exact amplitude polynomial combines the templates. Correcting only the already-combined vector would make the learned correction depend on the particular amplitude used to construct a target.

C. Target packing is an internal storage contract

When the base is frozen, staging can concatenate cached base and truth arrays:

$$T_{\text{packed}} = [T_{\text{base}} \mid T_{\text{truth}}].$$

The vertical bar denotes concatenation along the last, feature axis. It does not mean logical or. The loss owns the exact split width and unpacks the two blocks. A loader treats the packed target as an opaque tensor; duplicating the split rule in the loader would create another authority.

Packing is eager: the cached base is computed for the selected rows during staging. The resulting array may be file-backed when its memory cost is too large. Its temporary-file lifecycle belongs to the experiment so repeated sweep points release superseded files rather than waiting for process exit.

D. Optional joint refinement

For a supported family, refinement can unfreeze the base after the correction has converged. The optimizer then contains separate groups: correction parameters at the run learning rate and base parameters at a scaled rate. A base anchor limits drift from the original source.

The executed update order is optimizer, base anchor, then EMA. Moving EMA before the anchor would average a state that is never actually served. The artifact records original base tensors, refined tensors, and a truthful drift metric computed over the named trainable parameter set. Buffers and padding indices are not silently diluted into a quantity labeled weight drift.

E. Artifact self-containment

A transfer artifact embeds the base facts it needs at inference; it does not remain dependent on an external path that may later point to different weights. Pair integrity binds weight tensors to the HDF5 recipe, while an implementation identity records the executable law required to interpret them. These are distinct: the correct two files can still be interpreted by the wrong future formula if implementation identity is ignored.

XVIII. THE FIVE OUTPUT FAMILIES

The training machinery is shared, but the physical object predicted by the network changes by family.

XIX. COSMIC SHEAR: A MASKED, COVARIANCE-COUPLED DATA VECTOR

The cosmic-shear geometry reads a likelihood covariance, mask, data-vector center, and probe layout. A model predicts only the unmasked entries. The loss scatters their residual into the full vector and contracts the kept covariance block. Structured heads use the persisted angular/bin map.

A. What one target row means

One generated row is the set of two-point correlation values produced by one cosmology and nuisance-parameter point. The entries are not independent labels. They are ordered by correlation type, tomographic-bin pair, and angular separation. A likelihood mask keeps

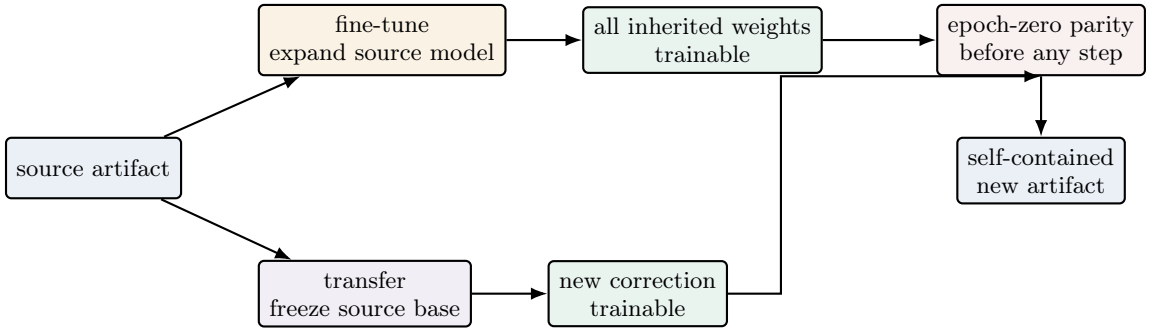

FIG. 7. The two warm-start strategies. They differ in which parameters may move, but both must reproduce the source function before training and publish a self-contained result afterward.

Family	Raw target	Output axis	Physical reconstruction
Cosmic shear	Masked 3×2-point data vector	Probe block and angular bins	Dense covariance geometry; optional IA templates.
Scalar	Named derived parameters	Output-name list	Per-output center and scale.
CMB	One spectrum C_ℓ	Multipole ℓ	Per-ℓ covariance scale and amplitude law.
Background	$H(z)$ or $D_M(z)$	Redshift z	Target-law inverse; distances derived from $H(z)$.
Matter power	$P_{\rm lin}(z,k)$ or nonlinear boost	Stored (z,k) grid	Target-law inverse and analytic-base multiplication.

the entries used in the analysis, and a covariance assigns coupled uncertainty to the kept vector.

Let $\mathbf{d}_{\rm full} \in \mathbb{R}^D$ be the generated row and let $m \in \{0,1\}^D$ be the analysis mask. The kept vector is not discovered by deleting zeros; it is selected by explicit indices

$$I = \{j : m_j = 1\}, \qquad \mathbf{d}_{\rm keep} = \mathbf{d}_{\rm full}[I].$$

Physical values can be zero, so value equality cannot identify masked positions. The sequence I is part of the geometry state.

B. Covariance geometry and the physical score

The likelihood covariance C is first restricted to the kept coordinates, $C_{\rm keep} = C[I,I]$. A geometry basis transforms the target into well-scaled network coordinates. Whether the implementation uses an eigenbasis, block structure, or another stored factor, the acceptance identity remains

$$\|\widehat{t} - t\|^2 = (\widehat{\mathbf{d}}_{\rm keep} - \mathbf{d}_{\rm keep})^{\mathsf{T}} C_{\rm keep}^{-1} (\widehat{\mathbf{d}}_{\rm keep} - \mathbf{d}_{\rm keep})$$

for the plain whitened form.

The data-vector center changes the zero point the network learns. It does not change a physical residual because the same center is subtracted from truth and prediction. This cancellation holds only when both use the identical persisted geometry.

C. Likelihood order and angular order

The likelihood's flat order is convenient for covariance contraction but not necessarily for a structured head. The convolutional and transformer heads use a second, fixed coordinate map that groups values by physical bin and angular position. This is a permutation/scatter operation, not a learned projection.

The geometry stores both directions:

$$\text{kept likelihood order} \underset{\text{gather}}{\overset{\text{scatter}}{\rightleftarrows}} \text{bin–angle rectangle with validity mask.}$$

The correction returns through the inverse map before it is added to the trunk result and scored. Persisting only bin lengths is insufficient when kept coordinates contain holes; the exact coordinate slots are the identity.

D. Factored intrinsic-alignment dependence

Intrinsic alignments are correlations in galaxy shapes that do not arise from gravitational lensing. When their amplitude dependence is a known polynomial, the model predicts templates rather than learning that polynomial from examples.

For the nonlinear alignment model, abbreviated NLA, with amplitude A_1, three network templates give

$$\xi(\boldsymbol{\theta}, A_1) = K_0(\boldsymbol{\theta}) + A_1 K_1(\boldsymbol{\theta}) + A_1^2 K_2(\boldsymbol{\theta}).$$

Here K_0 is the gravitational-lensing–gravitational-lensing contribution (GG), K_1 carries the gravitational-lensing–intrinsic-shape cross term (GI), and K_2 carries the intrinsic–intrinsic term (II).

For the more detailed tidal-alignment and tidal-torquing model, abbreviated TATT, the model emits the ten templates multiplying the allowed combinations of a_1, a_2, and b_{TA}. The amplitude parameters are appended to the encoded input record for the loss but are not fed through the template-producing trunk. The combine step is exact and differentiable with respect to the emitted templates.

At inference, the same saved template order and polynomial recombine the prediction. Reordering templates can preserve tensor shape while changing the physical function, so template identity belongs in artifact state and round-trip gates.

E. Training and diagnostic readouts

Validation reports one $\Delta\chi^2$ per held-out cosmology. The headline failure fraction is the fraction above a declared threshold such as 0.2. A diagnostic report also examines the distribution across parameter space, local behavior, and the physical coverage triangle.

The mean-predictor baseline is important. A smoke run must beat the score obtained by always returning the training center; otherwise the network may be disconnected while every file and plot still exists. Structured-head tests additionally prove that head parameters receive gradients and change, because the trunk alone can hide a dead correction in aggregate metrics.

XX. SCALAR EMULATORS: NAMED DERIVED QUANTITIES

For scalar emulation, both input and target columns come from one parameter table. Names, rather than a fixed slice, locate the outputs. The prediction API returns a dictionary from output name to physical value.

A. Why a scalar emulator is a separate family

A scalar artifact maps sampled parameters to a small set of named derived quantities such as H_0, Ω_m, or r_{drag}. It does not have a CosmoLike data vector, likelihood mask, or angular coordinate. Treating it as a width-one cosmic-shear vector would import irrelevant covariance and layout assumptions.

The generated parameter table may contain bookkeeping columns, sampled parameters, derived outputs, and diagnostic quantities. The loader resolves sampled and output columns from names. It then proves that every requested name appears exactly once and that input/output sets obey the declared relationship. A fixed slice is unsafe because adding one derived column changes every later position while the table remains rectangular.

B. Target geometry

For output q, the scalar geometry stores a center μ_q and scale s_q:

$$t_q = \frac{q - \mu_q}{s_q}, \qquad q = \mu_q + s_q t_q.$$

Each scale is tested after conversion to the storage/model dtype. A positive float64 scale smaller than the least representable positive float32 value becomes zero when stored; checking only before the cast would leave a division by zero in encode.

Correlated scalar outputs may use a covariance-aware geometry when the declared metric requires it. The same round-trip and physical-score rules apply, including minimum sample count and positive-definiteness. A one-row table cannot define a sample covariance and is refused with an explanation.

C. Training and prediction

The network output has shape $(\mathtt{B}, \mathtt{Q})$ for Q named quantities. A single-output run still retains its feature axis; it is $(\mathtt{B}, \mathtt{1})$, not $(\mathtt{B})$. Preserving the axis avoids batch-size-one squeeze ambiguity.

Prediction begins with a parameter dictionary. The predictor orders values by the artifact's saved input names, encodes, evaluates, decodes, and returns a dictionary keyed by saved output names. Cobaya's scalar adapter uses those names to advertise what it can provide and inserts the calculated values into the current state.

An analytic smoke such as

$$\omega_m = \Omega_m \left(\frac{H_0}{100}\right)^2$$

is particularly strong. It supplies truth at off-center cosmologies without calling the production helper. The off-center requirement prevents the degenerate implementation that always returns μ_q from passing.

D. What the scalar artifact persists

In addition to the shared model recipe, the artifact stores ordered input names, ordered output names, centers/scales or covariance factors, dtype, and family kind. The adapter refuses a non-scalar artifact even if its output width happens to equal Q. Family kind is a semantic fact, not an inferred shape.

XXI. CMB SPECTRA: MULTIPOLES, AMPLITUDE LAWS, AND COVARIANCE

One artifact predicts one spectrum type. The geometry stores the exact multipole sequence, units, fiducial spectrum, and diagonal uncertainty. The adapter may assemble several independently trained spectra, but it must prove their requested multipole layouts rather than assuming equal widths imply equal axes.

A. Four related but distinct spectra

The generator writes temperature TT, temperature–polarization TE, polarization EE, and lensing-potential $\phi\phi$ spectra. Each array column belongs to an explicit multipole L or ℓ. The artifact stores that coordinate sequence and a convention describing whether the values are raw C_ℓ or a scaled representation.

Convention is not recoverable from magnitude alone. For lensing,

$$\frac{[L(L+1)]^2}{2\pi} C_L^{\phi\phi}$$

can differ from raw $C_L^{\phi\phi}$ by many orders of magnitude. A producer and consumer that both make the same wrong assumption can round-trip perfectly. Identity evidence therefore compares the production contraction with an independent known-answer calculation under fixtures where the two representations are genuinely distinct.

B. From-fiducial diagonal geometry

For spectrum s at multipole ℓ, the current diagonal geometry is

$$t_{s\ell} = \frac{C'_{s\ell} - \overline{C}'_{s\ell}}{\sigma_{s\ell}}, \qquad \sigma_{s\ell} = C^{\mathrm{fid}}_{s\ell} \sqrt{\frac{2}{2\ell+1}}.$$

Here C' is the training target after any amplitude law, and $\overline{C}'$ is its training-row mean. The fiducial spectrum supplies the cosmic-variance scale, not the center. Mean and fiducial are both saved because they have different owners. A zero or non-representable $\sigma_{s\ell}$ is rejected or handled only through a physically declared zero-weight rule; dividing by an accidental zero is not a valid mask. The present constructor requires a strictly positive fiducial entry, which is an important explicit convention for a TE spectrum that can cross zero.

The diagonal loss becomes a sum of squared encoded residuals. Non-Gaussian covariance introduces band coupling and weights derived from raw spectra. A small banded fixture compares each accumulated contribution with explicit per-L loops. Zero-band input provides a physical exact-zero control, while a width-three nonconstant band exercises the projection stencil.

C. Amplitude and optical-depth law

CMB spectra contain a strong approximate dependence on primordial amplitude and optical depth. The family can factor a known law from the training target so the network learns a gentler residual. The executed `as_exp2tau_ref` law uses named finite reference values and forms

$$g(\boldsymbol{\theta}) = \frac{A_{s,\mathrm{ref}}}{A_s} \exp[2(\tau - \tau_{\mathrm{ref}})]\,,$$

so the factor is dimensionless and equals one at the reference cosmology. Training and decoding use inverse operations:

$$t = \mathcal{G}(gC)\,, \qquad \widehat{C} = \frac{\mathcal{G}^{-1}(\widehat{t})}{g},$$

where $\mathcal{G}$ is centering and diagonal whitening. The artifact persists the amplitude parameterization, parameter names, and both reference values. The retired `as_exp2tau` law used $\exp(2\tau)/A_s$, a dimensionful factor of order 10^8; loading an artifact that names it raises with a retraining instruction because metadata cannot repair weights fitted in the old target space.

The physical metric must compare $\widehat{C}$ with C. Since encoded prediction and truth both carry g, their encoded difference also carries g; subtraction does not cancel a common multiplicative factor. The loss therefore divides the residual by g before summing squares. Omitting that step would report $g^2\Delta\chi^2$ and make model selection depend on A_s and τ even at fixed physical error.

D. Roughness penalty

An optional roughness term discourages high-frequency error across neighboring multipoles. It acts on the factor-corrected residual in the declared spectrum coordinates, not on a representation whose amplitude law changes its strength. A constant residual has zero roughness. An alternating or rapidly varying fixture has a larger value. Turning the coefficient off must leave the original objective exactly unchanged.

Roughness is a training regularizer, not the headline physical $\Delta\chi^2$. Validation reports the latter after full reconstruction. This separation lets a sweep compare regularization while preserving one scientific score definition.

E. Serving spectra to Cobaya

Cobaya first calls `must_provide` with requested spectra and multipole ranges. The adapter validates the complete sequence against each artifact. Later, `calculate` gathers current cosmological parameters, evaluates each predictor, and inserts the physical spectra into state under the expected keys.

Several artifacts can contribute different spectrum blocks. Assembly proves that their coordinate layouts are compatible and destination blocks do not overlap. Blind concatenation would accept equal widths with shifted multipoles. Strict weight loading and artifact-axis validation occur before the first sampled calculation.

F. Diagnostics and evidence

Family diagnostics plot residuals and error summaries as functions of multipole and spectrum type. They handle TE zero crossings without dividing unmasked truth by zero. When a fractional statistic is undefined at a crossing, the report uses an explicit availability status or a declared mask rather than publishing NaN as an ordinary number.

The identity gate owns exact law, covariance, roughness, structured-head, and save/rebuild claims. The smoke runs real CAMB, the covariance script, training, Cobaya lifecycle, and diagnostics. Their separation prevents the real generator from becoming its own mathematical reference.

XXII. BACKGROUND FUNCTIONS: TRAINED GRIDS AND DERIVED DISTANCES

The background network predicts a function on a stored redshift grid. For an $H(z)$ artifact, the shared physics module computes

$$\chi(z) = \int_0^z \frac{c}{H(z')}\, dz', \tag{34}$$

$$D_A(z) = \frac{\chi(z)}{1+z}, \qquad D_L(z) = \chi(z)(1+z). \tag{35}$$

for a flat model. These derived quantities are deterministic physics, not additional neural outputs. Queries are valid only inside the trained domain recorded by the artifact.

When H is stored in $\mathrm{km\,s^{-1}\,Mpc^{-1}}$ and the speed of light c is in $\mathrm{km\,s^{-1}}$, the ratio c/H has units of Mpc; redshift is dimensionless, so the integral returns a comoving distance in Mpc. If the adapter exposes H in inverse Mpc instead, it first applies the named unit conversion. Mixing these conventions can return a finite result wrong by a factor of c, so the quantity–unit pair is an artifact fact.

A. Quantities and grid identity

A background artifact represents one named quantity such as $H(z)$ or transverse comoving distance $D_M(z)$ on a one-dimensional redshift grid. The configuration declares both quantity and units. A generic string is not enough: the validator checks an allowed quantity–unit pair, and the adapter requires the exact pair it knows how to serve.

The grid is an ordered numerical sequence

$$0 \le z_0 < z_1 < \cdots < z_{N_z-1}.$$

It is saved with the geometry. Input arrays must have last-axis width N_z, but width is only the first check. The generated axis sidecar, geometry state, validation grid, and requested adapter coordinates must agree in value and order.

B. Target laws

A direct target law may center and scale the physical quantity. A `log_offset` law instead uses

$$t(z) = \log[y(z) + a]$$

before the diagonal geometry, with inverse

$$y(z) = \exp[t(z)] - a.$$

The offset a is a finite numeric artifact fact. It must keep every training and validation value inside the logarithm's positive domain. A NaN offset can poison center and scale while bypassing ordinary ordered comparisons, so validation rejects nonfinite values before target construction.

The target law is owned by the geometry and reused at inference. An adapter does not reproduce the exponential or offset independently. This keeps save/rebuild and direct prediction on the same formula.

C. Piecewise trained windows and the desert

Some background products are trained in separated redshift windows rather than one continuous interval. The union of those windows is the trained domain. The gap between them is a *desert*: no training examples support interpolation there.

A numerical interpolation routine can return a smooth, finite value across the desert. That does not make the value trained. The public range checker first classifies the requested redshift by the persisted window set and raises for the gap. Boundary values have one explicit inclusion rule shared by startup validation and runtime getters.

D. Distances derived from $H(z)$

When $H(z)$ is the emulated primitive, comoving distance is derived by integration. For a flat model,

$$\chi(z) = c \int_0^z \frac{dz'}{H(z')}.$$

Angular-diameter and luminosity distances follow

$$D_A(z) = \frac{\chi(z)}{1+z}, \qquad D_L(z) = (1+z)\chi(z).$$

The numerical quadrature uses the trained domain. It cannot begin an integral below the artifact's minimum redshift by extrapolating H and still claim the result is emulator-supported. The load-time domain contract therefore proves that the stored grid covers every interval needed by the public distance getters, including the anchor at zero when the formula needs it.

a. Current gap. The current adapter can load a positive-minimum $H(z)$ grid and extrapolate both H and the integral below the trained domain. The extrapolated values can remain smooth, finite, positive, and monotonic, so value-only checks do not detect that the requested interval was never trained. Until the load-time domain refusal is gate-proven, a background artifact used for distances must be inspected manually for coverage from the required zero anchor.

b. Required closure. Artifact loading refuses a grid whose trained domain cannot support every advertised distance integral. This domain verdict occurs before constructing interpolators or exposing getters and is separate from checking that returned numbers happen to look physical.

An odd number of intervals requires the quadrature's explicit endpoint rule. The identity fixture uses low-degree polynomials whose integrals are known exactly, so a coefficient error cannot hide behind a loose percent tolerance.

E. Serving and diagnostics

The Cobaya adapter advertises the background quantities it can provide, validates requested redshifts, evaluates the primitive predictor, and calls the shared physics functions for derived quantities. One sampled cosmology owns one state update; a rejected provider result cannot expose arrays from the previous cosmology.

Diagnostics compare the primitive and derived functions with truth across redshift. They report absolute or well-defined relative errors, name unavailable quantities explicitly, and mark trained-window boundaries. The identity gate uses closed-form flat-ΛCDM controls; the smoke compares the full generated, trained, and served path with CAMB's own background.

XXIII. MATTER POWER: TWO-DIMENSIONAL SURFACES AND ANALYTIC BASES

The matter-power target has shape (N, N_z, N_k) and is flattened with redshift as the outer coordinate. A target law may divide by a known analytic base and take a logarithm. Staging performs this transformation in bounded row chunks and may thin the k axis before materializing the final width.

At inference, the predictor reshapes the network row to (N_z, N_k). The consumer multiplies the persisted law's analytic base back. Constant low-k boost points may be pinned only when the quantity, coordinates, and law make that constant physically valid.

A. Two quantities with different meanings

The family can emulate linear power $P_{\rm lin}(k,z)$ or a nonlinear boost

$$B(k,z) = \frac{P_{\rm nl}(k,z)}{P_{\rm lin}(k,z)}.$$

Both are rectangular surfaces but they do not share every physical invariant. The boost B is dimensionless. Linear power carries volume units fixed by the declared wavenumber convention: if k is in Mpc^{-1}, then P is in Mpc^3; an h-scaled convention changes both axes together. The artifact and request boundary therefore name units rather than inferring them from numerical magnitude. For example, a boost may approach one in a declared low-k region; linear power does not. A constant-column pin is therefore permitted only when the artifact quantity, law, coordinates, and observed center establish that identity.

The family kind and quantity are explicit state. A consumer cannot infer them from file name or array scale. Two artifacts assembled into a hybrid inference must prove compatible parameter names and exactly matching (z,k) grids.

B. Flattening convention

The generated payload has shape $(\mathtt{N}, \mathtt{Nz}, \mathtt{Nk})$. Training flattens each surface to $(\mathtt{N}, \mathtt{Nz} * \mathtt{Nk})$ under one documented order: redshift is the outer coordinate and wavenumber is the inner coordinate. In index form,

$$j = i_z N_k + i_k.$$

The inverse reshape uses the same N_z, N_k, and order. A transposition preserves the total width and can preserve global statistics, so the artifact stores actual axis arrays and the identity gate uses a surface whose value depends differently on z and k.

C. Target laws and analytic-base placement

The no-law path learns the physical surface after ordinary centering and scaling. A base-law path may factor a fast analytic approximation:

$$R(k,z) = \frac{P_{\rm truth}(k,z)}{P_{\rm base}(k,z)}, \qquad t(k,z) = \log R(k,z).$$

Inference reconstructs

$$\widehat{P}(k,z) = P_{\rm base}(k,z)\exp[\widehat{t}(k,z)].$$

The base is evaluated at the same cosmology and coordinates as the target. Its implementation identity, input names, domain, and version-independent facts are part of the artifact contract. A base from a different implementation can have the same file shape and a different scientific function.

For interpolation within an analytic base, both redshift and wavenumber grids are finite, strictly increasing, and large enough for the declared spline order. $k > 0$ is required before $\log k$. Tail extrapolation parameters are explicit, finite, and ordered. The code retains the vendored interpolation algorithm on valid inputs; validation makes its previously implicit preconditions public.

D. Bounded staging

Production surfaces are wide. Materializing every selected raw row in float64 before thinning can exceed host memory even when the final float32 training matrix would fit. Bounded staging therefore operates in chunks:

1. read a bounded set of selected disk rows,
2. cast to the payload dtype used by training,
3. apply the base and logarithmic law,
4. thin the k axis according to the declared stride,
5. update stable streaming moments, and
6. write the final compact rows to resident memory or an experiment-owned temporary memmap.

Streaming variance uses a Chan/Welford merge of chunk counts, means, and second central moments. A sum/sum-of-squares formula suffers catastrophic cancellation when values have a large offset and small spread. The reference is computed from the stored float32 payload because pre-cast float64 values are not the data the geometry will encode.

Host-memory accounting includes both staged parameters and targets. A decision based only on target bytes can select a resident path that exceeds the promised RAM fraction after the parameter copy is added.

E. Temporary-file ownership

A disk-backed staged target is a temporary resource owned by the experiment. Train and validation slots are independent because validation may remain live while successive training-size sweep points restage training rows. Restaging one slot unlinks the superseded file only after no live loader needs it. Explicit release and failure cleanup remove remaining files. Process-exit cleanup is only a last defense; a long sweep cannot retain one multi-gigabyte file per completed point.

F. Hybrid inference

EMUL2 is the repository's name for the hybrid inference configuration that replaces several expensive provider products with coordinated emulator artifacts. In this EMUL2-shaped path, matter power, background history, and scalar quantities can jointly replace expensive CAMB products inside a larger likelihood. The matter-power adapter validates requirements, rebuilds the linear and boost artifacts, evaluates the current cosmology, reconstructs each physical surface, and assembles $P_{\rm nl} = P_{\rm lin}B$ when requested.

a. Current gap. The adapter's derived σ_8 helper currently combines k in Mpc^{-1} with a radius of 8 Mpc, so it calculates $\sigma(8\,\mathrm{Mpc})$ rather than the conventional $\sigma(8/h\,\mathrm{Mpc})$. Matter-power serving itself is unaffected. The adapter also advertises products through the wrong Cobaya support hook and narrows the shared $A_s/A_{s,9}$ amplitude rule. Do not request the derived scalar or treat the stubbed adapter identity as an EMUL2 dependency-resolution verdict.

b. Required closure. The radius and wavenumber units are derived together, product registration uses Cobaya's output boundary, and the amplitude requirement follows the artifact's resolved parameterization. A real Cobaya lifecycle gate requests each product and reads it through the public provider.

The request boundary validates the full k and z sequences. It also rejects unsupported holes rather than zero-filling arbitrary missing coordinates. Every calculate call checks the external provider verdict before reading payload getters, so a rejected point cannot inherit a previous surface.

G. Diagnostics and evidence

Matter-power diagnostics plot residual surfaces and slices on the persisted axes. They avoid constructing a validation-rows by neighbors by full-width cube at production scale; bounded diagnostics sample or stream the required statistics.

The identity gate proves flatten/reshape, target laws, stable moments, constant-column permission, base assembly, structured-head behavior, and temporary-file lifecycle on controlled fixtures. The smoke runs real CAMB, two trainings, public $P(k,z)$ getters, and family pages under the no-law path. Together they establish exact internal algebra and real-provider connectivity without making the expensive provider its own reference.

XXIV. DIAGNOSTICS AND PLOTTING: COMPUTE A FACT BEFORE DRAWING IT

A *diagnostic* calculates a statement about predictions. A *plot* turns already calculated values into marks, lines, colors, and labels. The distinction is architectural:

`emulator/diagnostics.py` owns estimators and their validity conditions, while `emulator/plotting.py` owns visual layout. A plotting function must not silently invent a replacement statistic when the diagnostic is undefined.

A. The diagnostic record is a typed result

For validation rows $i = 1, \ldots, N$, the shared evaluation path first obtains physical scores $q_i = \Delta\chi_i^2$. A coverage diagnostic may report fractions such as

$$f_{<t} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}[q_i < t],$$

where $\mathbf{1}[\cdot]$ is one when its condition is true and zero otherwise. The threshold t, sample count N, and side of the inequality are part of the result. A bare number such as `0.93` cannot reveal them.

A diagnostic dictionary therefore carries values together with names, units, coordinate arrays, valid-row masks, and a status. Status `available` means the estimator's mathematical preconditions held. Status `unavailable` carries a reason such as "no rows in this class" or "reference variance is zero." NaN is a floating-point value, not a complete status record: it cannot distinguish a deliberate undefined result from numerical corruption.

B. Local-linear and hard-direction questions

The local-linear diagnostic asks whether a small neighborhood can already explain the validation target. It selects nearby training rows, fits a local linear approximation, and evaluates that approximation with the same physical loss used for the emulator. The neighborhood count, distance definition, rank condition, and memory bound are estimator inputs; the neural model is not the truth source for its own floor.

The hard-direction diagnostic relates poor scores to movement along named parameter combinations. Correlation and R^2 require variation. If a feature is constant or a target class is empty, division by zero does not mean "no relationship." It means the estimator is unavailable on that sample. The result names that condition so the figure can annotate it instead of plotting a fabricated zero.

C. Family diagnostics retain their physical axes

Scalar residuals are indexed by output name. CMB residuals are indexed by spectrum and multipole. Background residuals are indexed by redshift and may include distances derived through the real integration path. Matter-power residuals are two-dimensional surfaces over (z, k). Cosmic-shear residuals retain correlation type, tomographic pair, and angle. Reducing these objects to one aggregate score is useful for model selection but insufficient for finding a localized physical failure.

Fractional error, $(\widehat{y} - y)/y$, is undefined where $y = 0$. The CMB TE spectrum can cross zero, so a diagnostic uses an explicitly named mask or an absolute-error alternative at the crossing. A finite neural prediction does not make division by a zero truth valid.

D. The plotting layer is a consumer

The page builder receives NumPy arrays on the CPU. Model evaluation occurs under `torch.no_grad()` so the report does not retain a backward graph. Tensor values are detached, moved to the CPU, and converted before Matplotlib sees them. This is an eager data transfer; the figure does not own a live GPU tensor or a lazy loader.

Axis scale is chosen from the data domain. A logarithmic ordinate requires strictly positive finite values; a series containing zero remains linear or uses a separately defined signed transformation. Labels state estimator, units, mask, and threshold. A multipage writer saves completed figure objects and closes them so long campaigns do not accumulate GUI and array state.

a. Current gap. Several diagnostic functions can currently publish NaN from empty classes, zero-variance references, constant features, an unguarded standard deviation, or a fractional residual at a truth zero. Some smoke fixtures construct finite diagnostic dictionaries by hand rather than executing those producer functions. Those tests prove plotting layout, not estimator totality.

b. Required closure. Every estimator validates finite inputs and its own denominator/rank/sample preconditions, returns value plus status/reason, and is executed directly by at least one board-listed diagnostic gate. The save-before-diagnostics order remains deliberate: a reporting failure must not destroy a successfully trained artifact, but the report must never convert corruption into a green "unavailable" result.

XXV. SAVING, REBUILDING, AND PREDICTING

Training ends with a live Python object on one machine. Inference needs a self-contained record that can rebuild the same computation later.

A. The two-file artifact

The current format uses

A `state_dict` is not a model object. It contains numerical state but not the Python constructor that gives those tensors meaning. The Hierarchical Data Format

File	Contents
`root.emul`	The model `state_dict`: parameter and registered-buffer names mapped to CPU tensors.
`root.h5`	Input/output geometry state, model recipe, resolved configuration, histories, physical metadata, and optional NPCE/transfer base.

version 5 (HDF5) model recipe supplies the class name, dimensions, and constructor options.

a. Current gap. The two current files are written independently. The HDF5 record does not carry a digest of the weight file, and publication is not a two-file transaction. Replacing one file, mixing roots, or interrupting an overwrite can therefore create a pair that exists but was never published together.

b. Required closure. The pair is staged under temporary names, validated, and published as one logical unit. Each side binds the same artifact identity and the HDF5 record binds the exact weight payload and implementation identity. Pre-existing trusted roots are refused rather than silently overwritten. Strict tensor loading remains necessary, but matching tensor names and shapes is not a substitute for pair identity.

B. Moving tensors into storage

For a model tensor w, `w.detach()` returns a tensor disconnected from gradient history. `w.cpu()` moves its storage to host memory when necessary. `w.numpy()` presents CPU tensor storage as a NumPy array when the dtype is compatible. Saving CPU tensors lets a machine without the training GPU open the artifact.

HDF5 uses groups, datasets, and attributes. A group is a directory-like container. A dataset is an array payload. An attribute is scalar metadata attached to a group or file. Nested geometry dictionaries become nested groups through a recursive writer.

C. Rebuilding the geometry class

Each geometry group stores a class string such as `emulator.geometries.parameter.ParamGeometry`. Rebuilding performs four steps:

1. split the string into module path and class name,
2. import the module,
3. retrieve the class object, and
4. call that class's `from_state` constructor.

Using the persisted class prevents a factored or logarithmic geometry from silently rebuilding as its simpler base class.

D. Rebuilding the model

The model recipe stores factory choices as names because a Python callable cannot be represented directly in YAML. Rebuilding translates those names back into activation and normalization factories, constructs the eager model, and loads the weights with `strict=True`. Strict loading rejects both missing and unexpected keys.

A compiled model wraps the eager module and may prefix state keys with `_orig_mod.`. Saving removes that wrapper prefix. Rebuilding loads the plain state first and compiles afterward when requested and supported.

E. Prediction

`EmulatorPredictor.predict` performs

$$\begin{gathered}\text{parameter dictionary}\\ \downarrow\quad \text{order by saved parameter names}\\ \Theta\ (1\times P)\\ \downarrow\quad \text{parameter geometry encode}\\ X\ (1\times P_{\mathrm{enc}})\\ \downarrow\quad \text{model in evaluation mode under no-grad}\\ \widehat{T}\\ \downarrow\quad \text{physics-aware loss decoder and output geometry}\\ \widehat{d}\ \text{in physical units.}\end{gathered}$$

Evaluation mode disables training-only behavior. `torch.no_grad()` avoids constructing a backward graph, reducing memory and latency. The artifact's parameter names, axes, units, and laws determine the result; the caller does not redeclare them.

XXVI. COBAYA ADAPTERS: PRESENTING THE EMULATOR AS A THEORY COMPONENT

Cobaya asks a theory component for named quantities at a sampled cosmology. The adapters in `cobaya_theory/` translate that lifecycle into an `EmulatorPredictor` call.

A. Lifecycle methods

The adapter is a boundary between untyped configuration values and numerical code. Boolean, integer, float, path, and list options require explicit value validation; Python truthiness and numeric coercion must not reinterpret quoted strings or NaN.

Method	Responsibility
`initialize`	Validate options, choose device, load artifacts, and cache immutable metadata.
`get_requirements`	Declare sampled or provider-supplied inputs needed by the prediction.
`must_provide`	Validate the requested output coordinates and record what the likelihood asks for.
`calculate`	Build the parameter mapping, call the predictor, validate the returned payload, and place it in the current state.
Getter methods	Return named arrays from the accepted current state.

a. Current gap. The five adapters do not yet share a total value-schema boundary. Several options are interpreted with `bool(value)`, relative roots can inherit an empty environment default, and some requested coordinates are coerced before validation. The scalar adapter also does not yet exercise its real `state[derived]` publication path in the existing identity double.

b. Required closure. One shared validator checks actual booleans, finite numeric controls, explicit path roots, exact requested coordinates, and family-compatible artifact assembly before any artifact is loaded or provider result is published. A gate calls the public Cobaya lifecycle, not merely the predictor helper, and reads the value back through the same getter a likelihood uses.

B. Requested coordinates are part of the contract

An output width is not an axis. Two multipole arrays can have equal length and different values. Two redshift grids can have equal length and different sampling. `must_provide` validates the complete requested coordinate sequence against the persisted artifact sequence before any calculation can return a value.

a. Current gap. That exact-sequence sentence is the required contract, not uniform current behavior. The CMB adapter presently checks a maximum multipole and can scatter a request containing interior holes into a zero-filled array. CMB training dumps also lack an authoritative multipole sidecar, so equal width can silently relabel shifted columns. Other family adapters have analogous range-versus-identity gaps.

b. Required closure. The generator publishes the exact coordinate sequence; staging verifies it; the artifact persists the verified sequence; and `must_provide` requires exact ordered coordinates or a separately defined subset policy. Missing interior coordinates never become physical zeros.

When several artifacts contribute blocks to one output vector, the adapter must prove that the layouts are compatible and the destination blocks do not overlap. Blind concatenation is not a layout proof.

C. State ownership

One `calculate` call corresponds to one sampled cosmology. A rejected external calculation must not expose arrays cached by an earlier cosmology. The acceptance verdict is checked before any provider getter is called, and the current state is replaced only by payloads produced for the current point.

a. Current gap. The preceding ordering is not yet uniform across generators and adapters. Several paths infer rejection by scanning text or read a provider payload before establishing the current calculation's finite verdict; a stale cached array can therefore look successful. A fake that always returns a fresh payload does not exercise this failure mode.

b. Required closure. One verdict-first helper handles the real provider return object. Rejection performs zero getters and publishes no state. Acceptance is recorded before the first getter, and a two-call stale-cache fixture tags generations so any payload inherited from the previous cosmology fails visibly.

XXVII. GATES: EXECUTABLE EVIDENCE THAT THE PIPELINE STILL MEANS WHAT IT SAYS

A gate is a test whose failure blocks acceptance of a program unit. The board is the ordered list of gates. Each entry names its dependencies, required software or hardware, owning documentation, and executable check.

A. Identity checks and smoke checks

An identity check isolates mathematics and serialization with small synthetic inputs. A smoke check runs a short real external pipeline, such as CAMB or CosmoLike, and proves that the components connect on the production path. Neither substitutes for the other.

An identity check should contain four named pieces:

1. the production function or public boundary under test,
2. a small fixture, meaning a constructed input with known properties,
3. an independently calculated expected answer, and
4. a deliberately broken form that the check must reject.

The deliberately broken form establishes catch power: the assertion is not merely true of both correct and incorrect implementations.

B. Test doubles

A test double replaces an expensive or unavailable collaborator. A stub returns predetermined values. A fake implements a small working substitute. A monkeypatch temporarily replaces the referenced function or class during a test. Every double must state which behavior it reproduces and which external behavior it cannot prove.

An independently known answer cannot be computed by the same helper that produces the value under test. Shared code would reproduce the same defect on both sides of the comparison.

C. Exit status and the board

Each check records failures and exits nonzero when any assertion fails. The board judges the subprocess exit status and required output. Printing a skip message while returning zero is not a skip; without a distinct board status it is a pass.

Board resume records are scientifically valid only for the executable surface that produced them. The present runner records status, a digest of the gate body and directly named check scripts, an input/configuration digest, and a digest of the immutable raw log. A changed gate, fixture check, board config, or YAML therefore invalidates the stored PASS automatically.

The code digest is not yet a transitive digest of every imported production module, generator, adapter, and analytic base. A production-only change can therefore leave that digest unchanged. Force-rerun remains the manual safeguard after an affected production boundary changes.

Hardware-specific PyTorch evidence belongs in a board-listed gate. CPU-only checks can prove pure validation and algebra. CUDA compilation, GPU dtype, and accelerator-specific behavior require execution on the corresponding workstation.

XXVIII. ONE COMPLETE ROW THROUGH THE PROGRAM

The entire pipeline can now be read as one sequence. Consider generated row i, containing cosmology $\boldsymbol{\theta}_i$ and target $\mathbf{d}_i$.

1. The path resolver converts YAML file names to absolute paths.
2. The experiment validates the selected family and records parameter names from the covariance header.
3. Staging opens the data-vector dump lazily, loads the parameter table, verifies row counts, applies physical cuts, and selects row i as part of a seeded subset.
4. The source dictionary records how to address row i in resident or file-backed storage.
5. The parameter geometry transforms $\boldsymbol{\theta}_i$ into whitened input x_i.
6. The output geometry and any target law transform $\mathbf{d}_i$ into target t_i.
7. A loader returns x_i and t_i on the model device as `float32` tensors.
8. The model predicts $\widehat{t}_i$.
9. The loss reconstructs the final physical prediction, forms the residual, and computes $\Delta\chi_i^2$.
10. Backward differentiation computes parameter gradients. The finite checks, optional clipping, optimizer, anchor, and moving average act in their defined order.
11. Validation scores every held-out row and selects a best epoch whose weights and metrics refer to the same state.
12. Saving writes model tensors plus the resolved recipe and geometries.
13. Rebuilding reconstructs the eager model, loads weights strictly, and restores physical metadata from the artifact.
14. The predictor accepts a named cosmology, repeats the saved encode, model, and decode chain, and returns physical units.
15. A Cobaya adapter exposes that result under the requested theory-product name and coordinate grid.
16. Identity and smoke gates prove the mathematical, serialization, and external-pipeline boundaries independently.

The network is only one step in this chain. Most silent scientific failures occur at the boundaries: a mismatched row, an unnamed axis, a scale that underflows in storage, a target law applied on only one side, a stale external provider result, or a saved recipe that does not describe the weights. The architecture matters, but the program is trustworthy only when every boundary states and checks what its numbers mean.

Appendix A: The gate board, one executable claim at a time

This appendix explains all forty gates currently registered in `gates/board.py`, in exact board order. A first-time contributor needs to know not only what a gate is called, but what production path it executes, what its fixture means, which value determines the verdict, and which plausible defect it can catch.

1. How to read each gate description

Every description below answers five questions: the claim made credible by a pass; the production path executed; the controlled fixture; the numeric, artifact, or loud-error verdict; and the deliberately wrong behavior that establishes catch power.

An identity gate usually uses a small analytic fixture and asks whether a transformation, save/rebuild cycle, or composition preserves a known value. A smoke gate runs a short production-shaped calculation and asks whether independently implemented components connect. A golden run compares selected output from the current tree with a pinned earlier tree. Golden equality is useful for an advertised no-op mode, but it cannot validate behavior that did not exist in the pinned tree.

2. The board runner as a small execution system

`gates/run_board.py` is the command-line runner. `gates/board.py` is the declarative registry. Keeping them separate means the list of scientific claims does not reimplement subprocess, path, or resume handling. A board entry supplies an identifier, title, tier, home documentation, capability requirements, dependencies, and a Python function that expresses the gate body through a constrained context object.

The context object is the gate's interface to the outside world. It can resolve a configured path, launch a check script, launch a driver, record a message, and assert an expected condition. The gate body does not call `subprocess` directly. This restriction gives the runner one place to capture command lines, standard output, standard error, return status, and elapsed time.

Standard output is the normal text stream a process prints. Standard error is a separate stream intended for diagnostics. Exit status zero conventionally means success and nonzero means failure. The runner records all three. Checking only for a phrase is insufficient if the process later exits nonzero; checking only exit zero is insufficient if the expected scientific assertion never ran.

3. Preflight protects expensive evidence

Preflight runs before GPU time. It checks the expected Git tip, watched-tree cleanliness, required imports, and configured data paths. A workstation log is evidence about one executable surface. If the worktree contains unrecorded modifications, the log cannot be reproduced from the named commit.

Import checks are capability checks, not substitutes for gate execution. Importing PyTorch proves that the module loads; it does not prove CUDA kernels, compilation, or mixed precision. Opening a Cocoa data directory proves deployment availability, not that a family adapter requests the right physical quantity.

a. Current gap. The current dirty-tree watch covers `emulator/`, `gates/`, and the root drivers. It does not yet cover `compute_data_vectors/`, `cobaya_theory/`, or `syren/`. A clean preflight therefore does not prove that every executable contributor is committed.

b. Required closure. The watched surface includes every file capable of changing the claim: production modules, drivers, theory adapters, generators, analytic bases, gate bodies, fixtures, and board configuration. A digest over only `emulator/` would miss a changed CAMB generator or weakened check. The pass belongs to the full surface.

4. Selection, tiers, dependencies, and resume

The runner can select gate identifiers, a tier, or all gates at and after a named entry. Selection is validated against the registry before execution. A misspelled `-gate`, `-from`, or `-force-rerun` identifier produces a nonzero usage error with nearby-name suggestions. The board self-test drives these public arguments and proves that an empty or dependency-skipped selection cannot report success.

A dependency states that one gate consumes an artifact or boundary produced by another. For example, adapter parity depends on successful save/rebuild. A dependency is not merely earlier list order: the runner confirms its verdict before the dependent body proceeds.

Resume skips a prior pass only when the currently implemented gate-code and input digests still match. It distinguishes stale code, stale inputs, interruption, failure, and dependency skip; only current PASS is green. Force-rerun invalidates selected records without deleting unrelated logs.

a. Current gap. The gate-code digest covers the gate function and check scripts named literally inside it, not the complete transitive imported production surface. The dirty-tree watch is likewise narrower than the full scientific surface. Consequently, a change only in an imported adapter or generator can still require an explicit force-rerun even when resume calls the stored gate current.

b. Required closure. One canonical executable-surface definition is shared by dirty preflight and resume identity. It includes every imported production module, driver, adapter, generator, analytic base, fixture, and gate body that can affect the claim.

5. Capabilities and where a claim is allowed to run

Each gate declares capabilities such as PyTorch, CosmoLike, Cobaya, and GPU. The runner checks them before the body. Capability declaration prevents a missing dependency from being mistaken for a numerical failure, but it does not turn every missing capability into a pass.

Pure validation and NumPy algebra run on CPU. A claim about CUDA compilation must run on CUDA. A

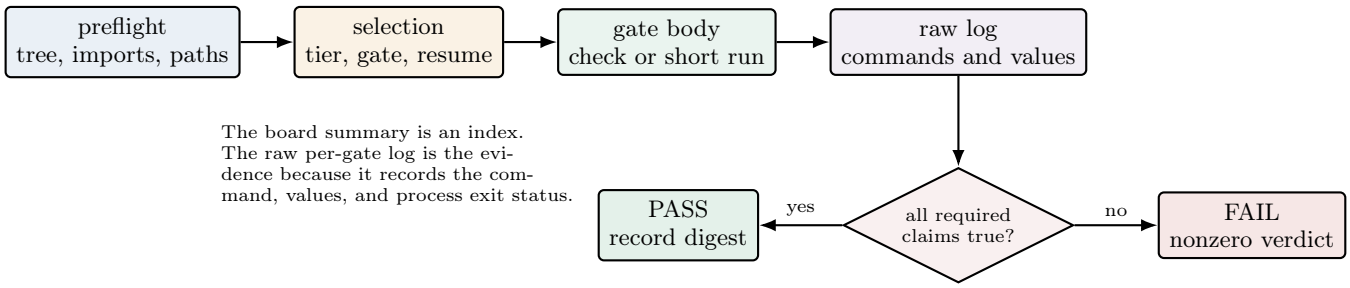

FIG. 8. The board's evidence flow. Preflight and selection determine what is eligible to run; the gate body produces evidence; the board converts all required claims into one explicit verdict.

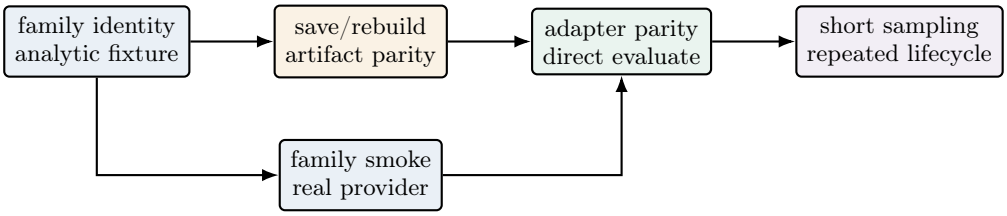

FIG. 9. A common evidence dependency. Identity establishes local mathematics, smoke establishes the real provider path, save/rebuild establishes persistence, and adapter/sampling gates establish external use.

claim about Apple MPS float16 must run on Apple hardware. When a capability is optional, the board uses a distinct non-green skip state and states what remains unproven. Catching every exception and exiting zero would erase the difference between unsupported, broken, and correct.

Hardware controls remain useful. A CPU or CUDA calculation that does not overflow records why that backend is only a control, while the affected MPS calculation supplies discrimination. The contract follows executed dtype and device, not a broad assumption that every accelerator behaves identically.

6. Anatomy of a pure numeric check

A pure check under `gates/checks` is an executable Python file with a `main` function. It imports the production function, creates a bounded fixture, calculates an independent reference, and accumulates named assertions. If any assertion fails, the process raises or exits nonzero.

The reference does not call the production helper under test. For a covariance contraction, the production side may use a banded implementation while the reference uses explicit loops and dense arithmetic on a tiny array. For a geometry, the reference may form the physical residual and direct inverse-covariance product. Agreement is valuable because the algorithms do not share the same possible mistake.

Fixtures are designed for discrimination rather than realism alone. A constant array can make correct and wrong axis orders coincide. An affine array can make several interpolation stencils exact. A strong fixture assigns distinct values to relevant coordinates and includes a control where the correct path is known to agree. The check prints actual and expected values so a red log is diagnostic rather than merely boolean.

7. Anatomy of a training smoke

A smoke YAML is small but production-shaped. It uses the real driver, resolver, loaders, model, loss, optimizer, evaluation, and saving path. Small means fewer rows or epochs; it does not mean a second miniature trainer.

A collapse bar requires the network to learn more than a dead baseline. For an analytic scalar target, the baseline may always return the training mean. The acceptance bar belongs below that baseline. Merely requiring a finite final loss allows an unchanged or disconnected network to pass.

Banners reveal resolved facts that are hard to infer from one metric: activation, normalization, phase, learning rate, scheduler cadence, and source artifact. They are readbacks, not truth sources. A behavioral leg accompanies a banner whenever the configuration might be printed but ignored.

8. Test doubles and monkeypatch scope

A stub returns predetermined values. A fake implements a small working collaborator. A monkeypatch temporarily replaces the object referenced by production

code. Replacing CAMB with a fake can prove that an adapter requests and combines keys correctly; it cannot prove CAMB returns the assumed convention.

Patch scope must match the lookup site. Replacing a class definition in one module does not affect another module that already imported and bound the old class. A gate patches the name looked up by the production function and restores it before later control arms.

A convention-honest fake makes confusable representations genuinely different. For CMB lensing it assigns different arrays to raw $C_L^{\phi\phi}$ and the scaled representation. If the fake used equal arrays, the wrong convention would pass.

9. Mutation arms establish catch power

A mutation arm deliberately reinstates one plausible defect. The unchanged gate must fail under the mutation and pass under the production path. Examples include restoring width-squared chi-square tolerance, feeding scaled lensing spectra where raw spectra are required, using the old non-Gaussian weight, and rebuilding under a drifted default.

The mutation is targeted. Changing many facts at once would not identify which assertion carries the catch power. Mutation evidence also detects a self-referential test: if actual and expected call the same helper, changing that helper changes both and the check remains green.

10. Tolerance is a derived part of the claim

Floating-point equality can be exact, absolute, relative, or combined. Bitwise equality is appropriate when saving and loading the same tensor should perform no arithmetic. Relative tolerance is appropriate when the expected magnitude varies. Absolute tolerance supplies a floor near zero.

For a sum of w float32 terms, the chi-square domain band scales with accumulation width w and machine epsilon. It does not scale with w^2, which would admit materially negative scores. The band is computed from validated shape facts, not the possibly corrupt value being screened.

Every widened tolerance needs a measured valid control and a recorded derivation. A tolerance is not adjusted merely until a red result turns green; that would let the defect define its own acceptance region.

11. Raw logs and evidence readback

Each raw log begins with gate identity, home documentation, Git commit, and capabilities. It records commands and complete output. Named lines state the value behind each check, and the final line is the gate verdict.

`logs/BOARD.md` is an index, not a replacement for evidence. A reviewer opens the raw log, confirms every advertised leg executed, and reads values near thresholds. A pass at 4.99 percent under a 5 percent bar carries different operational risk from a pass at 0.1 percent.

Readback comes from the real consumer surface. A value in an in-memory configuration does not prove the saved HDF5 record contains it. The gate reopens the artifact or reads the adapter's public result to close that loop.

12. Backlog tier: training and data contracts

a. `ema-off-identity`: the absent feature is a true no-op

Claim and path. Disabling the exponential moving average, EMA, leaves pre-EMA training byte-identical. The normal driver runs on the current tree and in a temporary worktree at the pinned pre-EMA commit. A worktree is a second checkout sharing Git objects; it does not switch the user's active checkout.

Fixture and verdict. Both runs use the short gate YAML. The scanner extracts declared epoch lines and compares their bytes. The current-tree run also exercises a functional smoke so an empty comparison cannot pass.

Catch power. Creating an averaged copy in the off branch, changing selection, consuming randomness, or changing optimizer order alters a line. This gate does not prove EMA-on behavior; the next gate owns that claim.

b. `ema-smoke`: the averaged state participates in a run

Claim and path. With EMA enabled, the epoch-based horizon is used and a plateau rewind restores coherent state. A production training runs with `ema.horizon_epochs=3`; the normal optimizer, scheduler, selection, and rewind code execute.

Fixture and verdict. The YAML causes a learning-rate cut during the short run. Banners must identify the horizon and rewind, and the process must finish with the expected metrics.

Catch power. Removing the EMA update, converting the horizon incorrectly, or rewinding live weights without their averaged partner removes a required observation.

c. `production-diagnostic`: one report crosses several boundaries

Claim and path. The production diagnostic command can stage cut data, train, and create the expected report while exposing accurate size and coverage facts. The cosmic-shear driver runs with `-diagnostic` and uses production plotting code.

Fixture and verdict. The gate checks the model-class census, rows surviving the density window, printed train/validation sizes, and shaded coverage triangle. A file suffix alone is not acceptance.

Catch power. Dead model branches, cuts applied after selection, misreported sizes, or a plot omitting the declared window breaks a leg.

d. `single-phase-demotion`: phase keys are handled deliberately

Claim and path. A single-phase `resmlp` can receive the documented phase-shaped configuration without a traceback, while a structured model retains two-phase behavior. Both pass through the real resolver and driver.

Fixture and verdict. The single-phase fixture contains head-shaped keys and must train under the declared demotion. A two-phase control must still execute separate phases.

Catch power. Forwarding a head-only option into `resmlp` reproduces the old failure. Globally deleting phase settings makes the two-phase control fail.

e. `head-scheduler-override`: the head owns its cadence

Claim and path. A head scheduler replaces the relevant top-level settings and cuts the head learning rate on its own patience. The real two-phase resolver constructs the scheduler and the epoch loop feeds its validation metric.

Fixture and verdict. The override uses patience ten. The resolved banner must name it and the cut must occur on the matching cadence.

Catch power. A shallow merge that retains top-level patience, a scheduler attached to the wrong optimizer, or a metric-free step shifts or removes the cut.

f. `eval-batch-invariance`: partitioning is not mathematics

Claim and path. Validation scores do not depend on how the same ordered rows are partitioned. The check calls real `eval_val` with several batch sizes, including its production final-batch padding path.

Fixture and verdict. Per-row scores and aggregate metrics agree at relative tolerance 10^{-6}. Timing is reported but is not a numerical verdict.

Catch power. Counting padded duplicates, leaking state between batches, batch-relative normalization, or dropping a final short batch changes at least one per-row score.

g. `finite-contract`: impossible scores never rank

Claim and path. NaN, infinity, and materially negative chi-square values are rejected at training, validation, diagnostics, and warm-start parity boundaries. The PyTorch check calls the production reduction, `eval_val`, source diagnostic, safe square roots, and parity helpers.

Fixture and verdict. Red fixtures include NaN in either parity arm, a nonfinite transfer surface, exact-fit zero, finite losses near the float32 maximum, and finite negative chi-square. Valid controls include analytic positive square roots and tiny roundoff from an ill-conditioned positive-definite contraction. The allowed roundoff band scales with kept per-row width, not width squared.

Catch power. A mutation restoring the old width-squared band crowns a materially negative production-width score as perfect. Finite-only validation lets NaN comparisons evade threshold counts. On a supported compile lane, an exception is failure rather than a green skip.

h. `board-selftest`: the runner reports what actually ran

Claim and path. The board's own selection, dependency, resume, atomic-status, and evidence-map machinery cannot manufacture a green command from an empty, skipped, stale, or interrupted run. The pure-Python check drives the real `run_board.main`, `select_gates`, and resume helpers with small fake Gate objects.

Fixture and verdict. Fake gates pass, fail, skip through a failed dependency, or raise an uncaught interruption. Unknown selectors and force targets must return nonzero with a suggestion. Mutating a gate body, referenced YAML, input config, evidence anchor, or cited log must invalidate the corresponding record. The shipped forty-entry evidence map must resolve to real note anchors with unique identifiers.

Catch power. A status-only resume rule, warning-only selector, log overwrite, or board row whose note anchor does not exist makes one of the fake scenarios falsely green and is rejected. This gate tests harness control flow; it does not prove any scientific gate body.

i. `generator-seed`: sampling owns one recorded random stream

Claim and path. Generator sampling is reproducible from one required integer seed. The seed constructs an owned NumPy `Generator` that is threaded through uniform draws, walker initialization, sampler moves, and thinning rather than reading process-global `np.random` state.

Fixture and verdict. Two runs with the same seed produce identical draws and the chain header records that seed. Invalid seed types fail before sampling. A

monkeypatch of global random functions must not affect the owned stream.

Catch power. One leftover global draw changes the same-seed result or triggers the monkeypatch. The pure gate proves ownership and local reproducibility; append/restart replay and MPI worker-count invariance require their generator smoke legs.

j. `cli-strict`: misspelled flags stop before expensive work

Claim and path. Every public executable uses strict argument parsing. The check performs a source census and drives representative driver `main` functions through their real parsers while replacing the expensive training boundary with an instrumented stub.

Fixture and verdict. A valid command reaches the stub exactly once. The misspelling `-activaton` exits nonzero, names the unrecognized argument, and reaches the stub zero times. All public entry points must use `parse_args`, not `parse_known_args`.

Catch power. Silently retaining unknown arguments or manually discarding parser leftovers lets the misspelled option train a default model; the zero-call assertion exposes that behavior.

k. `family-first`: each driver owns one data family

Claim and path. A direct driver selects one physical data-block family before experiment construction. The cosmic-shear driver cannot become a CMB, scalar, background, or matter-power run merely because the YAML contains that block, while each thin family wrapper accepts its own block.

Fixture and verdict. Wrong-family configurations fail at startup with both expected and received families. A clean cosmic-shear configuration reaches training, and the four family wrappers reach their matching boundary. A census verifies that every cosmic-shear campaign pins the same family.

Catch power. Restoring automatic family dispatch makes a wrong-driver fixture run. Pinning only the single-run driver leaves a sweep or search census failure, which is why the gate covers the driver family, not one filename.

l. `stage-ram`: the memory decision counts every copy

Claim and path. `stage_source` budgets the compact parameter copy, compact target copy, and row-reindex storage before choosing resident or file-backed staging. Parameter and target widths and dtypes are counted separately.

Fixture and verdict. A narrow target with a wide parameter table is chosen so target bytes alone fit but the coordinated copies exceed the budget. The real stage function must stay disk-backed. Resident and disk-backed controls return byte-identical selected rows, including an unequal-dtype case.

Catch power. A mutation restoring the old target-only estimate selects resident storage and fails the regime verdict even though its returned numbers remain correct. This separates resource truth from numerical truth.

m. `artifact-readback`: stored values keep their types

Claim and path. Artifact Boolean attributes are parsed by an explicit typed reader, not Python truthiness. The check exercises the shared reader and scans every artifact Boolean read site.

Fixture and verdict. A native Boolean is accepted, an absent key uses its named default, and string or integer lookalikes—including the truthy string `F̈alse¨`—raise with file and schema context. The census requires all read sites to use the shared boundary.

Catch power. Replacing the reader with `bool(value)` turns `F̈alse¨` into true and can select drifted transfer weights. This pure gate proves type parsing and source coverage; the live save/forge/rebuild artifact leg remains a separate workstation obligation.

n. `diagnostics-domain`: invalid scores do not become reports

Claim and path. Diagnostic producers apply the same finite, nonnegative chi-square boundary as training and evaluation before converting scores into coverage fractions or residual summaries. The check calls the real local-linear floor and CMB residual diagnostic and censes grid-family producer sites.

Fixture and verdict. Valid scores pass byte-identically; negative roundoff inside the width-derived band becomes exact zero; materially negative, NaN, and infinite values raise with boundary, rows, and band. A reachable one-coordinate negative floor is refused before `f_floor` is formed.

Catch power. Bypassing the screen recreates the former false `f_floor=0` "perfect" result. A valid diagnostic control prevents the repair from converting all small or difficult results into errors.

o. `berhu-loss`: the objective matches its piecewise law

Claim and path. The reverse-Huber objective has its declared value and derivative, and a head can select it independently of a square-root trunk. A pure check calls the production reduction and autograd; a short two-phase run exercises schema resolution.

Fixture and verdict. Analytic points lie below, on, and above the knot and cap. Required continuity is

checked, while banners name square root for the trunk and capped BerHu for the head.

Catch power. Using residual-norm rather than chi-square units, swapping a branch inequality, or resolving the head to the trunk loss changes an analytic value or banner.

p. `loss-schema-equivalence`*: syntax migration preserves meaning*

Claim and path. Moving loss settings into the nested schema does not change an equivalent old run. The ordinary driver resolves and trains both forms.

Fixture and verdict. Selected numerical epoch lines reproduce the pre-migration result.

Catch power. Reading a key from the wrong nesting level, falling back to a new default, or accepting conflicting spellings changes those lines. This equivalence check complements, but does not replace, the analytic loss gate.

q. `berhu-anneal`*: endpoints have mathematical meaning*

Claim and path. BerHu annealing begins as square root, stays continuous at the hold boundary, and reaches full BerHu at the declared endpoint. The shared production schedule supplies the blend coefficient.

Fixture and verdict. Values immediately around the hold boundary are compared, the epoch-15 fixture must have coefficient one, and a no-anneal control preserves absent-key behavior.

Catch power. Reversing an increasing schedule, an off-by-one boundary, or treating `const` as a generated ramp changes a sampled coefficient.

r. `ema-anneal`*: the average becomes live at the intended time*

Claim and path. EMA annealing excludes the deliberately poor early trajectory and evaluates averaged weights at the live point. The normal EMA update, shared schedule, and model-selection path execute.

Fixture and verdict. A no-anneal control is compared with an annealed run; output identifies the hold and metrics from the live averaged state.

Catch power. Updating during the hold, interpreting the horizon in the wrong unit, or evaluating raw weights under an EMA banner breaks the transition.

s. `param-window-cuts`*: selection follows physical cuts*

Claim and path. A tight density window uses named parameter columns, shrinks the candidate pool by the independent count, and then trains. The production cut resolver and staging selector operate on the real table.

Fixture and verdict. The expected surviving count is calculated independently and compared with the banner before the requested absolute number of rows is chosen.

Catch power. Cutting the wrong column, treating a missing side as zero, selecting before cutting, or silently accepting too few rows changes the count or run verdict.

t. `triangle-shading`*: visualization states the same domain*

Claim and path. The optional coverage triangle shades all declared physical windows on their corresponding panels. The CPU check calls the production plotting helper and inspects its Matplotlib artists.

Fixture and verdict. Four nondegenerate synthetic windows have known panels and extents. The test counts fill objects and verifies coordinates.

Catch power. Shading only the last window, swapping axes, or using untransformed bounds creates the wrong artist list.

13. New-features tier: model and family identities

a. `joint-training`*: a live trunk and head really train together*

Claim and path. With `freeze_trunk=false`, the second phase keeps both trunk and correction head trainable. The production phase transition sets `requires_grad`, creates optimizer groups, and runs the joint graph.

Fixture and verdict. The banner states joint training and the check compares continuity across the phase boundary. Phase-two epoch time must sit above a frozen-trunk control because backward now traverses the trunk as well as the head; timing is a supporting liveness signal, not the only proof.

Catch power. Leaving trunk parameters frozen, omitting them from the optimizer, or evaluating a detached trunk can still let the head improve. Trainable-parameter counts and the control comparison expose those partial implementations.

b. `head-activation-pin`*: one head may own a different curve*

Claim and path. A structured head can pin a `gated_power` activation while the trunk retains its model-wide family, and illegal phase combinations are refused before training.

Fixture and verdict. The resolved model-spec banner names the head family and gate count. The parameter census must increase by the analytically expected

activation tensors. A configuration that pins a head activation while requesting incompatible joint behavior must raise the teaching error.

Catch power. A resolver that prints the pin but still supplies the trunk factory to the head fails the parameter-count leg. Silently ignoring the illegal combination fails the error leg. This gate establishes construction; activation liveness is additionally covered by behavioral head checks.

c. `relu-tanh-norm`: *fixed baselines use the requested normalization*

Claim and path. Parameter-free ReLU and tanh activations can be constructed through the same factory interface, and tanh runs with either shared affine or per-feature normalization as declared.

Fixture and verdict. Short tanh runs require the resolved norm in the banner and a descending loss. An absent-key control reproduces the default normalization behavior.

Catch power. Passing the feature width into `nn.Tanh`, sharing one mutable normalization instance across layers, or silently selecting the default instead of `per_feature` changes construction, parameter count, or banner. Loss descent prevents a build-only false green, though separate head gates are needed for zero-initialization compatibility.

d. `weight-decay-census`: *decay follows module role*

Claim and path. AdamW decay applies exactly to weight matrices owned by `Linear`, `Conv1d`, and `BinLinear`; it excludes biases, normalization gains, and activation-shape parameters regardless of tensor shape.

Fixture and verdict. A model containing gated-power activations and structured layers is walked through the production optimizer factory. The check compares parameter object identities in the decay group with an independent owner-role census. A zero-decay golden control protects the off-mode behavior.

Catch power. The tempting shortcut `parameter.ndim > 1` incorrectly classifies some learned tensors and can miss custom weight owners. Because the expected set is named by owning modules, a shape-based mutation fails even if the total number of elements happens to match.

e. `npce-training`: *the analytic base and neural correction compose*

Claim and path. Neural polynomial-chaos emulation, NPCE, trains in both residual and ratio forms, persists enough state to rebuild, rejects mutually exclusive configurations, and refits its base independently at each training-size sweep point.

Fixture and verdict. Short runs exercise both combination laws. Error fixtures request incompatible PCE/transfer or composition choices. A two-point N_{train} sweep verifies that each point receives a base fit from its own selected rows rather than reusing the first point's coefficients.

Catch power. A module-global PCE, a cached term selection shared across points, or a rebuild that restores only the neural residual changes the second point or round-trip. Successful training alone would not prove the base was refit.

f. `finetune-identity`: *warm start begins as the source function*

Claim and path. Fine-tuning validates the source artifact, extends input geometry only according to the declared rule, pins the output geometry, and produces epoch-zero predictions equal to the source before any optimizer step.

Fixture and verdict. The pure PyTorch check covers equal parameter sets, allowed extensions, anchor masks, constant or degenerate columns, and configuration refusals. It compares both eager source predictions and the newly constructed training-side path at epoch zero.

Catch power. Recomputing the output center from new targets, mapping an extended input into the wrong column, loading weights non-strictly, or applying an anchor before parity changes the epoch-zero result. Loud-error legs ensure an unsupported mismatch cannot be mislabeled as numerical drift.

g. `transfer-identity`: *a zero correction is exactly the base*

Claim and path. Frozen-base transfer slices the inputs required by the source, evaluates the source in its own encoded space, initializes a zero correction, and composes gain or sum forms without altering the epoch-zero function.

Fixture and verdict. The check exercises several combinations of base-space and final-space composition, target packing, parameter surgery, and family-kind refusals. A grid-family arm passes through its logarithmic law rather than assuming all outputs are plain vectors.

Catch power. Applying the correction in the wrong coordinate space, double-encoding base inputs, unpacking the staged target at the wrong offset, or testing a family mismatch only after another invalid fixture condition changes parity or the expected error.

h. `scalar-identity`*: a named scalar survives every boundary*

Claim and path. Scalar geometry, model state, save/rebuild, predictor output, and automatic Cobaya provision preserve named derived quantities exactly.

Fixture and verdict. The check uses analytic scalar targets and exercises subsets, duplicate names, constant columns, wrong artifact kinds, trunk-only architecture constraints, and fine-tune parity. Bitwise equality is required where no floating-point recomputation is warranted.

Catch power. Width-only assembly can return the right number of scalars in the wrong order. The named subset and duplicate-name legs catch that class, while constant-column and wrong-kind legs prevent a finite-looking but undefined geometry from passing.

i. `cmb-identity`*: spectra, amplitude law, and covariance agree*

Claim and path. The CMB family preserves spectrum conventions, applies its amplitude/optical-depth law on the correct side of the metric, rebuilds exactly, evaluates roughness as specified, supports its structured head, and constructs non-Gaussian band covariance weights correctly.

Fixture and verdict. The check covers ruled per-ℓ constants, forward/inverse amplitude laws, physical-χ^2 invariance, required parameters, roughness zero and high-frequency controls, fine-tune parity, and a ResTRF identity-basis rebuild. Five covariance legs compare the production contraction with an independent known answer: exact contraction, deliberate old-weight miss, raw-versus-scaled lensing fixture integrity, width-three band projection, and exact zero-band weight.

Catch power. Feeding scaled $[L(L+1)]^2 C_L^{\phi\phi}/(2\pi)$ where raw $C_L^{\phi\phi}$ is required creates a large known miss while the raw control matches. A round-trip-only test could not expose a consistently wrong convention; the independent contraction does.

j. `bsn-identity`*: background functions obey their trained domain*

Claim and path. Background-grid geometry, logarithmic-offset law, piecewise redshift windows, loud desert between windows, save/rebuild, adapter getters, and fine-tune parity agree with closed-form flat-ΛCDM fixtures.

Fixture and verdict. Analytic $H(z)$ values span each trained window and approach its boundaries from both sides. Requests in the untrained gap must raise instead of interpolating across it. Derived distance relations are compared with independently integrated values.

Catch power. A generic interpolator can return smooth, positive, and monotonic values in an untrained region. Those properties are insufficient; the explicit desert leg makes that plausible extrapolation red. The save/rebuild arm catches a law or window omitted from state.

k. `mps-identity`*: a two-dimensional surface keeps its coordinates*

Claim and path. Grid2d staging, geometry laws, constant-column pinning, matter-power base assembly, fine-tune parity, structured correction heads, stable streaming moments, and staging-file lifecycle all preserve the declared (z, k) surface.

Fixture and verdict. Stub bases make the assembly algebra exactly known. Law-space rows are compared with independently transformed values. Stable-moment fixtures contain large offsets and float32 payloads, and several chunkings must reproduce the stored-payload population variance. Lifecycle legs restage train and validation slots, verify superseded temporary files are unlinked, and check cleanup after failure and sweep-point release.

Catch power. The retired sum/sum-of-squares variance changes with chunk order under cancellation. A precast float64 reference disagrees with the actual float32 payload. A gate that checked only `isinstance(memmap)` would miss leaked multi-gigabyte files; absence checks after restage provide the discriminating evidence.

l. `geo-paths`*: geometry classes have one import home*

Claim and path. New artifacts persist class paths under the `emulator.geometries` package, legacy flat paths fail loudly, and no live source import retains the retired geometry homes.

Fixture and verdict. The check creates fresh states, inspects their class markers, attempts legacy reconstruction, and performs an untruncated tree census for old imports.

Catch power. Compatibility shims can make a local rebuild pass while allowing new files to continue publishing obsolete paths. Requiring both the new marker and the loud old-path refusal prevents that half-migration.

14. Save-and-sample tier: artifacts and external execution

a. `save-rebuild-drift`*: an artifact is an executable recipe*

Claim and path. Saving and rebuilding produces a bitwise-equal function for plain, factored, NPCE, and

convolutional-head models, and the artifact remains governed by its persisted recipe when source-code defaults later drift.

Fixture and verdict. The check trains or constructs each supported form, saves the weight/HDF5 pair, rebuilds through `rebuild_emulator`, and compares predictions before and after. For a structured head, persisted bin splits must reconstruct the same scatter layout. The check temporarily perturbs a code default and proves that the stored resolved value wins. Version-one and pre-persistence head artifacts must be refused with migration guidance.

Catch power. A weight-only test can pass while rebuilding a different geometry or layout. A current-default test can pass until the default changes. The deliberate drift arm proves that artifact facts, not ambient source defaults, own reconstruction.

b. `cobaya-adapter`: *inference repeats the training-side function*

Claim and path. The Cobaya theory adapter reads named sampled parameters, calls the rebuilt predictor, and returns the same physical prediction as the training-side model, including factored recombination.

Fixture and verdict. A parity check evaluates the same off-center cosmology through both paths and requires relative agreement at 10^{-6}. A Cobaya `evaluate` call then verifies the public lifecycle and a short MCMC smoke confirms repeated calls under the sampler.

Catch power. Reordered parameters, a missing decode, a factor law applied only in training, or stale adapter state changes parity. The MCMC arm adds lifecycle evidence that one direct call cannot provide, while the direct known-input comparison supplies the precise numeric verdict.

c. `finetune-smoke`: *a warm start can improve and republish*

Claim and path. A real fine-tune begins at the source function, continues training on new data, writes source provenance, and saves an artifact that rebuilds to the final fine-tuned function.

Fixture and verdict. The run consumes the board's own previously saved emulator. Before the first update, it prints and passes epoch-zero parity. It then completes, improves the declared metric, writes `finetuned_from`, and survives save/rebuild.

Catch power. Loading source weights but recomputing incompatible geometry can still lead to eventual loss descent; the pre-update parity makes that path red immediately. Writing provenance without embedding the facts needed for reconstruction fails the final round-trip.

d. `transfer-smoke`: *a frozen base and correction complete a run*

Claim and path. A real transfer configuration loads the board's base artifact, composes a zero-start correction in the declared gain form, preserves epoch-zero parity, trains, and saves a self-contained transfer artifact.

Fixture and verdict. The banner names the base path and composition form. The pre-train parity line must pass, training must complete under its collapse bar, and the saved HDF5 record must contain transfer provenance plus the embedded base facts needed for rebuild.

Catch power. Depending on the external base file after publication, packing the base and truth in the wrong order, or composing in the wrong space may still produce a trainable correction. Parity and isolated rebuild separate those defects from ordinary optimization error.

e. `scalar-smoke`: *a complete analytic scalar example*

Claim and path. The scalar pipeline can learn a derived quantity whose formula is independently known, predict away from the training center, serve it through the scalar Cobaya adapter, and generate its diagnostics.

Fixture and verdict. The target is analytic $\Omega_m h^2$. A short training must beat a collapse bar chosen below the mean-predictor baseline. An off-center cosmology avoids the degenerate case where every implementation agrees at the center. Cobaya `evaluate` readback is compared with the formula, and expected diagnostic pages must be created.

Catch power. Returning the center, swapping parameter columns, or advertising the scalar without inserting its calculated value can look plausible on a centered fixture. The off-center analytic readback makes each wrong path visible.

f. `cmb-smoke`: *real CAMB reaches training and inference*

Claim and path. The CMB generator, covariance script, training driver, artifact rebuild, Cobaya C_ℓ lifecycle, and family diagnostics connect on real CAMB output. The non-Gaussian covariance path also executes with its normalization facts persisted.

Fixture and verdict. A smoke-scale CAMB run generates spectra. The covariance output is checked structurally, including non-diagonal blocks and provenance keys. Training must cross a relative collapse bar, the adapter must return the requested spectra, and diagnostic pages must complete.

Catch power. This gate detects interface and convention failures that a synthetic fake cannot: CAMB key names, raw versus scaled spectra, actual coordinate ranges, covariance file shape, and adapter lifecycle. It does not use its own non-Gaussian output as numerical

truth; the independent `cmb-identity` contraction owns that proof.

g. `bsn-smoke`: background emulators agree with CAMB

Claim and path. The background generator, two family trainings, saved predictors, Cobaya getters, derived-distance calculations, and grid diagnostics work end to end against CAMB's own background solution.

Fixture and verdict. Real CAMB supplies $H(z)$ and reference distances at smoke-scale redshifts. A stale-cache tripwire ensures each generated row belongs to the current cosmology. The served values must agree within the declared two-percent smoke tolerance, and the grid diagnostic pages must render.

Catch power. A generator can return a finite array cached from the previous point, and a smooth interpolator can make it look reasonable. The generation token/tripwire and independent CAMB comparison jointly expose that class. Exact formula and desert-boundary behavior remain in the identity gate.

h. `mps-smoke`: the matter-power surface completes EMUL2-shaped service

Claim and path. The matter-power generator, two trainings, grid2d artifacts, Cobaya `get_Pk` lifecycle, and residual-surface diagnostics connect on real CAMB data under the no-analytic-law path.

Fixture and verdict. CAMB produces a smoke-scale $P(k, z)$ surface on declared coordinates. Both trained pieces save and rebuild, the adapter returns surfaces on the requested grid, and values are compared with the real reference within the smoke contract. The family pages must finish without materializing the production-size diagnostic cube.

Catch power. Transposing z and k, relabeling an equal-width axis, using the training axes for validation, or assembling two artifacts in the wrong law space changes the surface. Stubbed identity legs prove exact assembly mathematics; this smoke proves the real CAMB and Cobaya interfaces.

15. Gate-by-gate failure triage

The first response to a red gate is to locate which named leg failed, not to raise a tolerance or rerun blindly. The following guide identifies the first evidence to inspect for every board entry.

a. `ema-off-identity`. Diff the first unequal epoch line and determine whether the difference begins before model construction, at optimizer setup, or at evaluation. A changed random draw suggests the off path consumed state. A changed metric with identical early banners suggests an off-mode branch entered the training or selection calculation.

b. `ema-smoke`. Read the resolved horizon and executed steps per epoch, then locate the learning-rate cut and rewind lines. If the cut exists but rewind is absent, inspect scheduler-to-rewind control flow. If rewind exists but later metrics jump, compare which of live weights, optimizer moments, and averaged weights were restored.

c. `production-diagnostic`. Separate a training failure from a report failure. Verify the cut-pool and size banners before opening the plotting traceback. If training succeeded but a page is absent, inspect the first diagnostic function that did not emit its completion marker rather than assuming the PDF writer failed globally.

d. `single-phase-demotion`. Inspect the resolved architecture before the exception. A constructor receiving head keys indicates demotion happened too late. If the single-phase arm passes and the structured control fails, the resolver likely deleted phase information globally rather than conditionally.

e. `head-scheduler-override`. Compare the printed head patience with the epoch of the first cut. Correct printing with wrong cadence points to scheduler construction or metric stepping. Wrong printing points earlier, to precedence resolution. Confirm the scheduler holds the head optimizer object, not the trunk optimizer.

f. `eval-batch-invariance`. Find the first row whose score changes and whether it lies in the final batch. A difference only at the end implicates padding or trimming. Differences at every boundary suggest batch-relative state or reduction. Compare per-row arrays before comparing rounded medians.

g. `finite-contract`. Use the failed part label to distinguish producer, boundary, accumulation, tolerance, parity, and compile arms. Print the offending row count and minimum. Do not weaken the predicate until the valid ill-conditioned control and the production-width negative mutation have been evaluated together.

h. `board-selftest`. Classify the failure as selection, dependency, resume digest, interruption, log atomicity, or evidence-map integrity. Inspect the fake gate's call count before the stored status: a body that did not run can never supply scientific evidence. For digest failures, change one input at a time and confirm which stale category the runner reports.

i. `generator-seed`. Compare the first differing draw and identify which sampling stage owns it. A difference before the sampler implicates uniform or walker initialization; a later difference implicates moves or thinning. Search for process-global random calls only after confirming the seed reached the owned generator.

j. `cli-strict`. Record parser exit status and expensive-boundary call count together. A nonzero traceback after the stub ran is too late; the typo must be rejected by argument parsing. A source-census failure names the executable that still accepts unknown flags.

k. `family-first`. Read the requested driver family beside the YAML's detected data block. If a wrong-

family run reaches experiment construction, the driver omitted its pin. If a correct thin wrapper fails, inspect only its explicit family argument before changing shared dispatch.

l. *stage-ram.* Recalculate parameter bytes, target bytes, index bytes, and budget separately. If values agree but the regime does not, the defect is accounting rather than staging algebra. Compare resident and disk row coordinates before comparing payloads.

m. *artifact-readback.* Print the stored HDF5 type and value before conversion. A string that looks like a Boolean is invalid, not a spelling to coerce. If the pure reader passes but a live rebuild fails, locate the read site that bypassed it.

n. *diagnostics-domain.* Identify the producer and its declared reduction width. Compare the raw score with the derived band before any plotting or availability logic. A within- band negative should normalize to zero; a material negative or nonfinite value should never reach a statistic.

o. *berhu-loss.* Identify the analytic region containing the first mismatch. A value mismatch at a knot suggests branch selection; a value match with gradient mismatch suggests a non-differentiable expression or misplaced detach. If only the training smoke fails, inspect loss schema resolution rather than the formula.

p. *loss-schema-equivalence.* Diff resolved configurations before diffing epochs. The first differing leaf usually identifies an inherited default or wrong nesting level. If resolved records agree but epochs differ, look for a code path that still reads raw configuration after resolution.

q. *berhu-anneal.* Print the coefficient on the last hold epoch, first anneal epoch, and final epoch. This distinguishes direction, boundary, and endpoint errors. Then verify the loss uses that coefficient; a correct banner with unchanged analytic values means the schedule is disconnected.

r. *ema-anneal.* Inspect whether an averaged copy was updated during hold and which weights evaluation used at the live point. If schedule values are correct but metrics match raw weights, follow the evaluation model selection rather than changing the horizon formula.

s. *param-window-cuts.* Resolve parameter names to column indices and independently recount each bound. A disagreement before combination indicates one formula or column; a disagreement only after combination indicates mask composition. Confirm the absolute row count is applied to the surviving pool.

t. *triangle-shading.* List artist types, panel coordinates, and bounds. A correct count with wrong panels is an axis-map failure. A missing fill with correct numeric masks is a plot ownership failure and does not invalidate the staging cut itself.

u. *joint-training.* Print trainable names and optimizer membership at phase entry. A parameter can have `requires_grad=True` yet remain absent from the optimizer. After one batch, compare trunk and head gradients and actual parameter deltas, not only epoch loss.

v. *head-activation-pin.* Compare the resolved head factory, instantiated class names, and activation parameter count. If the banner is correct but count is not, the pin was recorded without construction. If the illegal configuration trains, the validator was bypassed or ran after allocation.

w. *relu-tanh-norm.* Inspect each residual block rather than only the model's first norm. Shared object identity across layers means a factory returned one instance. For a finite but flat tanh run, inspect input range and normalization gains before assuming optimizer failure.

x. *weight-decay-census.* Report missing and unexpected parameter names with owning module types. A rank-based pattern appears as activation or normalization tensors in the unexpected set. A custom linear owner in the missing set means the role registry is incomplete.

y. *npce-training.* First identify residual versus ratio and the training-size point. If only the second point fails, compare PCE coefficient identities and selected term sets between points. If rebuild fails, inspect persisted domain, multi-indices, coefficients, and combination law separately.

z. *finetune-identity.* Compare source and new named parameter orders, then locate the first unequal stage: encoded input, first projection, network output, or decode. An output difference with equal network coordinates implicates geometry. A difference at the first projection implicates extension or weight surgery.

aa. *transfer-identity.* Print base input slices, base-space output, correction, and final composition in that order. This exposes double encoding and wrong-space composition. For an unexpected error message, verify the fixture is invalid only in the dimension under test.

bb. *scalar-identity.* Read named outputs beside their columns. Equal width is not enough. For a constant-column failure, inspect the stored dtype after scale construction; a positive float64 scale may underflow to zero in float32.

cc. *cmb-identity.* Identify spectrum, multipole, and representation before examining tolerance. For covariance failures, compare raw fixture arrays, independent per-band accumulation, and production weights. For amplitude failures, divide the reported metric by the expected physical factor to expose an accidental multiplicative law.

dd. *bsn-identity.* Classify the red point as trained window, exact boundary, or desert. A finite desert return is a domain-policy failure even if monotonic. A trained-window numeric mismatch should be traced through output law, interpolation, and distance conversion in that order.

ee. *mps-identity.* Separate coordinate, law, moment, assembly, and temporary-file legs. For moment

failure, record chunk order and compare against the stored float32 payload, not pre-cast data. For cleanup failure, list live train and validation slots and which restaging event should have unlinked each path.

ff. `geo-paths`. Inspect the class marker in a newly written state before chasing imports. If the marker is current but the census is red, a live source still depends on a retired home. If a legacy path loads, locate the compatibility shim that made the required loud refusal impossible.

gg. `save-rebuild-drift`. Compare predictions at four boundaries: pre-save eager, rebuilt eager, rebuilt under drifted defaults, and family public predictor. Then compare state keys, shapes, and resolved construction facts. Non-strict loading is never a repair for an unexplained missing key.

hh. `cobaya-adapter`. Print named sampled values in artifact order and compare the predictor result before adapter assembly. If direct parity passes but repeated sampling fails, inspect per-call state reset and provider verdict order. If parity itself fails, inspect units, axes, and decode before MCMC.

ii. `finetune-smoke`. Treat failed epoch-zero parity as a construction failure and stop before interpreting later improvement. If parity passes but save/rebuild fails, compare fine-tune provenance and inherited architecture facts. If training does not improve, inspect learning rate and trainable census.

jj. `transfer-smoke`. Verify the base digest and family identity, then epoch-zero parity, then correction learning. A missing embedded base is a publication failure even when the run succeeds locally with the external source still present.

kk. `scalar-smoke`. Evaluate the analytic formula at the exact off-center input printed in the log. If direct prediction agrees but Cobaya readback does not, inspect automatic provides and state insertion. If both return the center, inspect input ordering and encoding.

ll. `cmb-smoke`. Follow generator, covariance, training, rebuild, adapter, and diagnostics markers in order. The first absent marker locates the interface boundary. Use `cmb-identity`, not the smoke's generated covariance, to adjudicate a normalization disagreement.

mm. `bsn-smoke`. Check the stale-cache tripwire before comparing CAMB values. A tripwire failure invalidates every later numeric agreement. For a getter mismatch, compare requested redshift sequence with artifact axes before inspecting interpolation.

nn. `mps-smoke`. Confirm the stored z and k sequences on both artifacts and the request before comparing flattened arrays. A transpose can preserve shape and summary statistics. If coordinates agree, inspect law-space assembly and only then the real-provider tolerance.

16. Worked example: building a finite-contract gate

Consider the claim that validation must refuse a finite negative $\Delta\chi^2$. First identify the public boundary. The relevant function is `eval_val`, because it creates per-row validation scores and ranks epochs. Calling only a private predicate would not prove that `eval_val` uses it.

The fixture uses a small test double returning a score vector with a negative value well outside the derived roundoff band. All other inputs are finite and validly shaped. This isolates one invalid property: an exception cannot be misattributed to NaN input, shape mismatch, or an absent model parameter.

The expected result is a typed exception naming the validation side, affected row count, minimum, and allowed band. A low-level square-root warning does not satisfy the gate because it occurs after the program failed to classify its own scientific input.

Next comes a near-zero valid control. A small positive-definite covariance produces a tiny negative roundoff under finite precision. The shared predicate normalizes a value inside the derived band to exact zero. This proves the gate does not replace false acceptance with blanket refusal.

The mutation restores the retired width-squared band. At realistic output width, the chosen negative score then falls inside the wrong band and becomes zero. The mutation must make the gate red. Finally, the same predicate is exercised through training reduction and source diagnostics. One helper owns the rule, while integration legs prove every public producer calls it:

$$\begin{aligned}\text{public boundary} &\to \text{red fixture} \to \text{typed verdict}\\ &\to \text{valid control} \to \text{mutation}\\ &\to \text{boundary census.}\end{aligned}$$

17. Worked example: identity and smoke for the CMB family

The CMB family shows why one end-to-end test is not enough. The smoke can run CAMB, write spectra, construct a covariance, train, and serve C_ℓ. If every stage uses the same wrong lensing convention, the path can be internally consistent and scientifically wrong.

The identity fixture makes raw and scaled lensing spectra different. Let the fake raw spectrum be nonconstant $C_L^{\phi\phi}$. Independently form

$$\widetilde{C}_L^{\phi\phi} = \frac{[L(L+1)]^2}{2\pi} C_L^{\phi\phi}.$$

The fake returns each only under its proper request. Production receives the raw array. A separate explicit loop computes the expected band contribution. Agreement establishes the correct arm.

For catch power, feed the scaled array into the raw slot without changing the reference. The band weight

must miss by a large recorded margin. This is stronger than asserting that the arrays differ: it proves the final contraction is sensitive to the convention.

The real-CAMB smoke answers different questions. Does the generator request available keys? Are lengths and multipole sidecars consistent? Can training consume the covariance? Can the artifact answer Cobaya's requested sequence? Does the non-Gaussian configuration execute rather than fall back?

The logs meet through persisted provenance. The smoke asserts which path ran. The identity asserts that this path has the right mathematics. Neither uses the other as a numeric truth source.

18. Worked example: artifact drift

An artifact combines tensors with instructions for interpreting them. A drift test changes an ambient code default after writing the artifact. Suppose a model was saved with activation H, three gates, and affine normalization. The check records an off-center prediction y_{before}.

The source default is monkeypatched to another allowed setting. Public rebuild must read the persisted resolved configuration and produce y_{after}. Because this fixture requires no new arithmetic, the comparison can be bitwise:

$$y_{\text{after}} = y_{\text{before}}.$$

The off-center input matters. At zero input or identity initialization, several architectures coincide and a drifted build can pass accidentally. The check also inspects class, parameter names, output geometry, and head layout. Prediction equality for one row can be coincidental; state identity plus several discriminating predictions localizes failure.

An old artifact missing a required construction fact is refused with a migration instruction. Guessing from current defaults makes the file load today and change meaning tomorrow. Loud refusal is correct when reconstruction cannot be proven.

19. How to add a gate without creating a false green

Start with one sentence that can be false. For example: *the rebuilt predictor returns the same named spectrum on the same multipole sequence.* Avoid a broad goal such as *test CMB*. A narrow claim determines the public boundary, fixture, and verdict.

Then follow this construction order.

1. Name the production owner and call its public function. If a private helper also needs a unit check, retain the public integration leg.
2. Build the smallest fixture preserving the distinction. Use nonconstant, nonsymmetric values when order matters.
3. Derive the expected answer independently. Write down units, axis, dtype, device, and shape before calculating it.
4. Add a valid edge control: exact zero, one row, final short batch, or an ill-conditioned positive-definite matrix, depending on the claim.
5. Add a mutation or deliberately wrong form and verify the unchanged assertion fails it.
6. Give every failure a typed or named verdict. Printing a warning while exiting zero is not a red leg.
7. Declare hardware and external capabilities. Do not hide an unavailable required lane inside a broad exception handler.
8. Register the gate with home documentation and actual artifact dependencies.
9. Run the check directly and through the board. Direct success plus board failure usually indicates deployment, path, capture, or capability metadata.
10. Temporarily break the intended production line and observe the board turn red before accepting the new gate.

20. Common false-green patterns

a. Expected and actual share one helper. The test repeats the defect. Replace the expected side with explicit algebra, a closed-form solution, or a differently structured reference.

b. The fixture erases the distinction. Zeros, constants, identity matrices, and centered cosmologies make wrong paths agree. Keep them as edge controls, not the only fixture.

c. Any exception counts as success. An always-raising implementation passes. Require the documented exception type and message facts, then include a healthy in-range control.

d. A banner substitutes for behavior. The resolver can print a setting that construction ignores. Pair the banner with parameter counts, state readback, a numerical transition, or a changed weight.

e. Only aggregates are compared. Permuted or transposed rows can preserve a mean. Compare per-row and per-coordinate arrays before reducing.

f. A skip exits zero. The board records a pass without evidence. Use a distinct skip status that cannot contribute to a full-board denominator.

g. Tolerance follows the failure. The defect defines its own acceptance. Derive tolerance from dtype, operation count, and valid controls before looking at the red value.

h. The smoke is its own reference. Internal consistency replaces truth. Pair the smoke with an identity or an independent external implementation.

21. A compact evidence checklist

For each raw log, a reviewer asks:

1. Does the header identify the exact commit and watched surface?
2. Did every promised leg print a start and verdict?
3. Does the production public boundary appear in execution?
4. Are actual, expected, tolerance, dtype, device, shape, and coordinates visible?
5. Is the valid control close for the stated reason?
6. Does the mutation fail through the intended assertion?
7. Are skips excluded from the pass count?
8. Does artifact or adapter readback show the consumer sees the same resolved fact?
9. Is a thin margin to threshold recorded explicitly?
10. Can the evidence be reproduced without untracked workstation state?

22. Reading one registry entry field by field

A `Gate` entry is data describing execution. Its `id` is the stable command-line name and log-file stem. Changing an identifier breaks resume continuity and external run instructions, so titles may improve while identifiers remain stable.

The `title` is human-facing. It states the claim in ordinary language. The `tier` controls grouping and scheduling, not pass semantics. The `home` points to the maintained design note that owns the acceptance contract. The optional map text gives a reviewer a more precise route into that note, but executable assertions remain the verdict.

The `run` field holds a function object. Passing a function as a value does not execute it. The runner calls it later with the context for the current log, configuration, capabilities, and dry-run state. This delayed call is why constructing the board at import time need not launch tests.

`needs` is a tuple of capability names. A tuple is an immutable ordered Python sequence; the runner iterates it before the gate body. `deps` is a separate tuple of gate identifiers whose accepted products or boundaries are required. Capability availability and scientific dependency are not the same relationship.

The optional golden-worktree commit identifies a comparison tree for a no-change claim. It is not a general expected-answer source. If a feature is new, an older commit lacks the behavior and cannot adjudicate its mathematics. The gate instead uses analytic, mutation, or real-provider evidence.

Inside the body, `ctx.run_check(path)` launches a repository check script and returns its exit status and captured output. Returning values does not raise automatically; the body passes the status into `ctx.expect`. `ctx.expect` records a named check and changes the gate verdict when its boolean condition is false. The label names the scientific claim, not merely the Python comparison.

Dry-run mode follows the same registry and dependency traversal but prints commands without executing them. A gate body must not interpret absent dry-run output as failed science. Conversely, dry-run success is never stored as a gate pass because no assertion values were produced.

23. Choosing the right evidence form

a. Use exact identity when no arithmetic should occur. Examples include loading the same tensor bytes, an off mode that should not enter a branch, and a zero correction whose composition is algebraically the base. Exact comparison gives the narrowest contract and catches accidental casts or reordered operations.

b. Use a tolerance when the correct path performs floating-point work. The tolerance is derived from dtype and operation structure. Record absolute and relative parts explicitly. Compare in the dtype in which the verdict is defined; upcasting after a float32 computation cannot recover lost information.

c. Use a golden run for a promised no-op migration. Schema rearrangement and disabled optional features are good candidates. Pin the base commit and select stable, science-bearing output lines. Do not use a golden run to freeze a known defect or to replace a new feature's reference.

d. Use an analytic fixture for a mathematical law. Closed-form scalar outputs, polynomial surfaces, diagonal covariance, and simple distance relations make good fixtures. Exercise off-center and nonconstant values so competing formulas do not coincide.

e. Use a fake for an external protocol. A fake can record requested keys, enforce call order, and return generation-tagged arrays. It proves the code handles the protocol it was given. Follow with a real-provider smoke to show the protocol matches the installed collaborator.

f. Use a smoke for production connectivity and liveness. A smoke should cross the same public entry points used in production and beat a baseline that a dead component cannot beat. Keep it short by reducing rows and epochs, not by removing the component under test.

g. Use mutation when a plausible wrong path also looks reasonable. Axis relabeling, stale cache, raw/scaled convention, tolerance inflation, and configuration printing without construction can all yield finite outputs. A targeted mutation proves which assertion distinguishes them.

24. Why gate names pair identity and smoke

The family pairs are an intentional reading aid: `scalar-identity`/`scalar-smoke`, `cmb-identity`/`cmb-smoke`, `bsn-identity`/`bsn-smoke`, and `mps-identity`/`mps-smoke`. The shared prefix says the gates refer to one physical family. The suffix says which evidence layer they own.

Warm-start and transfer use the same pattern. Their identity gates establish epoch-zero composition and error paths without external training cost. Their smokes consume a saved artifact and prove improvement, provenance, and final rebuild. If the identity is red, running the smoke spends accelerator time on an undefined starting function. If identity is green but smoke is red, the failure lies in training, persistence, deployment, or the real data path rather than the core algebra already isolated.

Some cross-cutting features instead use an off-identity/on-smoke pair. EMA first proves that absence preserves old behavior, then proves the new averaged state is live. These are not interchangeable naming conventions; the name states the two claims that the feature actually needs.

The board order places cheap, pure identities before dependent expensive smokes where practical. This is fail-fast scheduling: discover a local algebra or schema failure before allocating a workstation for an end-to-end run. Order never allows a smoke pass to override a failed identity. Both remain separate verdicts in the final board.

25. From a red log to a bounded repair

A repair begins at the first failed invariant, not at the last traceback. Reproduce the smallest failing gate directly. Preserve the original raw log, then instrument values without changing the acceptance predicate. Once the mechanism is understood, state a bounded implementation contract and add the counterexample that current code gets wrong.

The implementer changes the production owner and the gate together only when the existing gate lacks the required distinction. A gate must not be relaxed to match the implementation. If a previously accepted behavior was itself wrong, the record states that the acceptance is reopened and replaces its reference with independently justified truth.

After the repair, run the direct gate, its mutation arm, dependent family identity, and the relevant smoke. Then run the complete board according to the unit's risk. The landing record names the exact commit and raw logs. A reviewer can therefore reconstruct this chain:

$$\begin{aligned}\text{counterexample} &\to \text{root cause}\\ &\to \text{bounded change}\\ &\to \text{discriminating leg}\\ &\to \text{regression evidence.}\end{aligned}$$

This discipline keeps a red result useful. The goal is not to make the board green as quickly as possible; it is to make the scientific claim true and leave executable evidence that the same failure class will be caught later.

26. Mapping gates to the pipeline boundaries

The board can also be read horizontally, by the boundary each gate protects. At the configuration boundary, `single-phase-demotion`, `head-scheduler-override`, `loss-schema-equivalence`, `head-activation-pin`, `relu-tanh-norm`, `cli-strict`, and `family-first` prove that commands and text become the intended constructible objects for the intended physical family. Their common risk is a value that is accepted or printed but does not control construction.

At the generation and data-selection boundary, `generator-seed`, `stage-ram`, `param-window-cuts`, `triangle-shading`, and `production-diagnostic` connect reproducible cosmology draws, physical support, staged rows, resource policy, and the human coverage report. The plot gate does not establish row selection by itself, but it ensures the report teaches the same domain that selection enforced.

At the objective boundary, `berhu-loss`, `berhu-anneal`, `finite-contract`, and `eval-batch-invariance` protect the meaning of a score. `diagnostics-domain` extends the same boundary to post-training estimators. They distinguish the per-row physical metric from its training transform, and they ensure batching, invalid values, or schedule position cannot redefine which epoch looks best.

At the optimizer-state boundary, `ema-off-identity`, `ema-smoke`, `ema-anneal`, `weight-decay-census`, `joint-training`, and `head-scheduler-override` establish which parameters move, which auxiliary state advances, and which metric controls the learning rate. This group is about state transitions, not only tensor formulas.

At the representation boundary, the scalar, CMB, background, and matter-power identity gates protect names, axes, units, geometry laws, and family-specific assembly. Their smoke partners then carry the verified representation into the real physics provider. `geo-paths`

protects the Python class identity that reconstructs those representations.

At the reuse boundary, NPCE, fine-tune, and transfer gates establish how an existing function participates in a new model. Identity gates prove the initial composition. Smokes prove that the composed model trains and publishes. Their central invariant is that reuse starts from a known function rather than merely from loadable tensors.

At the publication boundary, `save-rebuild-drift` proves that the saved recipe owns reconstruction, `artifact-readback` protects typed metadata, and `cobaya-adapter` proves that the external consumer sees the same function. Family smokes add repeated lifecycle calls. These gates close the final gap between an in-memory result and a theory product used by inference.

Finally, `board-selftest` protects the evidence-control boundary itself: selection, dependency, status, digest, log, and note-map machinery. It cannot make a scientific claim true, but it prevents the harness from reporting a claim as executed when it was empty, skipped, stale, or interrupted.

This horizontal reading reveals coverage gaps. A new component that changes configuration, representation, training state, and publication needs evidence at each affected boundary. Adding one end-to-end smoke at the last boundary does not automatically cover the earlier three. Conversely, a pure identity at construction does not show the component is reachable from the public driver. The gate plan is complete when every changed arrow in the ownership chain has a named executable claim.

27. Running the board and preserving the evidence

The workstation operator begins in an environment where Cocoa, PyTorch, and the relevant external packages are importable. The first command is

```
python gates/run_board.py --check
```

This performs preflight without launching GPU work. A clean preflight states which repository commit, deployment configuration, watched paths, imports, and data roots will govern the run. If any fact is wrong, fix the deployment and run preflight again; do not treat an intended different tree as close enough.

Next,

```
python gates/run_board.py --dry-run
```

prints the selected order, dependencies, capabilities, and commands. Dry run is the place to catch a wrong YAML path or accidentally broad selection. Because it produces no scientific results, it never writes pass verdicts.

The full command

```
python gates/run_board.py
```

streams each command into its per-gate raw log. The terminal output is useful for monitoring, but the log file is the durable record. If the process is interrupted, completed matching passes remain eligible for resume and the in-flight gate runs again.

For a bounded investigation, `-gate` selects named gates and `-from` selects the suffix beginning at one gate. `-tier` selects a registry tier. Force-rerun applies only to explicitly named matching gates unless the all-selected form is requested. Before trusting the plan, the operator reads the printed selected count and identifiers; zero selected gates is an error rather than a successful no-op.

After the run, begin with the summary to locate red or skipped entries, then open every corresponding raw log. For a full acceptance run, sample green logs as well, especially thin-margin and hardware-specific gates. Confirm the last gate verdict, but also confirm every advertised leg appears above it. A gate body that returned early could otherwise produce an incomplete log whose last executed check happened to be green.

Logs are committed only after the operator verifies that they name the intended commit and contain no unrelated workstation secrets. The log commit is evidence metadata; it does not alter the source commit that was tested. The landing record names both. If source changes after the run, the old logs remain historical evidence and a new run is required for the new executable identity.

When a gate needs CUDA or another workstation-only capability, the repository maintainer supplies the check under `gates/checks`, registers it in the board, and gives the operator an exact selection command. The operator is not asked to paste an untracked Python snippet into a GPU shell. Keeping the check in the repository makes its source part of the watched evidence surface and lets later board runs repeat it.

The final handoff reports counts only after excluding skips and unexecuted gates. On the board described by this appendix, a phrase such as 40/40 means forty registered required gates executed and passed on the recorded surface. If the environment lacks a required lane, the honest result is fewer certified claims plus an explicit outstanding capability, not a smaller denominator silently presented as a complete board.

28. What a full green board does and does not mean

A green board means that every registered executable claim passed on the recorded source tree, configuration, dependencies, and hardware. It is not a proof that unregistered states are correct. The board therefore persists the Git commit and should bind resume records to a digest of the complete executable surface: production modules, gate bodies, fixtures, and relevant configuration.

The strongest pattern is paired evidence:

1. an identity fixture proves an exact mathematical or serialization statement;

2. a mutation or deliberately wrong form proves the assertion can fail;

3. a smoke run proves the real external components reach the verified boundary; and

4. artifact readback proves the published object carries the facts the executed path used.

When only one layer is available, the log should say exactly what remains unproven. A CPU identity cannot certify CUDA compilation. A CUDA smoke cannot establish an independent analytic truth if its reference repeats the production helper. A PDF existing cannot prove its plotted coordinates. The board is useful because those claims stay separate and named.

Appendix B: A file-by-file route for studying the code

This appendix is a reading itinerary. It is not alphabetical. Each file is placed after the concepts needed to understand it and before the files that consume it. The reader should keep one example YAML, one parameter table, and one small artifact in view while following the route.

1. How to use the route

For each file, answer four questions before moving on:

1. What object enters the file's public function?

2. What object leaves, including shape, dtype, device, and ownership?

3. Which physical or lifecycle invariant does the file own?

4. Which later file consumes the returned object?

Do not begin by reading every helper from top to bottom. Locate the public entry named below, follow one call path, and return for variants after the baseline path is clear.

2. Stage 0: orientation before Python

a. 1. Root `README.md`. Read the pipeline diagram, one-run command, YAML anatomy, and the definitions of data vector, whitening, and chi-square. Stop when the command's three path arguments and the two top-level YAML blocks have distinct meanings.

b. 2. `emulator/README.md`. Read the layout and five-family table. Use its function census as an index, not as the first explanation. Stop when each physical family can be mapped to one geometry, one loss, and one Cobaya adapter.

c. 3. One file under `example_yamls/`. Choose the simplest `resmlp` training example for the family of interest. Trace indentation and write the dotted path of each model/loss control. This concrete recipe will anchor later configuration code.

3. Stage 1: the shortest complete training path

a. 4. `emulator/cocoa.py`. Start with the path resolver. Follow how `-root`, `-fileroot`, and `-yaml` become concrete locations. The file owns project layout, not numerical cosmology. Next read the experiment constructor that consumes those paths.

b. 5. `emulator/experiment.py`, *first visit.* Read only `EmulatorExperiment.from_yaml` and `EmulatorExperiment.from_config`. Follow family dispatch until the cosmic-shear or selected-family branch has named its source files. Defer the large validators and grid2d staging details. Stop when the order *validate, stage, build geometry, build specs, train* is visible.

c. 6. `emulator/data_staging.py`. Read `load_source`, `stage_source`, and the source-dictionary docstrings. Draw resident and memmap cases separately. Track `idx` and `dump_rows`; one is the coordinate used to index the staged object and the other retains original disk-row identity for sibling files. Then read `phys_cut_idx`, streamed statistics, and scalar named-column loading.

d. 7. `emulator/geometries/parameter.py`. Read `ParamGeometry.from_covmat`, `encode`, `decode`, and `state`. Write the center, rotation, and scale operations as equations. Confirm which arrays are NumPy during fitting and which become device tensors during loading. Defer log and amplitude-factor subclasses until the baseline round trip is clear.

e. 8. `emulator/geometries/output.py`. Read `DataVectorGeometry` in the order squeeze, center, whiten, unwhiten, unsqueeze. Identify the mask and covariance as physical owners. Then read `DiagonalGeometry`, `BlockDiagonalGeometry`, and `build_shear_angle_map` as alternative coordinate structures.

f. 9. `emulator/losses/core.py`. Read `CosmolikeChi2.chi2`, `encode`, `decode`, and `_reduce`. Separate the physical per-row statistic from the training transform. Then read `anneal_value` and `make_chi2`. Stop when a loss object can be described as geometry plus composition plus reduction.

g. 10. `emulator/designs/blocks.py`. Read `Affine`, normalization construction, and one `ResBlock` forward pass. Name every registered parameter and buffer. Then read `BinLinear`, `TRFBlock`, and

`FiLMGenerator`; these are components used by structured heads, not complete emulators.

h. 11. ***emulator/designs/plain.py.*** Begin with `ResMLP`: entry projection, residual-block list, exit projection, and final affine map. Follow tensor shapes through `forward`. Only then read `ResCNN` and `ResTRF`, focusing on scatter, validity mask, correction, gate, and inverse gather.

i. 12. ***emulator/activations.py.*** Now the `act(dim)` factory has context. Read baseline H, then multi-gate, power, and gated-power forwards. For multi-gate, write the shape created by `unsqueeze(-2)` and the axis removed by `sum(-2)`. Finish with `make_activation`, which returns factories rather than shared instances.

j. 13. ***emulator/batching.py.*** Read memory estimators before loader construction. Follow `_build_loaders_one` once for device-resident, host-stream, and memmap-stream regimes. The returned closures have the same interface while retaining different storage objects.

k. 14. ***emulator/training.py,*** *first visit.* Read `make_model`, `make_optimizer`, and `make_scheduler`; note which computed values the factories inject. Then follow one epoch through `training_loop_batched`: clear gradients, load, forward, loss, finite checks, backward, unscale/clip, step, anchor, EMA, and metric accumulation. Finish with `eval_val` and best-state selection.

l. 15. ***emulator/results.py.*** Read `save_emulator` beside `rebuild_emulator`. For every written field, find its readback. Distinguish weight state from the constructor recipe and geometry state. Do not infer transactional integrity from the existence of two successful writes; the current pair is not yet digest-bound.

m. 16. ***emulator/inference.py.*** Read `EmulatorPredictor.__init__`, then one `predict` call. Verify that saved parameter names define input order and that the training-side decoder is reused. The baseline path is now complete from YAML to physical prediction.

4. Stage 2: learn the other geometry families

a. 17. ***emulator/geometries/scalar.py.*** Study named output columns, per-output center/scale, batch-by-output shape, and the constant-column guard. Pair it immediately with `emulator/losses/scalar.py`, whose `ScalarChi2` also wraps the grid families because their physical law lives inside their geometry.

b. 18. ***emulator/geometries/cmb.py.*** Track spectrum name, multipole sequence, units, fiducial values, and per-multipole scales. Read `attach_head_coords` to see how one spectrum becomes a structured-head coordinate block. Next read `emulator/losses/cmb.py`: amplitude-law registry, factored target, roughness, and the plain/factored loss factory.

c. 19. ***emulator/geometries/grid.py.*** Read the target-law registry before `GridGeometry`. Follow `log_offset` through encode and decode, and identify the persisted redshift/quantity/unit facts. Then read `emulator/background.py` in the order cumulative integration, comoving distance, and interpolators.

d. 20. ***emulator/geometries/grid2d.py.*** Write down the flattening rule $j = i_z N_k + i_k$. Follow law-space standardization, stored thinned k, constant-mask handling, and head coordinates. Then read `emulator/syren_base.py`, which maps resolved cosmological parameters into the analytic base consumed by generator and adapter.

e. 21. ***emulator/analytics.py.*** Read this optional cosmic-shear preprocessing only after ordinary geometry and loss are understood. Track exactly which broadband analytic factor is divided out and where it is multiplied back.

5. Stage 3: design variants and warm starts

a. 22. ***emulator/designs/ia.py.*** Start with the template count and output shape. Verify that intrinsic-alignment amplitudes do not enter the template network. Pair the design with `emulator/losses/ia.py`; read NLA/TATT coefficient builders before the template-combine loss.

b. 23. ***emulator/designs/pce.py.*** Read `pce_multi_index`, `pce_design`, and `select_lars_loo` in that order. Then follow `PCEEmulator.from_training`: box map, Legendre matrix, target SVD, per-mode selection, and frozen buffers. Finally read `emulator/losses/pce.py` to see cached base output packed beside truth and combined with the neural refiner.

c. 24. ***emulator/warmstart.py.*** Read source resolution and validation first. Then read recipe reconstruction, input-geometry extension, weight transfer, output pinning, and epoch-zero parity. Read anchor-mask machinery as a mapping of named source/new parameters; remember that the public fine-tune anchor key is currently refused until its queued contract lands.

d. 25. ***emulator/losses/transfer.py.*** Follow frozen base, correction, composition form, and composition space. Read cosmic-shear and diagonal-family variants separately. Identify which tensor blocks the packed target contains and which owner defines the split.

e. 26. ***emulator/experiment.py,*** *second visit.* Return for `build_geometry`, `build_specs`, fine-tune/transfer branches, grid2d law staging, two-phase resolution, and every family validator. The earlier files now make this large orchestration module readable.

6. Stage 4: campaigns, reports, and publication

a. 27. ***emulator/family_drivers.py.*** Read sweep-block parsing and dotted-path assignment. Then inspect one root-level train driver, one N_{train} sweep, one one-knob sweep, and the Optuna driver. Family-specific driver files are thin wrappers; verify what they pin before reading duplicated command-line setup.

b. 28. ***emulator/scheduling.py.*** Read assignment functions, worker entry/exit protocol, then VRAM-token packing. Distinguish job scheduling from model-distributed training: one job still owns one complete model and device.

c. 29. ***emulator/diagnostics.py.*** Start with coverage, local-linear floor, and hard-direction functions. For each, list return keys and undefined-data policy. Then read the CMB, scalar, grid, and grid2d diagnostics that produce physical arrays for plotting.

d. 30. ***emulator/plotting.py.*** Read public plotting functions after diagnostics so data preparation and rendering do not blur together. Trace one returned diagnostic dictionary into one page builder and the multipage writer.

e. 31. ***emulator/results.py,*** *second visit.* Return for learning-curve and sweep tables, histories, provenance, PCE/transfer groups, and family flags. Compare every persisted fact with the campaign or adapter that later relies on it.

7. Stage 5: generation and external inference

a. 32. Shared generator core. Open `compute_data_vectors/generator_core.py`. Read command parsing, parameter sampling, master/worker distribution, checkpoint load/save, and failure masks. Mark the current manifest and failure-exit gaps before assuming a dump is training-ready.

b. 33. Family generator files. Read `dataset_generator_lensing.py`, `dataset_generator_cmb.py`, `dataset_generator_background.py`, and `dataset_generator_mps.py` as thin physics adapters over the shared core. For each, record provider requirements, payload shape, dtype, axis sidecars, and output file names. Read `compute_cmb_covariance.py` after the CMB generator; it produces the separate scale/covariance input consumed by CMB geometry.

c. 34. Cobaya theory adapters. Read `emul_cosmic_shear.py` first for the simplest predictor wrapper. Then read `emul_scalars.py`, `emul_cmb.py`, `emul_baosn.py`, and `emul_mps.py`. In each, trace `initialize`, `get_requirements`, `must_provide`, `calculate`, and getters. Compare requested coordinates with artifact coordinates and identify per-call state replacement.

8. Stage 6: executable evidence

a. 35. ***gates/README.md.*** Learn runner vocabulary, tiers, logs, resume, and workstation commands. Treat its gate table as navigation into the detailed Appendix A.

b. 36. ***gates/board.py.*** Read the `Gate` record and shared helper functions, then one pure identity gate and one smoke gate. Follow capability/dependency metadata into the ordered board registry.

c. 37. ***gates/checks/*.py.*** Read the check matching the feature just studied. Locate production import, fixture, independent reference, valid control, mutation/catch-power arm, and nonzero failure exit. Family identity checks precede family smokes.

d. 38. ***gates/run_board.py.*** Read this last. The scientific claims already live in `board.py`; the runner owns preflight, selection, subprocess capture, logs, and resume. Compare current behavior with the current-gap notes in Appendix A.

9. Three shorter routes

a. To change a model. Read `designs/blocks.py` → `designs/plain.py` or `designs/ia.py` → `activations.py` → model construction in `training.py` → save/rebuild in `results.py` → the matching identity gate.

b. To change a physical family. Read its generator → geometry → loss → experiment family branch → predictor branch → Cobaya adapter → identity and smoke gates.

c. To change training behavior. Read config validation in `training.py` → the executed epoch loop → histories in `results.py` → campaign driver/scheduling if parallel → finite, evaluation, and feature-specific gates.